What is a network?

clients, servers and the conversation between them

Watch the 8-min video

In one line: a network links devices so they can send messages; a client asks with a request, a server does the work and answers with a response, and every round trip costs time.

WORDS YOU'LL MEET TODAY

- **Network**: devices linked so they can send messages
- **Client**: the one who asks (your phone's app, a browser)
- **Request**: the question the client sends
- **Round trip**: a request going there + its response coming back
- **Timeout**: "I waited too long": the client gives up
- **Internet**: a network of networks
- **Server**: the one who answers: a computer that waits for requests
- **Response**: the answer that comes back
- **Millisecond (ms)**: a thousandth of a second
- **Retry**: send the same request again

1 The question: who talked to whom?

You open the Flipkart app, type *running shoes* and tap Search. Half a second later, a page full of shoes appears. But those shoes were never on your phone. Something far away sent them. This lesson follows that conversation, one step at a time.

A **network** is devices linked so they can send messages to each other. Your home Wi-Fi is a small one: it links your phone, laptop and TV. The **internet** is many networks joined together, a network of networks; a message to a computer in another city passes through several of them.

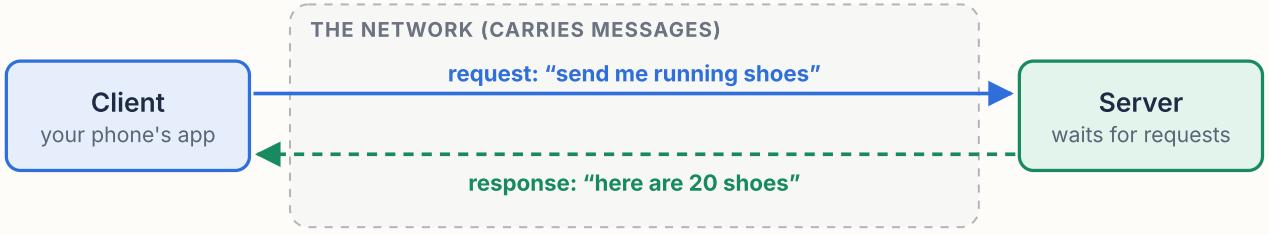
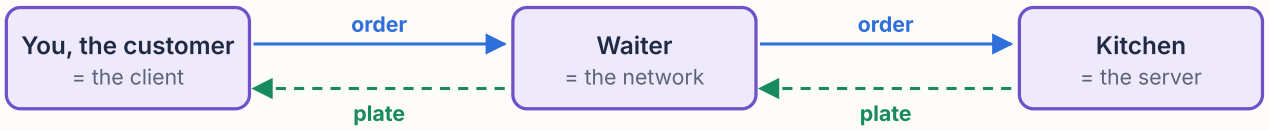


Figure 1. A network lets devices send messages. The client asks with a request; the server answers with a response.

2 An everyday picture first: a restaurant

You sit at a table, hungry. You don't walk into the kitchen; you call the waiter and order one masala dosa. The waiter carries the order to the kitchen, the kitchen cooks, and the waiter carries the plate back. You asked, and you got an answer.

- **You always speak first.** The kitchen never sends food nobody ordered.
- **One kitchen cooks for many tables** at the same time.



Order = request · plate of food = response

Rule 1: you always speak first. The kitchen never sends food nobody ordered.

Rule 2: one kitchen cooks for many tables at the same time (one server, many clients).

The waiter carries your order but never cooks: the network carries messages, it doesn't answer them.

Figure 2. The everyday picture: customer, waiter and kitchen are client, network and server.

3 Now the real thing

In the restaurant	In a network
You, the customer	the **client**: the Flipkart app on your phone
The kitchen	the **server**: a computer in a data centre that waits for requests
Your order	the **request**
The plate of food	the **response**
The waiter, carrying both ways	the **network**

The same rules hold: the client always speaks first, and one server answers many clients at once, thousands of phones in the same second.

4 Step by step: one request

1. **Ask.** You tap Search. The app writes a request: send me shoes that match *running shoes*. Its job is to say exactly what you want.
2. **Travel.** The request goes from your phone, over Wi-Fi or mobile data, through several networks, to Flipkart's server. The network carries the message; it doesn't answer it.
3. **Work.** The server finds matching shoes, with prices and photos. The thinking happens here, not on your phone.
4. **Answer.** The server sends a response: here are twenty shoes. It travels back across the networks.
5. **Show.** The app reads the response and draws it on your screen. One request, one response: a conversation of two messages.

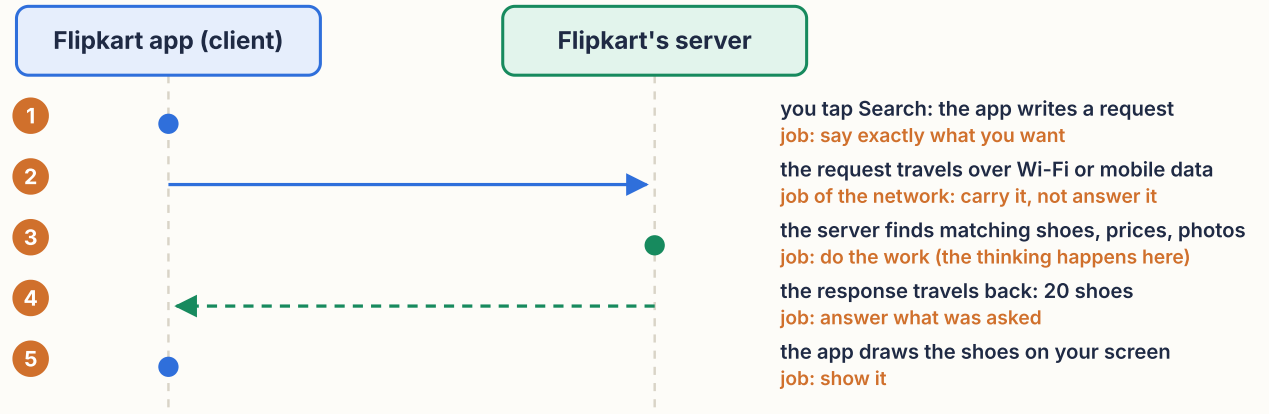


Figure 3. One request, step by step. Each step has a job: ask, carry, work, answer, show.

5 Real numbers: round trips add up

The time for a request to go and its response to come back is a **round trip**. On a mobile network, **50 ms** is common (a millisecond is a thousandth of a second). A shopping page is not one response: a **3 MB** page is made of about **40 files** (the page itself, photos, styles and code), and each file is its own request and response.

How the files are fetched	Arithmetic	Waiting on round trips
one after another	40 × 50 ms	**2,000 ms = 2 s**
six at a time	40 ÷ 6 ≈ 7 rounds; 7 × 50 ms	**350 ms**

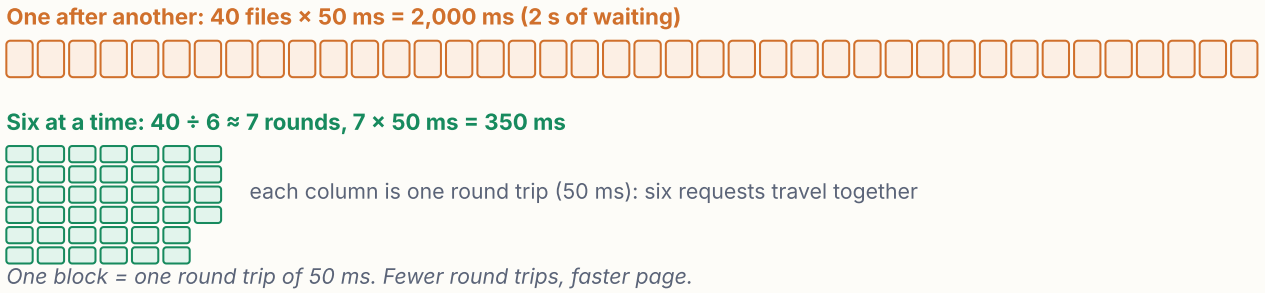


Figure 4. Real numbers: a 3 MB page of about 40 files, at 50 ms per round trip.

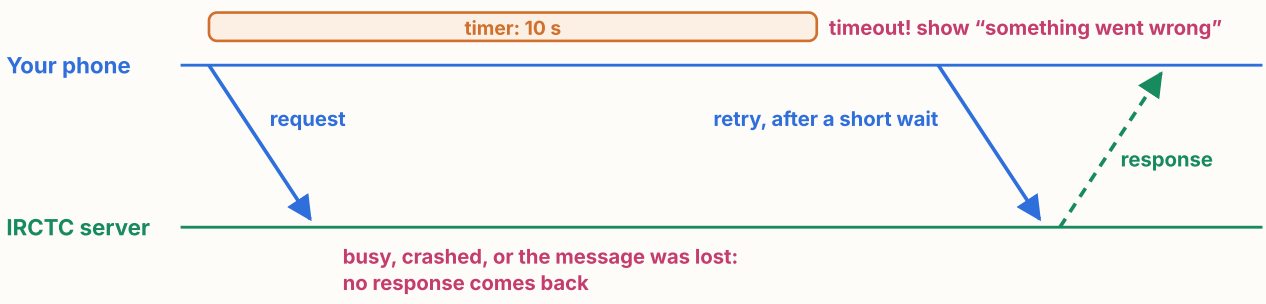
For the curious. Round trips are only half the cost; moving the bytes takes time too. 3 MB is 24 megabits (1 byte = 8 bits), so on a 20 megabit-per-second connection the data alone takes 24 ÷ 20 = 1.2 s. Browsers really do fetch several files at once (HTTP/1.1 browsers open about six connections per site; HTTP/2, lesson 23, sends many requests over one connection).

6 What goes wrong: no answer

At 10 in the morning Tatkal booking opens on IRCTC, and lakhs of phones send requests in the same minute. A busy server may answer slowly, or answer "I'm busy, try again later". Worse, sometimes no answer comes at all: the server may have crashed, or a message was lost on the way.

The client can't wait forever. It starts a timer; if no response arrives in, say, 10 seconds, it gives up. That is a **timeout**, and the app shows "something went wrong, please try again".

- **Timeouts**, so the app never hangs forever.
- **Retries after a short wait**, so a busy server isn't hit even harder.
- **Safe retries.** Asking twice for a train list is harmless; paying twice is not. (How systems prevent double payments comes later in the course.)
- **More servers** that share the requests, like a restaurant opening more kitchens.



A retry must be safe: asking twice for a train list is harmless, paying twice is not.

Figure 5. What goes wrong: no response comes. The client's timer runs out (a timeout), and it retries safely after a short wait.

PAUSE AND THINK

You send “hi” to a friend on WhatsApp. Who is the client? Who is the server? And is your friend's phone a server? Cover the answer and decide.

Answer. Your phone is a client: it sends a request to WhatsApp's server. The server holds the message and delivers it to your friend's phone, which is a client too. The two phones don't talk directly; they talk through the server.

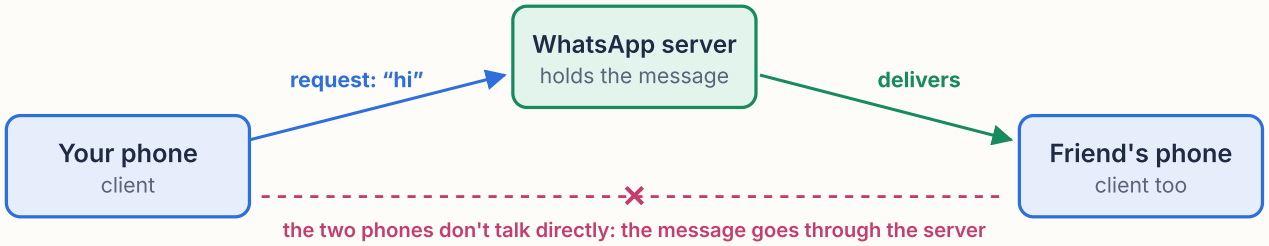


Figure 6. The pause-and-think answer: both phones are clients; the WhatsApp server sits in the middle.

TWO COMMON MISTAKES

- **“A server is a special, giant machine.”** No. A server is any computer running a program that waits for requests; even your laptop can be one. Client and server are roles: who asks, and who answers.
- **“The network understands your message.”** No. It only carries it, like a waiter who carries your order but never cooks. The meaning lives at the two ends, the client and the server.

RECAP IN THREE LINES

1. A network links devices so they can send messages.
2. A client sends a request; a server does the work and sends back a response.
3. Every round trip costs time, and when no answer comes, a timeout and a safe retry keep things moving.

7 Interview questions: what to say in 30 seconds

Q1. What is a network, and what are a client and a server? BASIC

A network is devices connected so they can exchange messages. In the client-server model the client starts the conversation with a request, and the server, a program that waits for requests, does the work and sends back a response.

Q2. Is a server a special kind of machine? BASIC

No. A server is a role: any computer running a program that waits for requests. A laptop can be a server. Client and server describe who asks and who answers, not the hardware.

Q3. A page needs 40 files and a round trip is 50 ms. How long does it wait if it fetches them one by one, or six at a time? SCENARIO

One by one: 40 × 50 ms = 2,000 ms, two seconds of waiting before moving any data. Six at a time: about 7 rounds, 7 × 50 = 350 ms. That's why designs try to cut round trips.

Q4. Tatkal booking opens and the server is overloaded. What should the client do? SCENARIO

Never wait forever: use a timeout, then retry after a short, growing wait so the busy server isn't hit even harder, and only retry requests that are safe to repeat. On the server side, add servers to share the load.

Q5. When you message a friend on WhatsApp, is your friend's phone the server? TRICKY

No. Both phones are clients. Your phone sends the message to WhatsApp's server, which holds it and delivers it to your friend's phone. The phones don't talk directly.

Q6. Why must retries be safe? TRICKY

Because a timeout doesn't tell you whether the server did the work. The request may have succeeded and only the response was lost. Asking twice for a train list is harmless; paying twice is not, so such requests need protection against duplicates.

Read more. RFC 1122 (requirements for internet hosts) and MDN's "How the web works". The full written lesson is at backendroadmap.com.

Next: SD-N-002 · Packets: why data travels in small pieces backendroadmap.com

Packets

why data travels in small pieces

Watch the 8-min video

In one line: data crosses the network in small packets, each a header (addresses and a number) plus a payload; the MTU, usually 1,500 bytes, sets the size, and the numbers let the receiver reassemble everything and ask again for just the missing piece.

WORDS YOU'LL MEET TODAY

- **Packet**: one small piece of data that travels on its own
- **Payload**: the data inside the packet
- **Reassemble**: put the pieces back together, in order
- **Router**: a machine that passes packets along toward their destination
- **Header**: the label on a packet: where from, where to, its number
- **MTU (maximum transmission unit)**: the largest packet size; 1,500 bytes on most networks
- **Byte**: a tiny unit of data, about one letter of text
- **Megabyte (MB)**: about a million bytes; a phone photo is about 3 MB

1 The question: why pieces?

You're on a train and a friend sends you a photo on WhatsApp. The download circle creeps round, bit by bit: the photo is not arriving all at once, it is arriving in pieces. On the way one piece is lost and two arrive in the wrong order, and yet the photo that opens is perfect. Why cut a photo into pieces at all, and how does your phone put them back together?

One unit first: a **byte** is a tiny unit of data, about one letter of text. A phone photo is around **3 MB**, about three million bytes.



Figure 1. A photo travels as packets: cut into pieces, each labelled with a header, sent on its own, and reassembled by number at the other end.

2 An everyday picture first: moving house

You're moving from Bengaluru to Pune, and your sofa is too big for the courier, whose rule is: nothing bigger than one box. So you take the sofa apart and pack it into 20 boxes, each with a label: from your old flat, to your new flat, box 3 of 20. The boxes travel in different vans on different roads and arrive out of order; box 9 may come before box 2. At the new flat you read the labels, sort the boxes 1 to 20 and put the sofa back together. If box 7 never arrives, you see the gap and ask for box 7 again: only box 7, not the whole sofa.



Boxes go in different vans and arrive out of order (box 9 before box 2).

At the new flat: read the labels, sort 1 to 20, rebuild the sofa (reassemble).

Box 7 missing? You see the gap and ask for box 7 again: only box 7, not the whole sofa.

Sofa = your photo · box = packet · label = header · contents = payload · courier's size rule = MTU

Figure 2. The everyday picture: moving a sofa as 20 numbered boxes.

3 Now the real thing

Moving house	In a network
The sofa	your photo (any data: a message, a web page)
One box	a **packet**: one small piece of the photo
The courier's size rule	the **MTU**: the largest packet size
The label on the box	the **header**: where it comes from, where it's going, its number
What's inside the box	the **payload**: a piece of the photo
Vans and roads	the network
You, sorting boxes at the new flat	the receiver, which **reassembles** the photo

4 Step by step: one photo

1. **Cut.** The sender's phone cuts the photo into pieces, each small enough to fit the size limit. Job: make every piece fit.
2. **Label.** Each piece gets a header in front: from this phone, to the WhatsApp server, and a number. Job: let each piece travel on its own and still find its place.
3. **Send.** The packets leave one after another and each makes its own way across the networks; some take a longer road. The network's job is to carry each packet, nothing more.
4. **Reassemble.** At the other end the packets are sorted by number and joined back into the photo. Job: rebuild the exact original, byte for byte.

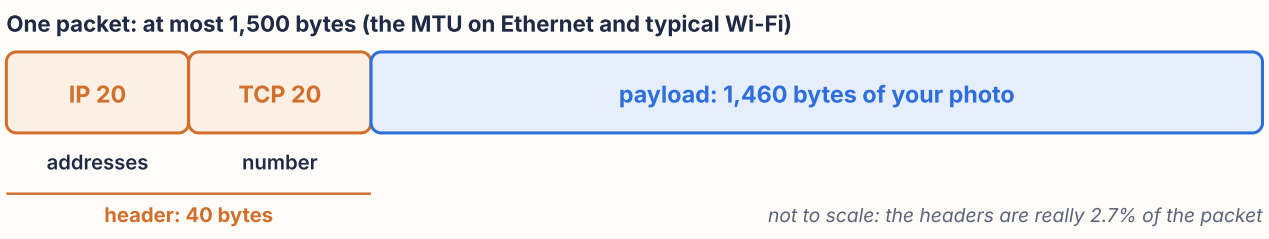


Figure 3. Inside one packet: a 40-byte header (20 bytes IP, 20 bytes TCP) and up to 1,460 bytes of payload.

5 Real numbers

On most networks, including Ethernet and typical Wi-Fi, the largest packet size, the **MTU**, is **1,500 bytes**. Not all of it is photo: 20 bytes are the **IP header** (the addresses, lesson 3) and 20 bytes the **TCP header** (the numbering, lesson 8), which leaves **1,460 bytes** of payload.

Question	Arithmetic	Answer
payload per packet	1,500 − 20 − 20	**1,460 bytes**
packets for a 3 MB photo	3,000,000 ÷ 1,460	**≈ 2,055 packets**
cost of the headers	40 ÷ 1,500	**≈ 2.7%** (under 3%)
resend after one loss	one packet's payload	**1,460 bytes**, not 3,000,000
time for one packet on 10 Mbit/s	1,500 × 8 bits ÷ 10,000,000 bits/s	**1.2 ms**
the photo as one piece on 10 Mbit/s	3,000,000 × 8 ÷ 10,000,000	**2.4 s**

So why pay for the labels? **Loss:** one lost piece costs 1,460 bytes to resend, about two thousand times less than the whole photo. **Sharing:** your family shares one Wi-Fi; while your brother streams a cricket match and you send the photo, the small packets take turns on the wire and nobody waits long.



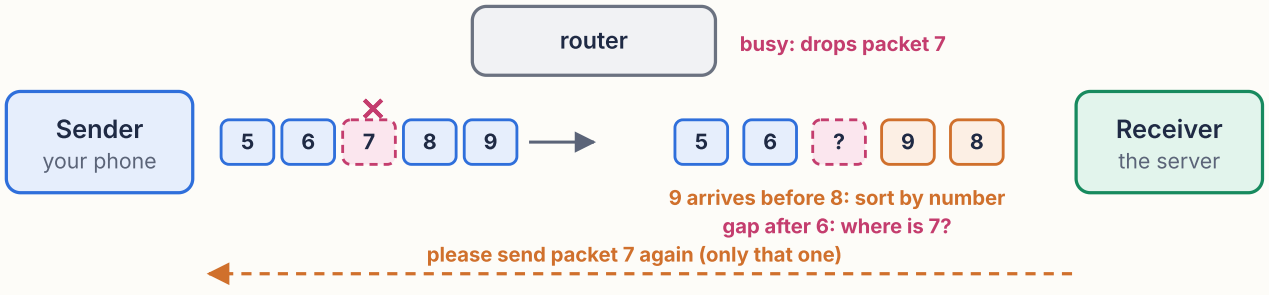
Figure 4. Real numbers: a 3 MB photo is about 2,055 packets; a loss costs one packet; small packets let everyone take turns.

For the curious. 1,500 bytes is the Ethernet limit for the packet itself; the Ethernet frame around it adds a few more bytes that are not counted. Headers can be longer than 20 bytes when options are used, and IPv6 headers are 40 bytes, so real payloads are often a little under 1,460. Strictly, the number in the TCP header counts bytes, not packets; lesson 8 shows how.

6 What goes wrong: lost and out of order

Packet 7 is lost: a busy **router**, the machine that passes packets along, had no room and dropped it. And packets 8 and 9 arrive the wrong way round.

- **Out of order is easy.** The receiver sorts by number, and 8 goes before 9. No harm done.
- **The loss is the real problem.** The receiver sees a gap, 6 then 8: where is 7? It holds on to 8 and 9, and the sender is asked to send packet 7 again. Just that one.
- Packet 7 arrives, the gap is filled, and the photo is rebuilt perfectly. That's why your photo is fine even when the network isn't. Exactly how the asking works is lesson 8.



The receiver holds 8 and 9, gets the new copy of 7, and rebuilds the photo perfectly. How the asking works: lesson 8.

Figure 5. What goes wrong: packet 7 is dropped and 8 and 9 arrive the wrong way round. The receiver sorts by number, sees the gap and asks for 7 again.

PAUSE AND THINK

A friend sends you a voice note of 1.46 MB. Each packet carries 1,460 bytes of payload. How many packets is that? And if packet 500 is lost, what is sent again? Cover the answer and decide.

Answer. 1,460,000 ÷ 1,460 = **1,000 packets**. If packet 500 is lost, only packet 500 is sent again: 1,460 bytes. Your phone keeps packets 501 onwards, waits for 500, and then reassembles the voice note.



Figure 6. The pause-and-think answer: 1,000 packets, and only packet 500 is sent again.

TWO COMMON MISTAKES

- **"A packet is the whole message."** No. A photo or a web page is usually many packets. A packet is one piece, at most 1,500 bytes on most networks.
- **"One lost packet means the whole download starts again."** No. Only the missing piece is sent again; the other pieces wait until the gap is filled.

RECAP IN THREE LINES

1. Data travels in small packets, each a header plus a payload.
2. The MTU, usually 1,500 bytes, makes a 3 MB photo about two thousand packets.
3. Numbers in the headers let the receiver reassemble them, and ask again for just the missing piece.

7 Interview questions: what to say in 30 seconds

Q1. What is a packet, and what is inside one? BASIC

A packet is one small piece of data that travels across the network on its own. It has a header, the label with the source and destination addresses and a number, and a payload, the piece of data itself. A photo or a web page is usually many packets.

Q2. What is the MTU, and what is it on most networks? BASIC

The maximum transmission unit: the largest packet a network link will carry. On Ethernet, and on typical home Wi-Fi, it is 1,500 bytes. After a 20-byte IP header and a 20-byte TCP header, that leaves 1,460 bytes of data per packet.

Q3. How many packets does a 3 MB photo need? SCENARIO

3,000,000 bytes ÷ 1,460 bytes per packet ≈ 2,055 packets, about two thousand. The headers cost 40 bytes in every 1,500, under 3%.

Q4. Why not send the whole file as one big piece? SCENARIO

Two reasons. Loss: if one piece is lost, only 1,460 bytes are resent instead of the whole file. Sharing: small packets take turns on a link, so one big transfer does not block everyone else; on a 10 Mbit/s link a 1,500-byte packet takes 1.2 ms, while a 3 MB file as one piece would hold the link for 2.4 s.

Q5. Packets arrive out of order and one is missing. Is the file broken? TRICKY

No. The receiver sorts the packets by the number in their headers, holds the ones after the gap, and the missing one is sent again. Only when the gap is filled is the data handed over, complete and in order.

Q6. Does one lost packet mean the download restarts? TRICKY

No. Only the missing packet is resent; everything that arrived is kept. (With TCP, lesson 8, the number actually counts bytes, not packets, and the receiver's replies tell the sender what to resend.)

Read more. RFC 791 (Internet Protocol), RFC 9293 (TCP) and RFC 894 (IP over Ethernet, the 1,500-byte limit). The full written lesson is at backendroadmap.com.

Next: SD-N-003 · IP addresses: the street address of every device backendroadmap.com

IP addresses

the street address of every device

Watch the 9-min video

In one line: an IP address is a device's address on a network, with a network part (which network, like the building) and a device part (which device, like the flat number); IPv4 is 32 bits, about 4.3 billion addresses, so homes use private addresses behind one public address, and DHCP keeps them unique.

WORDS YOU'LL MEET TODAY

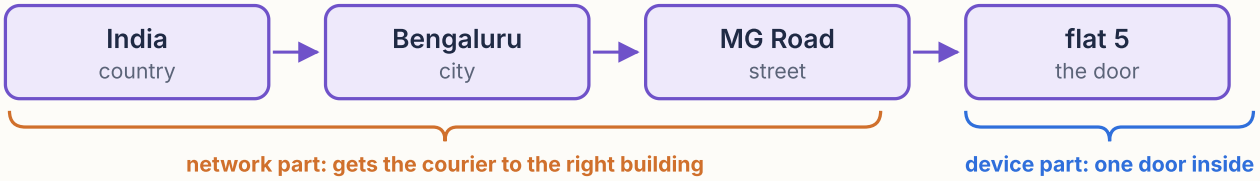
- **IP address (internet protocol address)**: a device's address on a network
- **IPv4**: the classic kind: four numbers 0 to 255, 32 bits
- **IPv6**: the newer kind: 128 bits, far more addresses
- **Public / private address**: reachable from the internet / only inside a network
- **Router**: the box that joins networks and passes packets on
- **DHCP (dynamic host configuration protocol)**: lends each device a free address
- **Bit / byte**: a bit is one 0 or 1; 8 bits make a byte
- **Localhost (127.0.0.1)**: always means this same device

1 The question: how does a message find one phone?

You type a friend's name in WhatsApp and press send. Out there are about two billion other phones, yet your message reaches exactly one: your friend's. A courier does the same thing every day: out of a million flats, he finds the right door, because every flat has an address. So what is a computer's address, what does it look like, and what happens when two devices share one?

2 An everyday picture first: the courier

A courier has a parcel for **flat 5, MG Road, Bengaluru, India**. He goes country, then city, then street; only then does he look for the flat. There are two parts: India, Bengaluru, MG Road gets him to the right building, and flat 5 finds one door inside it. And no two flats in a building share a number, or he wouldn't know where to knock.



No two flats in one building share a number; if two doors said 5, the courier would not know where to knock.

Figure 1. The courier's address: country, city and street get him to the right building; the flat number finds one door inside it.

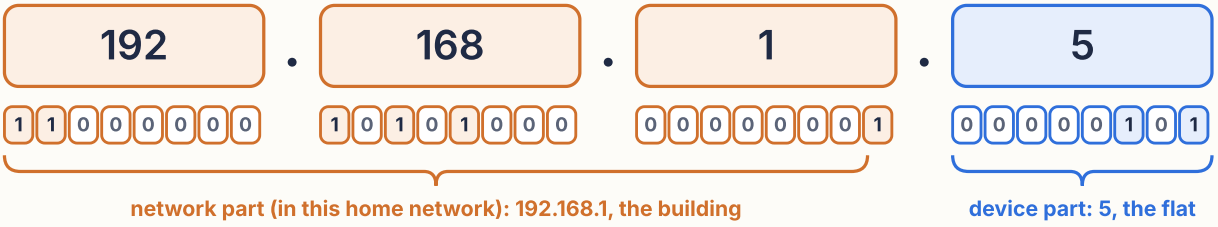
3 Now the real thing

The courier	In a network
The parcel	your message, cut into packets (lesson 2)
The courier	the network and its **routers**
The address on the parcel	the **IP address**, written in each packet's header
Country, city, street (the building)	the **network part** of the address
The flat number	the **device part** (also called the host part)

Exactly where the line between the two parts falls is lesson 13, subnets and CIDR.

4 What an IP address looks like

Here is a real one: **192.168.1.5**. Four numbers, with dots between them. This kind is called **IPv4**, version four, and it's the one you'll see most. Each number goes from 0 to 255, because each one is stored in 8 bits, one byte. A **bit** is a single 0 or 1, and 8 bits can count 256 values: 0 (00000000) to 255 (11111111). So the whole address is four bytes: 32 bits. In this home network, 192.168.1 is the network part, the building, and 5 is this phone's flat number.



Each number is 8 bits = 1 byte, so it goes from 0 (00000000) to 255 (11111111): 256 values.

4 numbers × 8 bits = 32 bits. Where the line between the two parts falls is lesson 13 (subnets and CIDR).

Figure 2. Anatomy of an IPv4 address: four numbers, each one byte (0 to 255), 32 bits in all; here 192.168.1 is the network part and 5 the device part.

5 Real numbers: why we ran short

Question	Arithmetic	Answer
values in one number	2^8	**256** (0 to 255)
bits in an IPv4 address	4 × 8	**32 bits**
IPv4 addresses	256 × 256 × 256 × 256 = 2^{32}	**4,294,967,296** ≈ 4.3 billion
IPv6 addresses	2^{128}	**≈ 3.4 × 10^{38}** (a 39-digit number)

4.3 billion sounds like a lot, but there are over 8 billion people, many with a phone, a laptop and a TV, plus every server. The last big free blocks of IPv4 addresses ran out in 2011. There are two fixes. One: **IPv6**, with addresses 128 bits long, written in hexadecimal groups like `2001:0db8:85a3:0000:0000:8a2e:0370:7334`: enough for every grain of sand on Earth. Two: **private addresses**, reused in every home.

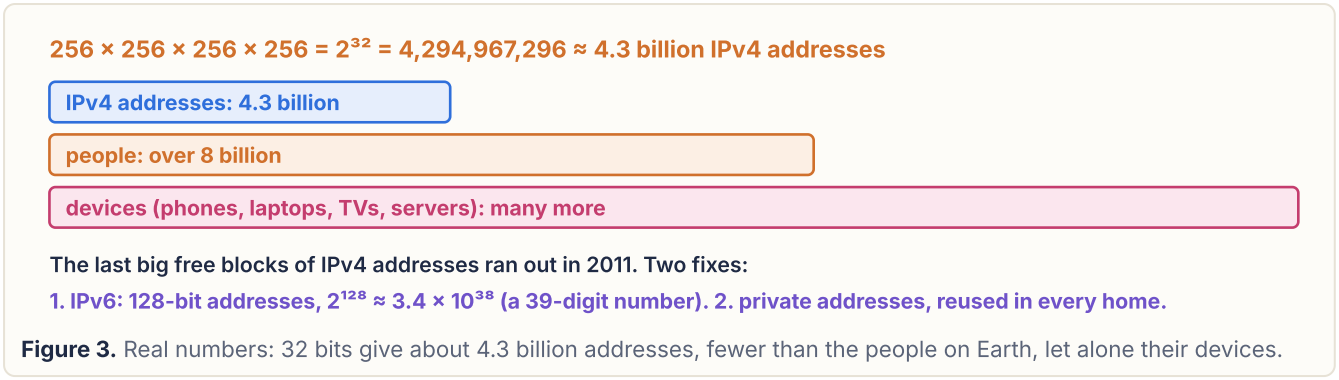


Figure 3. Real numbers: 32 bits give about 4.3 billion addresses, fewer than the people on Earth, let alone their devices.

6 At home: private addresses, one public address

Your Wi-Fi router gives each device a **private** address: your phone gets 192.168.1.5, your laptop 192.168.1.6 and the TV 192.168.1.7. Three ranges are kept for private use, and millions of homes reuse them; that's fine, because they never leave the home:

Private range	Written as (lesson 13)	Size
10.0.0.0 to 10.255.255.255	10.0.0.0/8	16,777,216 addresses
172.16.0.0 to 172.31.255.255	172.16.0.0/12	1,048,576 addresses
192.168.0.0 to 192.168.255.255	192.168.0.0/16	65,536 addresses

Your router also has one **public** address on its outside, for example 203.0.113.7. The whole internet can reach it, and to the internet your whole family is that one address. How the router shares it is lesson 14 (NAT, network address translation).

One special address: **127.0.0.1**, called **localhost**, always means this same device.

See your own: on a phone, open Wi-Fi settings and tap your network; on Windows, type `ipconfig`; on a Mac or Linux, `ifconfig` (or `ip addr` on Linux). That shows your private address. For your public address, search “what is my IP”.

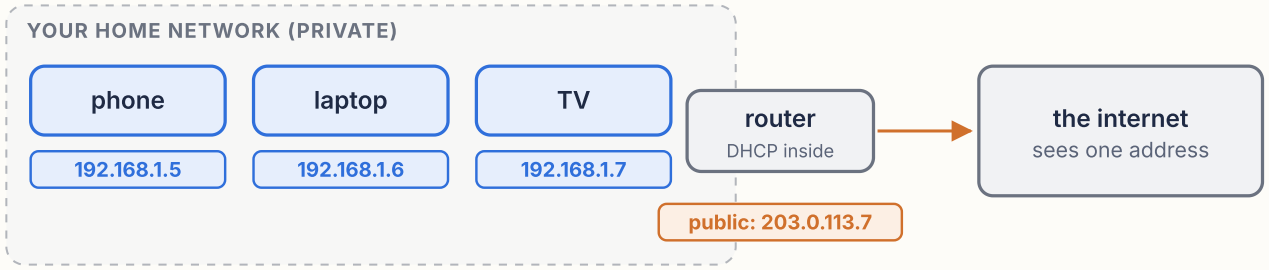
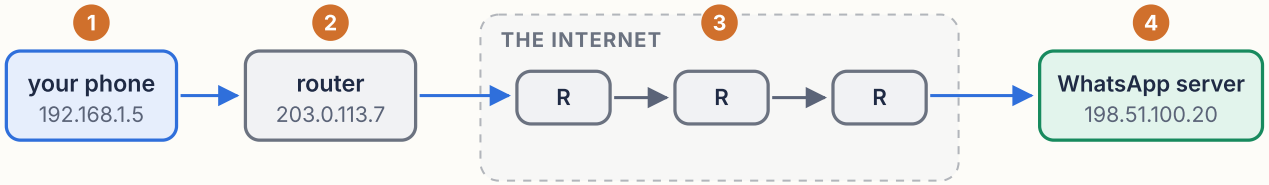


Figure 4. At home, the router gives each device a private address and shows the internet one public address. How that one address is shared (NAT) is lesson 14.

7 Step by step: one message

1. **Address it.** Your phone writes the header: from 192.168.1.5, to the WhatsApp server's public address. Job: say where to go, and where to reply.
2. **Leave home.** The router sees the destination isn't at home and sends the packet out, with the public address as the sender. Job: get the packet out of the building.
3. **Hop by hop.** Routers on the internet read the network part and pass the packet to one that's closer. Job: one step closer each time.
4. **Deliver.** The server's network uses the device part to reach that one server. The reply comes back the same way, to your phone.



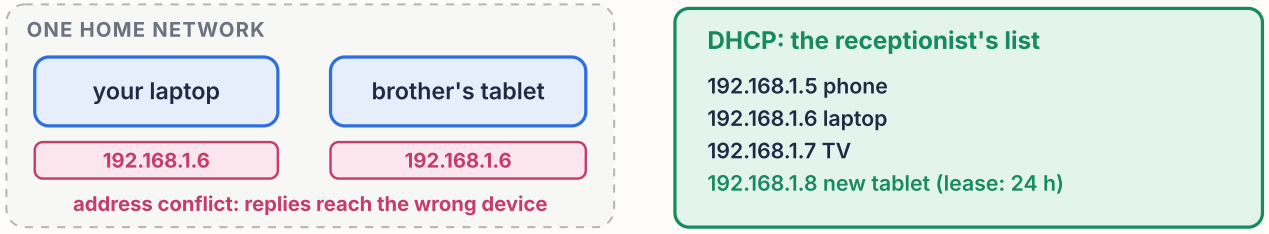
1 the phone writes from: 192.168.1.5, to: 198.51.100.20 2 the router sends it out, from: its public address
3 each router reads the network part and passes it one step closer 4 the device part finds the one server
The reply comes back to 203.0.113.7, and the router hands it to the phone.

Figure 5. One message, step by step: address it, leave home, hop by hop, deliver; the reply comes back the same way.

The addresses 203.0.113.7, 198.51.100.20 and 198.51.100.9 come from ranges reserved for examples in documentation (RFC 5737), so they belong to nobody real.

8 What goes wrong

- **An address conflict.** Your laptop and your brother's tablet both end up as 192.168.1.6: two doors with one flat number. The router can't tell them apart, replies go to the wrong device, and both connections keep breaking.
- **The fix: DHCP**, short for dynamic host configuration protocol. Think of it as the building's receptionist: a new device asks “which flat can I have?”, and DHCP picks a free number, writes it down, and never gives it to two devices at once. The number is lent, often for a day: a **lease**. At home, DHCP runs inside your router. Most conflicts come from someone typing an address in by hand.
- **A wrong address.** Send to an address where no device is, and there's no door to knock on. Nothing answers, and your app gives up with a timeout.



DHCP picks a free address, writes it down and lends it for a while (a lease), so it never gives one address to two devices.
Most conflicts come from an address typed in by hand. A wrong address, where no device is, gets no answer: a timeout.

Figure 6. What goes wrong: two devices with one address on one network. The fix is DHCP, which lends each device a free address and keeps a list.

PAUSE AND THINK

Your phone shows 192.168.1.5, and so does a friend's phone in another city. Is that a conflict? Cover the answer and decide.

Answer. No. Those are private addresses in two different homes: two different buildings. A conflict needs one address twice on the same network. On the internet, each home shows its own public address.



Same private address, two different networks: no conflict. A conflict needs one address twice on the same network.

Figure 7. The pause-and-think answer: two homes can both use 192.168.1.5; each shows the internet its own public address.

TWO COMMON MISTAKES

- **"Every device has its own address on the internet."** No. Most devices have a private address; the internet sees the router's one public address.
- **"A number in an address can be anything, like 300."** No. Each number is one byte, so it is always 0 to 255; 192.168.1.300 is not a valid address.

RECAP IN THREE LINES

1. An IP address names a device, with a network part and a device part.
2. IPv4 is four numbers from 0 to 255, 32 bits, about 4.3 billion addresses, so we also have IPv6 and private addresses.
3. At home, DHCP hands out unique private addresses, and the router shows one public address.

9 Interview questions: what to say in 30 seconds

Q1. What is an IP address, and what are its two parts? BASIC

An IP address identifies a device on a network, so packets can be delivered to it. It has a network part, which says which network the device is on, and a host (device) part, which says which device on that network. Routers use the network part to move a packet toward the right network; the host part finds the device at the end.

Q2. What does an IPv4 address look like, and how many are there? BASIC

Four numbers from 0 to 255 separated by dots, such as 192.168.1.5. Each number is 8 bits, one byte, so an address is 32 bits, and there are 2^{32} = 4,294,967,296 of them, about 4.3 billion. That is not enough for every device, which is why IPv6 (128 bits) and private addresses exist.

Q3. What is the difference between a public and a private address? BASIC

A public address is unique on the internet and reachable from it. Private addresses come from three reserved ranges, 10.0.0.0/8, 172.16.0.0/12 and 192.168.0.0/16; they are reused in millions of networks and are not routed on the internet. A home router gives its devices private addresses and shows the internet one public address.

Q4. Your laptop and another device on the same Wi-Fi have the same IP address. What happens, and how is it avoided? SCENARIO

An address conflict: the network cannot tell the two apart, replies reach the wrong device and both connections keep breaking. DHCP avoids it by handing out addresses from a pool, keeping a list and lending each one for a lease, so it never gives one address to two devices. Most conflicts come from an address typed in by hand.

Q5. Two phones in different cities both have 192.168.1.5. Is that a problem? TRICKY

No. Those are private addresses on two different networks. A conflict needs the same address twice on the same network. On the internet, each home shows its own public address.

Q6. What is 127.0.0.1? TRICKY

The loopback address, called localhost: it always means this same device. A program that sends to it is talking to itself, and the packets never leave the machine. (The whole 127.0.0.0/8 range is reserved for loopback.)

Read more. RFC 791 (Internet Protocol, version 4), RFC 1918 (private address ranges), RFC 2131 (DHCP), RFC 8200 (IPv6), RFC 5737 (example addresses). The full written lesson is at backendroadmap.com.

Next: SD-N-004 · Ports and sockets: which app gets the packet backendroadmap.com

Ports and sockets

which app gets the packet

Watch the 9-min video

In one line: the IP address gets a packet to the device, and the port, a 16-bit number from 0 to 65,535, gets it to one app on that device; servers listen on well-known ports such as 443, clients are lent a random high port, and a connection is a pair of sockets: four values that make it unique.

WORDS YOU'LL MEET TODAY

Port: a number (0 to 65,535) that names one app on a device

Listening: an app waiting for packets on its port

Well-known port: a fixed number for a common service, 0 to 1023 (443 = HTTPS)

Connection: a pair of sockets: client IP and port, server IP and port

Socket: one end of a conversation: an IP address plus a port

Protocol: agreed rules for talking; TCP and UDP are the two main ones

Ephemeral port: a high random port lent to a client for one conversation

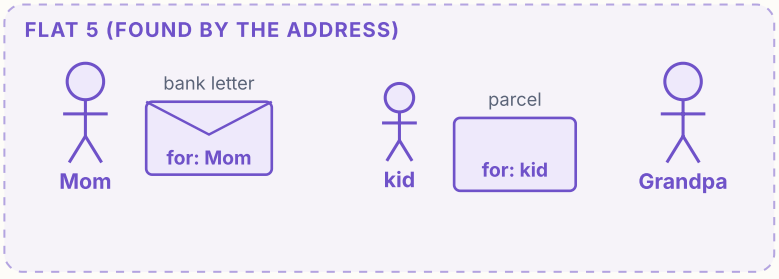
Address already in use: the error when a second app tries to listen on a taken port

1 The question: which app gets the packet?

Your phone is running WhatsApp, Chrome and Spotify, all at the same time. A message arrives, a web page arrives, a song arrives. But in lesson 3 we saw that your phone has one IP address, and all three apps share it. So when a packet arrives, how does the phone know which app it's for?

2 An everyday picture first: the people inside the flat

In lesson 3, the address got the courier to one flat. But a flat has people in it: Mom, the kid and Grandpa. The courier brings a bank letter and a parcel. The flat number gets both to the door; the **name** on each one says who inside wants it. The letter is for Mom, the parcel is for the kid. And someone must be home, waiting for it: if nobody with that name lives there, the letter can't be delivered.



The flat number gets the post to the door; the name says who inside wants it. No such person: it can't be delivered.

Figure 1. The flat (lesson 3's device) has people inside: the apps. The name on each envelope is the port, so the right person gets it.

3 Now the real thing

The flat	In a network
The flat	your device, found by its **IP address** (lesson 3)
The people inside	the **apps** running on it
The name on the envelope	the **port number**, written in each packet's header
Someone at home, waiting	an app **listening** on its port

So every packet carries two things: the IP address, to reach the device, and the port, to reach one app on it. Together they say: this device, this app.

4 What a port looks like

A port is just a number, written after the address with a colon: **192.168.1.5:443** means the device 192.168.1.5, port 443. A port is stored in 16 bits, and 2^{16} = 65,536, so ports go from **0 to 65,535**: over 65,000 possible ports on every device.

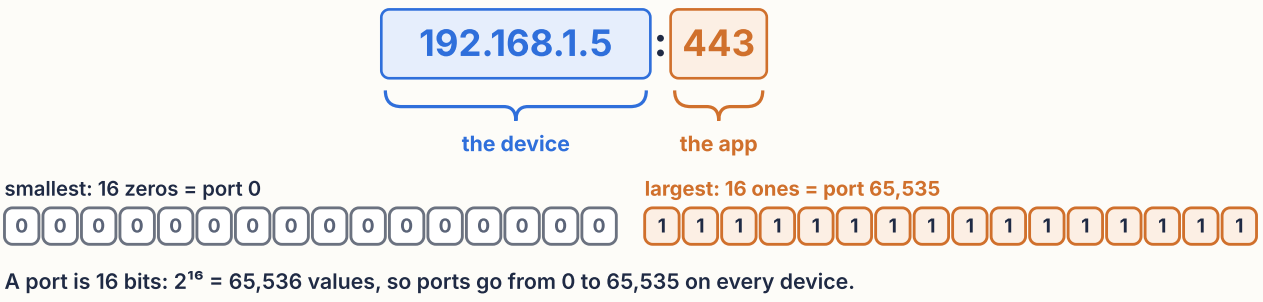


Figure 2. Anatomy of 192.168.1.5:443: the IP address finds the device, the port after the colon finds the app. A port is 16 bits, 0 to 65,535.

The low ones, **0 to 1023**, are **well-known ports**, fixed for common services:

Port	Service	What it's for
80	HTTP	the web
443	HTTPS	the secure web
22	SSH	logging in to a server
53	DNS	looking up names (lessons 15 to 17)
25	SMTP	sending email between mail servers

That's why you never type a port: open a web address, and the browser uses 443 by itself (`https://backendroadmap.com` means `backendroadmap.com:443`).

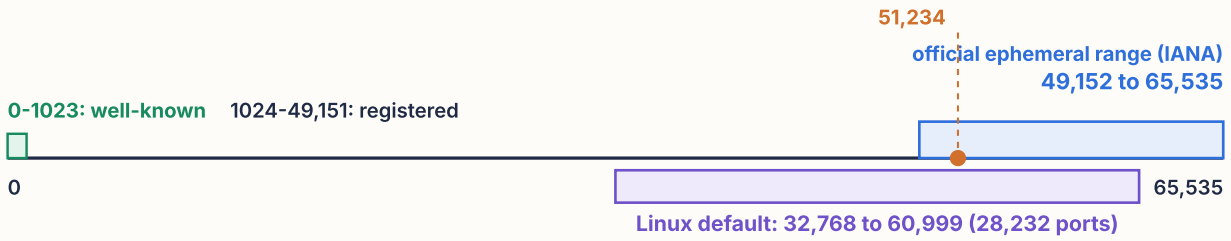
5 Listening, and the protocol

On a server, an app opens a port and waits. That's called **listening**, like Mom at home, waiting for her letter. A web server listens on 443, and a packet for port 443 goes to it, and only to it. One more detail: each port belongs to a **protocol**, TCP or UDP, two sets of rules for sending data. Port 53 over TCP and port 53 over UDP are separate. TCP is lessons 8 to 11, and UDP is lesson 12.

6 Your side: an ephemeral port

Your browser isn't a well-known service. So when it connects, your laptop lends it a port: a high random number, for example **51,234**. It's called an **ephemeral port**, because it lasts only as long as the conversation.

The official range is **49,152 to 65,535**; Linux uses **32,768 to 60,999** by default (28,232 ports).



Servers listen on well-known ports. Clients are lent a high random (ephemeral) port, such as 51,234, for one conversation.

Figure 3. All 65,536 ports on one line (to scale): well-known 0 to 1023 at the very start, the official ephemeral range 49,152 to 65,535, and the Linux default 32,768 to 60,999.

7 Sockets and connections

A **socket** is one end of a conversation: an IP address plus a port. Your laptop's end is **203.0.113.7:51234**. The server's end is **198.51.100.10:443**. A **connection** is the pair of sockets: four values, client IP, client port, server IP, server port. No two connections share all four.

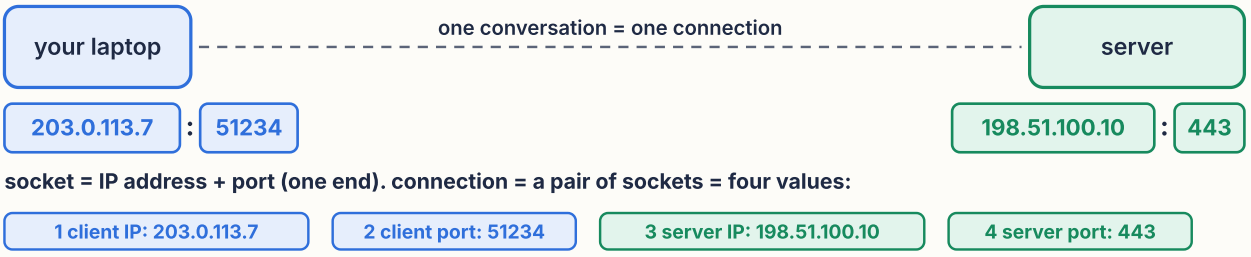


Figure 4. A socket is one end of a conversation, an IP address plus a port. A connection is the pair of sockets: four values.

Connection	Client IP	Client port	Server IP	Server port
your tab 1	203.0.113.7	51234	198.51.100.10	443
your tab 2	203.0.113.7	**51235**	198.51.100.10	443
a friend	**192.0.2.25**	50112	198.51.100.10	443

So one server port can serve many people: same server port, but every connection is unique.

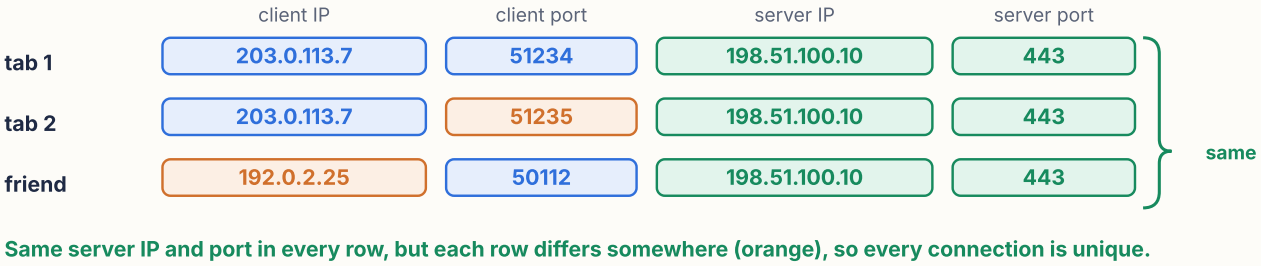
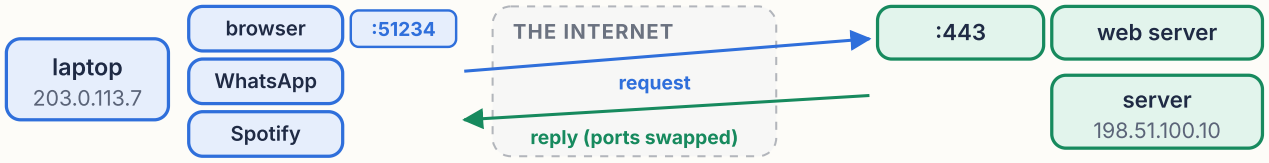


Figure 5. One server port serves many connections: a second tab gets another client port, a friend has another address, and every four-value row is still unique.

8 Step by step: one request

1. **Listen.** The web server starts and listens on 443. Job: be ready before anyone calls.
2. **Get a port.** Your browser asks for a socket and gets port 51,234. Job: give the reply a way back to this one app.

3. **Send.** The packet goes out from 203.0.113.7:51234 to 198.51.100.10:443. The server reads the port and hands it to the web server. Job: carry both ends, so each side knows where to reply.
4. **Reply.** The reply comes back with the ports swapped: from 443, to 51,234. Your laptop reads the port and gives it to the browser, not to WhatsApp, not to Spotify. Job: the right app, every time.



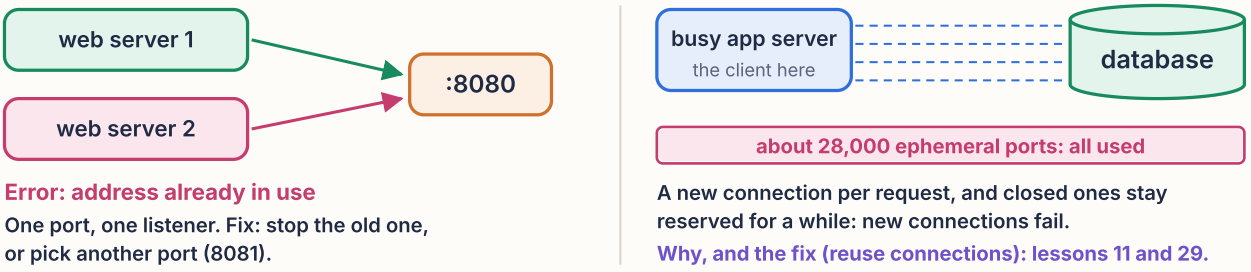
1 Listen. The web server starts and listens on 443. Job: be ready before anyone calls.
2 Get a port. The browser asks for a socket and is lent port 51234. Job: give the reply a way back to this one app.
3 Send. From 203.0.113.7:51234 to 198.51.100.10:443; the server reads port 443 and hands it to the web server.
4 Reply. From 198.51.100.10:443 to 203.0.113.7:51234; the laptop reads 51234 and gives it to the browser, not WhatsApp.

Figure 6. One request, step by step: listen, get a port, send, reply. The ports in the header pick the app at each end.

For the curious: at home, your router also rewrites the port as it shares one public address, so the server may see a different client port. That's NAT, lesson 14. 203.0.113.7, 198.51.100.10 and 192.0.2.25 come from the ranges reserved for documentation (RFC 5737), so they belong to nobody real.

9 What goes wrong

- **Port already in use.** You start a web server on port 8080, then a second one, also on 8080. It fails: `address already in use`. One port, one listener; otherwise the system couldn't know which app gets the packet. The fix: stop the old one, or pick another port.
- **Running out of ports.** Linux has about 28,000 ephemeral ports. A busy server that opens a new connection to its database for every request can use them all, because closed ones stay reserved for a while. New connections fail. Why, and the fix, is lesson 11 (and reusing connections, lesson 29).



Error: address already in use
One port, one listener. Fix: stop the old one, or pick another port (8081).

A new connection per request, and closed ones stay reserved for a while: new connections fail.
Why, and the fix (reuse connections): lessons 11 and 29.

Both failures are about the same rule: each socket on a device must be unique.

Figure 7. What goes wrong: two apps trying to listen on one port (address already in use), and a busy client using up its ephemeral ports.

PAUSE AND THINK

A shop's server listens on port 443. 100,000 phones connect at the same time. Does it need 100,000 ports? Cover the answer and decide.

Answer. No. They all use the same server port, 443. Each connection is still unique, because the client's address or port is different. One port, 100,000 different pairs.

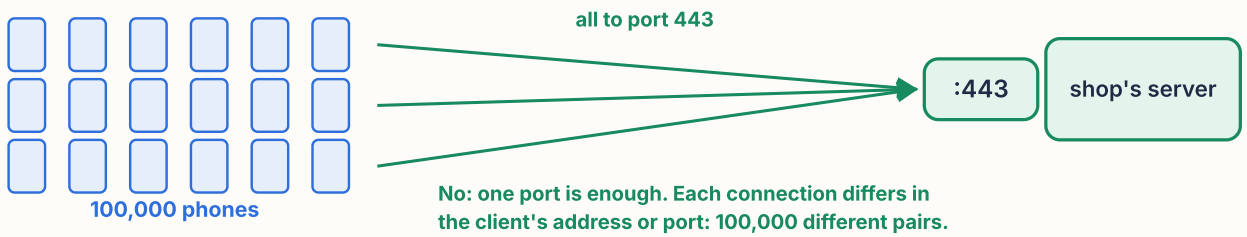


Figure 8. The pause-and-think answer: 100,000 phones can all connect to port 443; each four-value pair is different, so one server port is enough.

TWO COMMON MISTAKES

- **"A server needs a new port for every client."** No. Every client connects to the same listening port; the four values keep the connections apart.
- **"A port and a socket are the same thing."** No. A port is just a number (0 to 65,535). A socket is an IP address plus a port: one end of a conversation.

RECAP IN THREE LINES

1. The IP address finds the device, and the port, a number from 0 to 65,535, finds the app.
2. Servers listen on well-known ports, like 443, and clients get a random high port.
3. A socket is an IP address plus a port, and a connection is a pair of sockets: four values.

10 Interview questions: what to say in 30 seconds

Q1. What is a port, and why do we need one if we already have an IP address? BASIC

A port is a 16-bit number, 0 to 65,535, that identifies one app (one service or one conversation) on a host. The IP address only gets a packet to the device; many apps share that one address, so the port in the packet's header tells the device which app gets it.

Q2. What is a socket, and what identifies a connection? BASIC

A socket is one end of a conversation: an IP address plus a port, such as 203.0.113.7:51234. A connection is a pair of sockets, so it is identified by four values: client IP, client port, server IP and server port (plus the protocol, TCP or UDP). No two live connections share all of them.

Q3. What are well-known ports and ephemeral ports? BASIC

Well-known ports, 0 to 1023, are fixed for common services: 80 HTTP, 443 HTTPS, 22 SSH, 53 DNS, 25 SMTP. Servers listen on them so clients know where to connect. A client is lent an ephemeral port, a high random number, for one conversation: the official range is 49,152 to 65,535, and Linux uses 32,768 to 60,999 by default.

Q4. You start your app and get "address already in use". What happened, and what do you do? SCENARIO

Another process is already listening on that port, maybe an old copy of your app. Only one app can listen on a given IP address and port, otherwise the system could not tell which one gets the packet. Find and stop the old process, or start your app on a different port.

Q5. How can one server port, 443, serve 100,000 clients at once? TRICKY

The server does not need a port per client. Each connection is the four values, and while the server IP and port are the same for all of them, the client IP or client port is different in every one, so each connection is still unique. The limits are memory, file handles and CPU, not the port number.

Q6. Can a client run out of ports? TRICKY

Yes. A client has only a few tens of thousands of ephemeral ports (about 28,000 on Linux by default) for connections to one server address and port. A busy service that opens a new connection for every request, while closed ones stay reserved for a while (TIME_WAIT), can use them all and new connections fail. The fixes, such as reusing connections, are in lessons 11 and 29.

Read more. RFC 793 / RFC 9293 (TCP), RFC 768 (UDP), RFC 6335 (port number ranges and the IANA registry), RFC 6056 (choosing ephemeral ports), RFC 5737 (example addresses). The full written lesson is at backendroadmap.com.

Next: SD-N-005 · Protocols and layers: agreed languages, and why we stack them backendroadmap.com

Protocols and layers

agreed languages, and why we stack them

Watch the 9-min video

In one line: a protocol is a set of agreed rules so two machines understand each other; the internet splits the work into four layers, application, transport, network and link, each with its own protocol, and each layer adds its own header on the way down and removes it on the way up.

WORDS YOU'LL MEET TODAY

- **Protocol**: agreed rules for talking, so two sides understand each other
- **Header**: a label a layer adds in front of the data
- **Segment**: data plus a TCP header (transport layer)
- **Frame**: the package for one link: a packet plus a link header and check value
- **Layer**: one job in a stack of jobs, with its own protocol
- **Encapsulation**: wrapping data inside a header, layer by layer
- **Packet**: a segment plus an IP header (network layer)
- **MTU**: the biggest packet one link carries: 1,500 bytes on Ethernet

1 The question: how do strangers understand each other?

You open a shopping app and tap Buy. Your phone was made by one company, the shop's server by another, and your order crosses Wi-Fi, a fibre cable and routers built by many others. None of them ever met, yet your order arrives and is understood perfectly. Two ideas make that possible: **protocols** and **layers**.

2 A protocol is agreed rules

You already use one every day: a phone call. Someone picks up and says hello, you say who you are, you take turns to talk, then you both say bye and hang up. Nobody wrote these rules down, but both sides follow them. If one side stayed silent, or spoke a language the other didn't know, the call would fail.

Computers need the same, but exact. In **HTTP**, the protocol of the web, the browser sends `GET /shoes`, and the server must answer with a status, like `200 OK`, and then the page. Both follow the same rulebook, so machines from different companies understand each other.



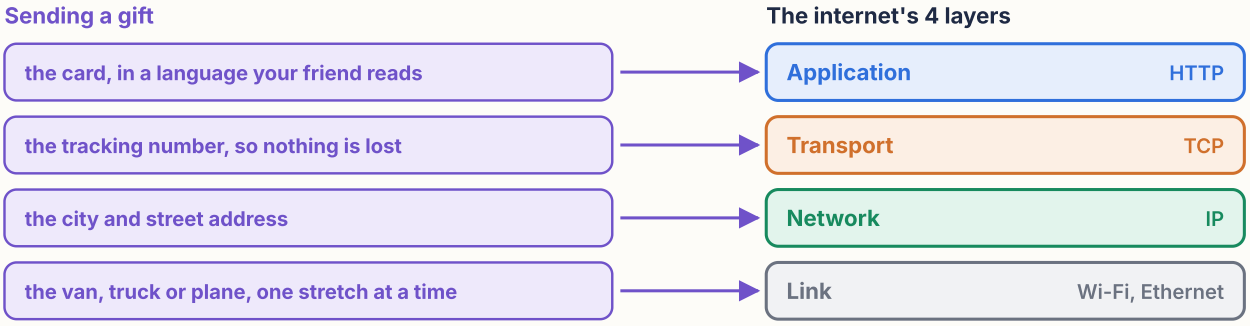
A protocol is agreed rules, so two sides understand each other. If one side breaks them, the conversation fails.

Figure 1. A protocol is a set of agreed rules. A phone call follows unwritten ones; HTTP writes them down exactly, so machines from different companies understand each other.

3 Why one protocol isn't enough: layers

One protocol can't do everything. Someone must find the address, someone must check that nothing was lost, someone must move the bits over Wi-Fi. So the work is split into **layers**. Think of sending a gift to a

friend in Pune: you write the card; the courier packs it with a tracking number; the hub writes the city and street; a van, a truck or a plane each moves it one stretch. Each person reads only their own label (the van driver never opens your card), and you can swap one part, a train instead of a truck, without changing the card.



Each person reads only their own label. Swap the truck for a train, and the card doesn't change.

Figure 2. The gift, layer by layer: the card is the application layer, the tracking number is transport, the address is the network layer, and the van is the link layer.

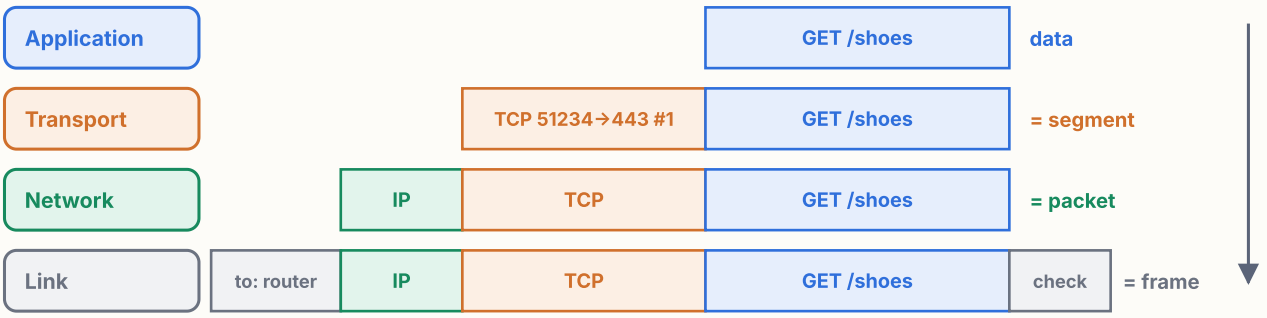
Layer	Protocols	Its one job	Unit
Application	HTTP, DNS, SMTP	what the app means	data
Transport	TCP, UDP	the right app (ports); for TCP, nothing lost, in order	segment
Network	IP	from and to addresses: find the device anywhere	packet
Link	Wi-Fi, Ethernet, mobile data	move the bits across one link, to the next device	frame

This four-layer version is the TCP/IP (internet) model. Lesson 6 covers the seven-layer OSI model you'll hear in interviews, and lesson 7 follows a URL through the TCP/IP model.

4 Step by step: encapsulation

1. **Application.** Your app writes the data: `GET /shoes`.
2. **Transport.** TCP adds its header in front: from port 51,234 to port 443, plus a sequence number. Data plus this header is a **segment**.
3. **Network.** IP adds its own header in front of that: from your IP address to the server's. Now it's a **packet**.
4. **Link.** The link layer wraps the whole packet in a **frame**: the hardware address of the next device (your Wi-Fi router) in front, and a check value at the end to catch damage.

This wrapping, layer by layer, is called **encapsulation**. At the other end it runs in reverse: each layer reads only its own header, removes it and passes the rest up, link, then IP, then TCP, until the web server reads `GET /shoes`. Like opening boxes inside boxes.

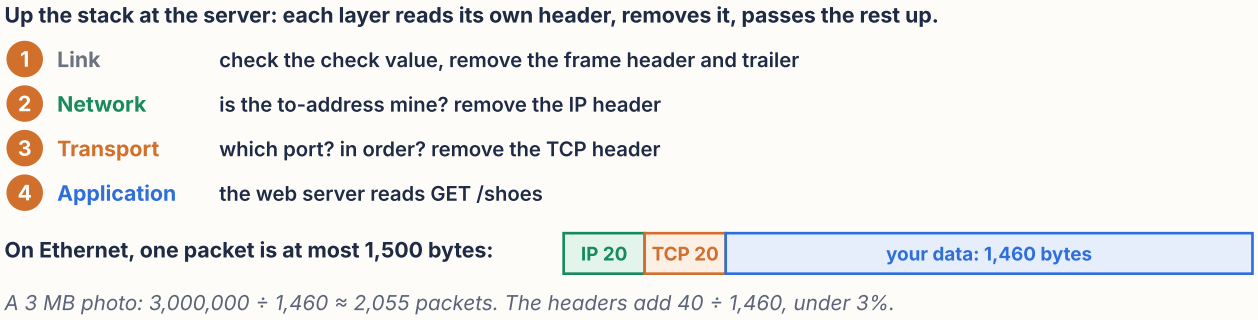


Encapsulation: each layer adds its own header in front (the link layer also adds a check value at the end).

Figure 3. Encapsulation on the way down: the data gets a TCP header (a segment), then an IP header (a packet), then a link header and trailer (a frame).

5 Real numbers

On Ethernet, one frame carries a packet of at most **1,500 bytes** (the MTU). The IP header usually takes 20 bytes and the TCP header at least 20, so each packet carries up to **1,460 bytes** of your data. A 3 MB photo is 3,000,000 bytes: 3,000,000 ÷ 1,460 ≈ **2,055 packets**, the photo in 2,000 pieces from lesson 2. All those headers add 40 ÷ 1,460, **under 3%**.



A 3 MB photo: 3,000,000 ÷ 1,460 ≈ 2,055 packets. The headers add 40 ÷ 1,460, under 3%.

Figure 4. On the way up each layer removes only its own header. The sizes: 20 bytes of IP header and at least 20 of TCP leave up to 1,460 bytes of data in a 1,500-byte packet.

6 Why bother with layers

- **One job each.** Each layer does one thing and does it well.
- **Swap a layer.** Replace Wi-Fi with a cable, and HTTP, TCP and IP don't change at all.
- **Different builders.** The app maker, the operating system and the chip maker each build different layers, and they still fit together.

7 What goes wrong

- **A damaged frame.** Noise on the Wi-Fi damages a frame. The link layer checks the check value, sees it doesn't match, and drops the frame. The layer above notices: TCP sees a missing piece and sends it again (exactly how, in lesson 8). Your app never knows. Each layer handles its own problems.
- **The wrong protocol.** Send plain HTTP to a port that expects HTTPS, and the server can't read it: you get an error like `400 Bad Request`. Both sides must speak the same protocol, at every layer.

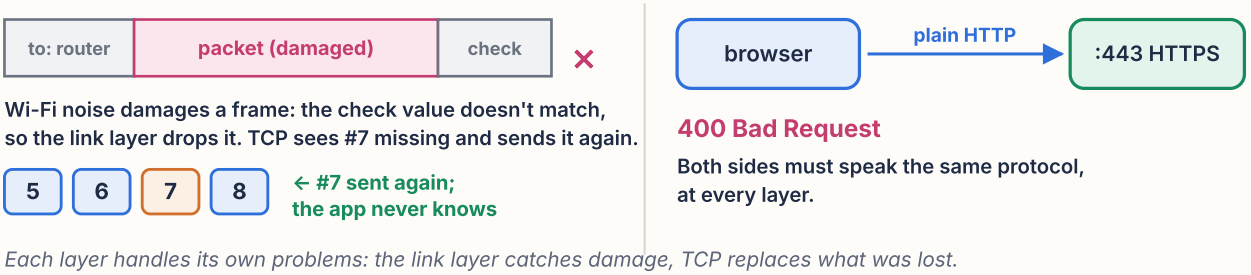


Figure 5. What goes wrong: a damaged frame is dropped by the link layer and resent by TCP, and a protocol mismatch (plain HTTP to an HTTPS port) gets an error.

PAUSE AND THINK

Your home internet is upgraded, from an old copper line to fibre. Does your browser need an update? Cover the answer and decide.

Answer. No. Only the bottom layer changed, on that one link. The network, transport and application layers above it work exactly as before. That's the whole point of layers.

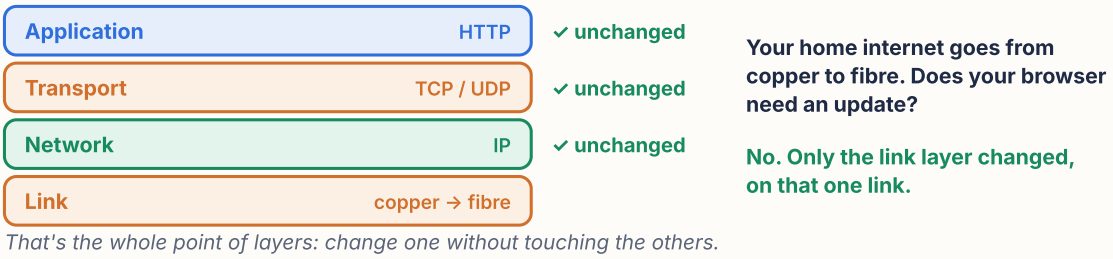


Figure 6. The pause-and-think answer: swapping copper for fibre changes only the link layer; the network, transport and application layers work exactly as before.

TWO COMMON MISTAKES

- **"TCP/IP is one protocol."** No. It's a family named after its two main members: TCP at the transport layer and IP at the network layer, with HTTP above and Wi-Fi or Ethernet below.
- **"Every device reads every header."** No. A router in the middle looks at the link and IP headers to pass the packet on; only the two end devices read the TCP header and the data.

RECAP IN THREE LINES

1. A protocol is a set of agreed rules, so two machines understand each other.
2. The internet stacks four layers, application, transport, network and link, each with one job.
3. Going down, each layer adds its header (encapsulation); going up, each one removes it.

8 Interview questions: what to say in 30 seconds

Q1. What is a protocol? Give an example. BASIC

A protocol is a set of agreed rules for how two sides exchange messages: what to send, in what format, in what order, and what to do when something goes wrong. HTTP is one: the client sends a request such as GET /shoes, and the server must answer with a status line such as 200 OK, headers and a body.

Q2. What are the layers of the internet (TCP/IP) model, and what does each do? BASIC

Four layers. Application (HTTP, DNS): what the app means. Transport (TCP, UDP): which app, by port, and for TCP, reliable in-order delivery. Network (IP): addressing and routing to the right device anywhere. Link (Ethernet, Wi-Fi): moving frames across one physical link to the next device.

Q3. What is encapsulation? BASIC

On the way down, each layer wraps what it gets from the layer above with its own header: data plus a TCP header is a segment, plus an IP header is a packet, plus a link header and trailer is a frame. On the way up, each layer reads and removes only its own header and passes the rest up.

Q4. Why do we use layers at all, instead of one big protocol? SCENARIO

Separation of concerns. Each layer does one job and only talks to the layers next to it, so you can change one without touching the others: replace Wi-Fi with fibre, or HTTP/1.1 with HTTP/2, and the rest keeps working. Different vendors can build different layers, and each layer handles its own errors.

Q5. How much of a 1,500-byte packet is your data? TRICKY

With a 20-byte IPv4 header and a 20-byte TCP header (no options), 1,460 bytes, which is the usual TCP maximum segment size on Ethernet. Options (timestamps are common) take 12 more, and IPv6 has a 40-byte header, so the real figure is often a bit less.

Q6. A frame is damaged on Wi-Fi. Which layer notices, and which layer fixes it? TRICKY

The link layer notices: the check value at the end of the frame doesn't match, so it drops the frame. It does not fix it; TCP, at the transport layer, sees the missing data and sends it again. The application never knows. Each layer handles the problems of its own job.

Read more. RFC 1122 (requirements for internet hosts: the layers), RFC 9110 (HTTP semantics), RFC 9293 (TCP), RFC 791 (IPv4), RFC 894 (IP over Ethernet, the 1,500-byte MTU), RFC 879 (TCP maximum segment size). The full written lesson is at backendroadmap.com.

Next: SD-N-006 · The OSI model, made simple backendroadmap.com

The OSI model

made simple

Watch the 9-min video

In one line: the OSI model splits networking into seven layers, physical, data link, network, transport, session, presentation and application; the real internet runs the four-layer TCP/IP model, and engineers use the OSI numbers as shorthand: a switch is layer 2, a router layer 3, and load balancers layer 4 or 7.

WORDS YOU'LL MEET TODAY

OSI model: a seven-layer reference model for naming where things happen in a network

Data link layer (2): moves frames to the next device on the same network

Switch: a box that connects devices on one network (layer 2)

Session / presentation (5, 6): the conversation, and the data's format and encryption

Physical layer (1): cables, fibre and radio carrying raw bits

MAC address: a 48-bit hardware address of a network card, used at layer 2

Router: a box that moves packets between networks (layer 3)

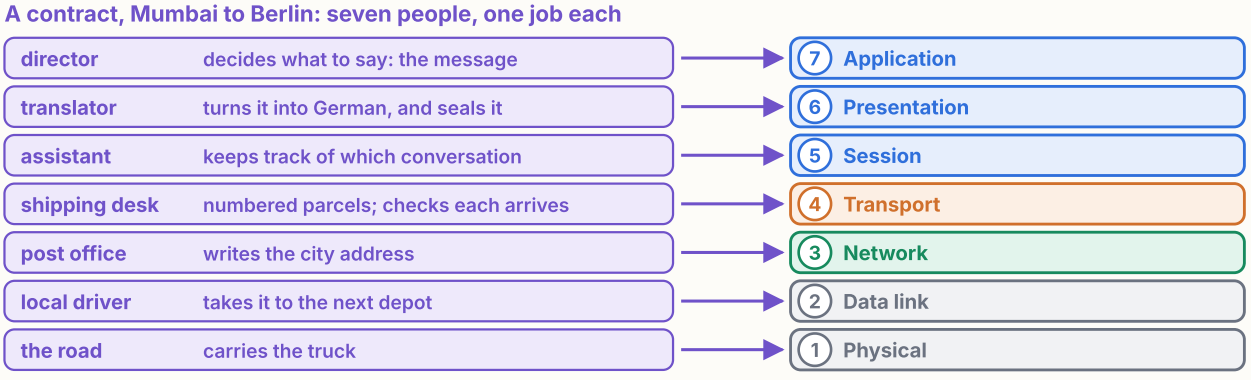
Load balancer: a box that spreads requests over many servers (layer 4 or 7)

1 The question: layer 7 of what?

In a system design interview, someone says: put a layer 7 load balancer in front. Or: that's a layer 4 problem. Layer 7 of what? Of the **OSI model**: seven layers, and every engineer uses their numbers as shorthand. In lesson 5 we met four layers; today we split them into seven and learn what each number means.

2 An everyday picture first: a contract, Mumbai to Berlin

A company in Mumbai sends a signed contract to a company in Berlin. Seven people touch it, each with one job. The director decides what to say: the message. A translator turns it into German, and seals it so nobody else can read it. An assistant keeps track of which conversation this belongs to. The shipping desk splits it into numbered parcels and checks each one arrives. The post office writes the city address. The local driver takes it to the next depot. And the road itself carries the truck.



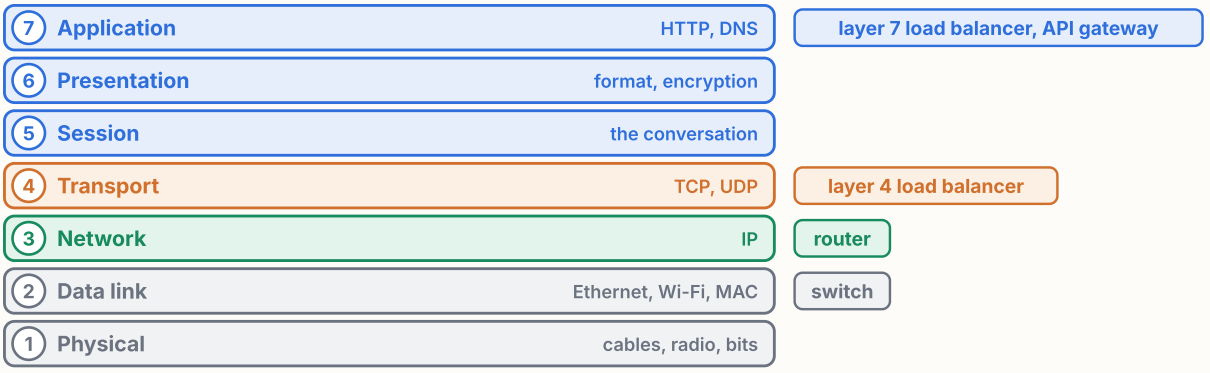
Each person reads only their own part, from the top down at Mumbai and from the bottom up at Berlin.

Figure 1. The analogy: seven people each do one job with the contract, and each job is one OSI layer, from the director (application) down to the road (physical).

3 The seven layers, from the bottom up

#	Layer	Its job	Examples	In the picture
1	**Physical**	carry raw bits	cables, fibre, radio	the road
2	**Data link**	a frame to the next device on the same network	Ethernet, Wi-Fi, MAC addresses, switches	the local driver
3	**Network**	addresses and routes across many networks	IP, routers	the post office
4	**Transport**	the right app (ports); for TCP, numbered pieces, resend what's lost	TCP, UDP	the shipping desk
5	**Session**	open, keep and close a conversation	often inside the app	the assistant
6	**Presentation**	the data's format, compression, encryption	text encoding, TLS's job	the translator
7	**Application**	the protocol your app speaks	HTTP, DNS	the director

Note: layer 7 is the protocol, not the app itself. Chrome isn't layer 7; HTTP is. To remember the order from 1 to 7: **Please Do Not Throw Sausage Pizza Away**.

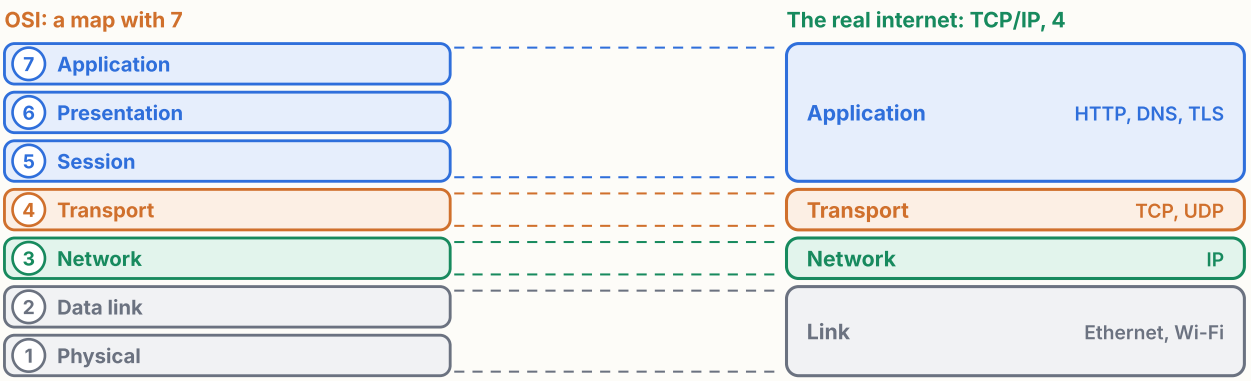


Remember 1 to 7: Please Do Not Throw Sausage Pizza Away. Note: layer 7 is HTTP, not the app (Chrome) itself.

Figure 2. The seven OSI layers with what each one handles, and the box that works at each: a switch at 2, a router at 3, load balancers at 4 or 7.

4 The honest part: the internet uses four

The real internet doesn't use seven separate layers. It uses the four from lesson 5. OSI is a map, used to name where something happens. The mapping is simple: layers 1 and 2 are the link layer, layer 3 is the network layer, layer 4 is transport, and layers 5, 6 and 7 are squeezed into the application layer. For example, TLS does the encryption job of layer 6, but it runs inside the app.



OSI 5-7 = application layer, 4 = transport, 3 = network, 1-2 = link. TLS does layer 6's job (encryption), inside the app.

Figure 3. OSI is a map; the internet runs the four-layer TCP/IP model. Layers 5 to 7 are the application layer, 4 is transport, 3 is network, 1 and 2 are the link layer.

5 Step by step: one request, layer by layer

1. **Layer 7.** Your browser writes an HTTP request: `GET /shoes`.
2. **Layer 6.** TLS encrypts it.
3. **Layer 4.** TCP adds ports: 51,234 to 443.
4. **Layer 3.** IP adds the from and to addresses.
5. **Layer 2.** A frame, addressed to your router's MAC address.
6. **Layer 1.** Radio waves over Wi-Fi.

Each router on the way unwraps only up to layer 3, reads the IP address, and sends it on in a new frame. Only the server unwraps all seven and reads `GET /shoes`.

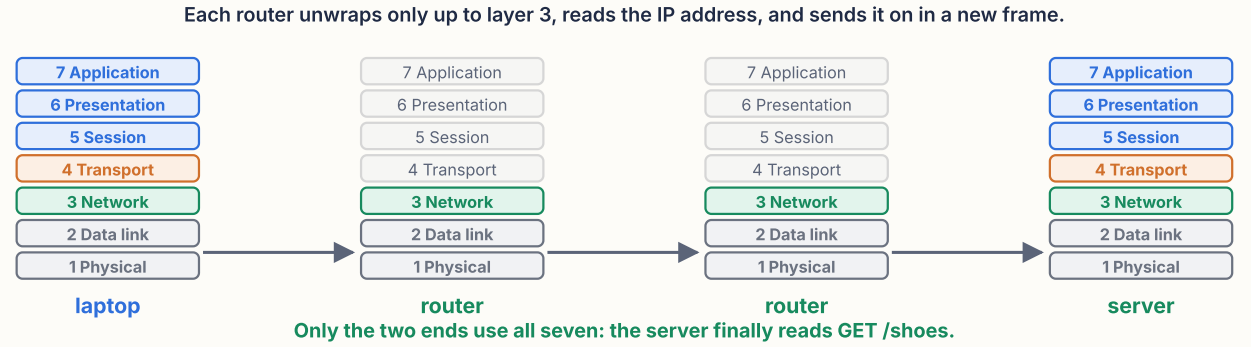
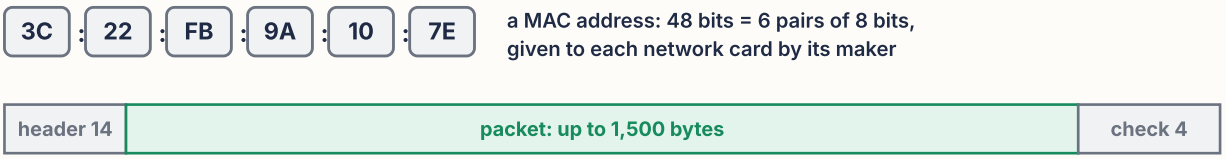


Figure 4. One request on its way: the laptop wraps it through all seven layers, each router unwraps only to layer 3, and only the server unwraps all seven.

6 Real numbers

A **MAC address** is 48 bits, written as six pairs, like `3C:22:FB:9A:10:7E`. Every network card is given one by its maker. An Ethernet frame adds **14 bytes** in front and **4** at the end, around a packet of up to 1,500 bytes, so the largest normal frame is **1,518 bytes**.



The largest normal Ethernet frame: 14 + 1,500 + 4 = 1,518 bytes (a VLAN tag adds 4 more).

Figure 5. Real numbers at layer 2: a MAC address is 48 bits, and an Ethernet frame adds a 14-byte header and a 4-byte check value around a packet of up to 1,500 bytes.

7 Why interviewers love layer numbers: load balancers

A **layer 4 load balancer** sees only IP addresses and ports. It's fast, and it can't read the request. A **layer 7 load balancer** reads the HTTP request itself: the path, the headers, the cookies. So it can send `/api` to one group of servers and `/images` to another. It does more work, but it's smarter.



Routing /api and /images to different servers needs the path, which is inside the HTTP request: layer 7.

Figure 6. Layer 4 versus layer 7 load balancing: a layer 4 box routes by IP address and port only; a layer 7 box reads the HTTP request, so it can route /api and /images to different servers.

8 What goes wrong: find the broken layer

Your laptop says: connected, no internet. Check from the bottom up. Layer 1: is the cable plugged in, is Wi-Fi on? Layer 2: did you join the right network? Layer 3: did you get an IP address? An address starting 169.254 means you didn't. Layer 4: is a firewall blocking the port? Layer 7: does the site itself return an error, like 500? The first layer that fails is your problem. Everything above it fails too, so don't start at the top.

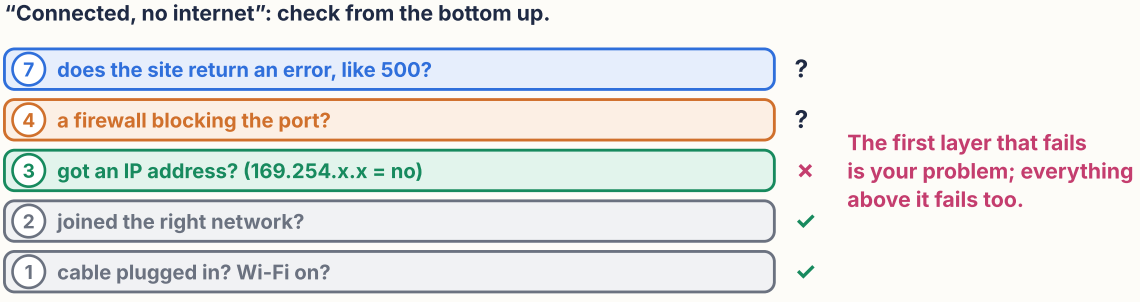


Figure 7. Troubleshooting with layers: check from layer 1 upwards; the first layer that fails is the problem, because every layer above it fails too.

PAUSE AND THINK

A load balancer sends every request for `/api` to one server group, and `/images` to another. Is it layer 4 or layer 7? Cover the answer and decide.

Answer. Layer 7. The path, /api or /images, is inside the HTTP request. A layer 4 box only sees addresses and ports, so it can't tell them apart. And with HTTPS, the layer 7 box must decrypt the request first.

TWO COMMON MISTAKES

- **"The internet runs on the OSI model."** No. It runs on TCP/IP's four layers; OSI is the reference map whose numbers we borrow.
- **"Layer 7 means the app."** No. It means the app's protocol, such as HTTP. A browser or a mobile app uses layer 7; it isn't layer 7.

RECAP IN THREE LINES

1. The OSI model has seven layers: physical, data link, network, transport, session, presentation and application.
2. The real internet uses four, and OSI names where things happen.
3. Switches work at layer 2, routers at 3, and load balancers at 4 or 7.

9 Interview questions: what to say in 30 seconds

Q1. What are the seven layers of the OSI model? BASIC

From the bottom: 1 physical (cables, radio, bits), 2 data link (frames on one network, MAC addresses, switches), 3 network (IP, routing between networks), 4 transport (TCP, UDP, ports), 5 session (the conversation), 6 presentation (format, compression, encryption), 7 application (the protocol the app speaks, such as HTTP or DNS).

Q2. How does the OSI model relate to the TCP/IP model the internet uses? BASIC

OSI is a reference model; the internet follows the four-layer TCP/IP model. OSI layers 1 and 2 are the link layer, 3 is the network layer, 4 is transport, and 5 to 7 are squeezed into the application layer. For example, TLS does layer 6's job, encryption, but it runs inside the application. We keep the OSI numbers as shorthand.

Q3. What is the difference between a switch and a router? BASIC

A switch works at layer 2: it forwards frames between devices on the same local network, using MAC addresses. A router works at layer 3: it forwards packets between networks, using IP addresses, unwrapping each frame and putting the packet in a new frame for the next hop.

Q4. When would you choose a layer 7 load balancer over a layer 4 one? SCENARIO

When routing needs the request's content: send /api and /images to different servers, route by header or cookie, or end TLS and add security checks. A layer 4 balancer only sees IP addresses and ports, so it is faster and simpler and suits plain TCP traffic, such as databases, or very high throughput.

Q5. Your laptop shows "connected, no internet". How do you debug it? SCENARIO

Bottom up. Layer 1: cable in, Wi-Fi on. Layer 2: joined the right network. Layer 3: got an IP address (169.254.x.x means DHCP failed) and a route. Then DNS and layer 4: is a firewall blocking the port? Layer 7: does the site itself answer with an error? The first failing layer is the problem, since everything above fails too.

Q6. Which layer does TLS belong to? TRICKY

It doesn't fit neatly, which is a good thing to say. Its job, encryption and agreeing a format, is OSI layer 6, but it runs on top of TCP inside the application, so in the TCP/IP model it is part of the application layer. That's why a layer 7 load balancer must terminate TLS before it can read the HTTP path.

Read more. ISO/IEC 7498-1 (the OSI basic reference model), RFC 1122 (the internet's own layers), RFC 3439 section 3 (“layering considered harmful”: why the internet is not strictly layered), IEEE 802.3 (Ethernet frames) and IEEE 802 (MAC addresses), RFC 3927 (169.254.x.x link-local addresses). The full written lesson is at backendroadmap.com.

Next: SD-N-007 · The TCP/IP model: what happens when you type a URL backendroadmap.com

What happens when you type a URL

the TCP/IP model in action

Watch the 9-min video

In one line: pressing Enter starts four conversations in order, DNS, the TCP handshake, the TLS handshake and HTTP, each about one round trip, so a cold page load waits about four round trips before the first byte; the error you see tells you which step failed.

WORDS YOU'LL MEET TODAY

URL: a web address: protocol, server name and path

DNS: the internet's phone book: a name to an IP address

DNS resolver: the server that looks names up for you, usually your provider's

Handshake: the opening exchange before any data

TLS: the encryption behind the S in HTTPS

Certificate: the server's proof of who it is

Round trip (RTT): a message there and the reply back

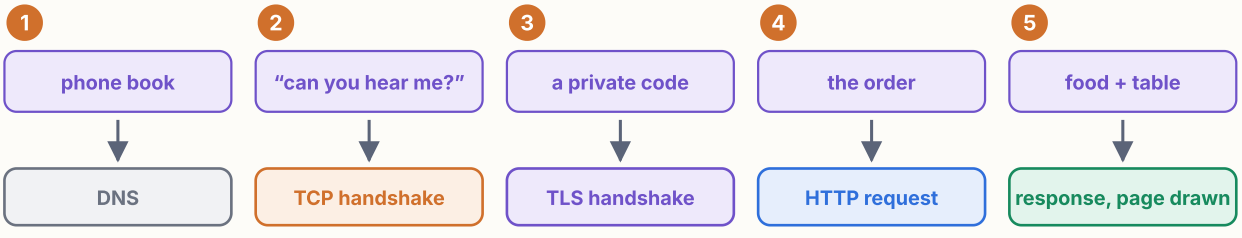
Cold start: a first visit, with nothing cached and no connection open

1 The question: what happens in a quarter of a second?

You type backendroadmap.com and press Enter. A quarter of a second later, the page is on your screen. In that blink, your browser held at least four separate conversations, using every layer from lesson 5. Today we follow them in order, one step at a time. It's also one of the most famous interview questions.

2 An everyday picture first: calling a new restaurant

You want to order food from a restaurant you've never called. You look up its number in a phone book. You dial, and you both check you can hear each other. You agree on a private code, so nobody listening can understand. You place the order. The food arrives, and you set the table. Every step must finish before the next one starts.

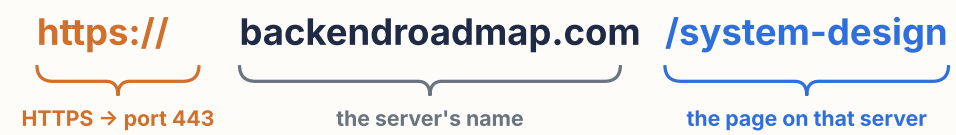


Calling a restaurant you've never called: every step must finish before the next one starts.

Figure 1. The analogy: looking up the number, checking you can hear each other, agreeing a private code, ordering and eating map onto DNS, the TCP handshake, the TLS handshake, the HTTP request and the response.

3 Step 0: read the URL

The browser reads the URL. `https:` means use HTTPS, so port 443. **backendroadmap.com** is the name of the server. And `/system-design` is the page you want on it.

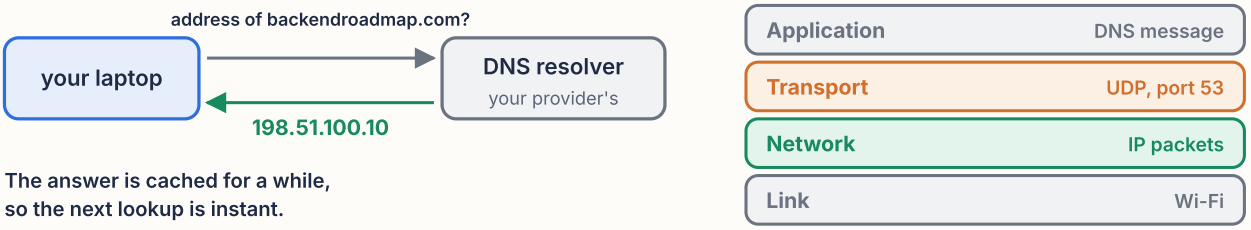


Step 0: the browser reads the URL. Packets need an IP address, not a name, so step 1 is DNS.

Figure 2. Anatomy of a URL: the protocol (HTTPS, so port 443), the server's name, and the path of the page you want.

4 Step 1: DNS, the phone book

Computers don't send packets to names; they need an IP address. Your laptop asks a **DNS resolver**, usually run by your internet provider: what's the address of backendroadmap.com? The resolver answers **198.51.100.10**. The answer is stored for a while (cached), so next time it's instant. Notice the layers: the question is a DNS message at the application layer, it travels in UDP to port 53, inside IP packets, over Wi-Fi. The same stack as every other message. DNS gets two full lessons, 15 and 16.



The question is an application-layer DNS message, in UDP to port 53, inside IP packets, over Wi-Fi: the same stack as everything else.

Figure 3. Step 1, DNS: the laptop asks a resolver for the server's IP address and caches the answer. The question travels through the same four layers as everything else.

5 Steps 2 and 3: the TCP and TLS handshakes

The TCP handshake. Your laptop sends a SYN: can we talk? The server replies SYN-ACK: yes, can you hear me? Your laptop answers ACK: yes. Now a connection exists (every message in detail: lesson 9).

The TLS handshake. Browser and server agree on keys for encryption, and the server proves who it is with a **certificate**. From now on, nobody in between can read or change the data (lessons 20 to 22).

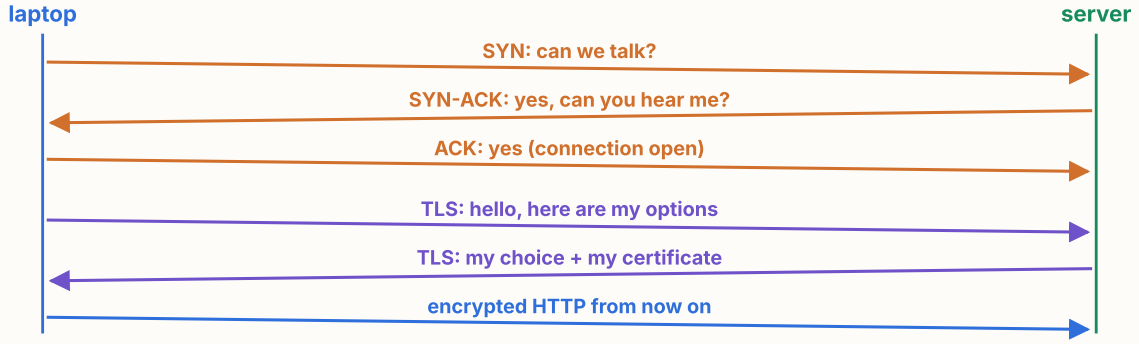


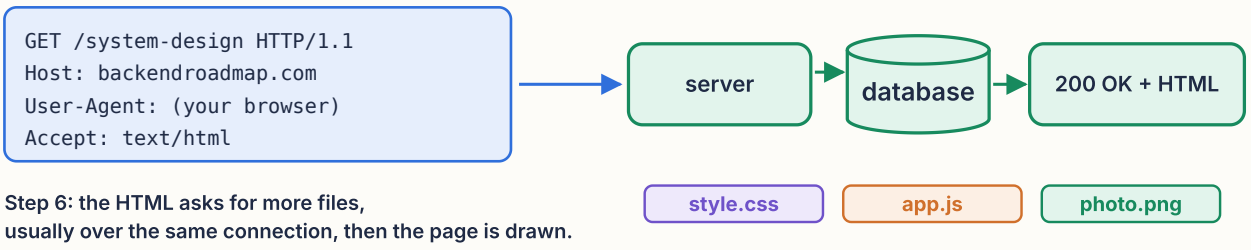
Figure 4. Steps 2 and 3: the TCP three-way handshake (SYN, SYN-ACK, ACK) opens a connection; the TLS 1.3 handshake agrees keys and proves the server's identity with a certificate.

6 Steps 4 to 6: HTTP, the server, the page

4. **The HTTP request.** `GET /system-design`, with a few headers, like which browser you use and which formats you accept.
5. **The server does its work.** Maybe it reads a database and builds the page, then replies: `200 OK`, and the HTML.

6. **The browser draws the page.** The HTML says it needs more: a style sheet, some scripts and images. The browser asks for those too, usually over the same connection, and draws the page.

So the four conversations are DNS, the TCP handshake, the TLS handshake and HTTP, and each one needs the one before it.



Steps 4 and 5: the HTTP request with a few headers; the server does its work and replies with a status and the page.

Figure 5. Steps 4 to 6: the HTTP request with its headers, the server's work and reply (200 OK and the HTML), then the extra files the page needs.

7 Real numbers: count the round trips

Say a round trip to the server takes 50 ms. DNS, if it's not cached: one round trip, often more. The TCP handshake: one. TLS 1.3: one more. And the HTTP request and reply: one more. That's **four round trips, about 200 ms, before the first byte** of the page arrives. Add the server's own work, say 50 ms, and you have the quarter of a second from the start.

Notice: most of that time isn't data; it's waiting for replies. That's why the speed of light, and the distance to the server, matter so much, and why CDNs, servers close to you, exist (lesson 28).

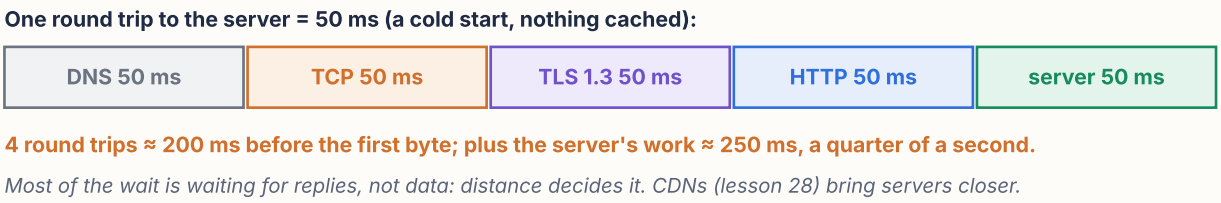
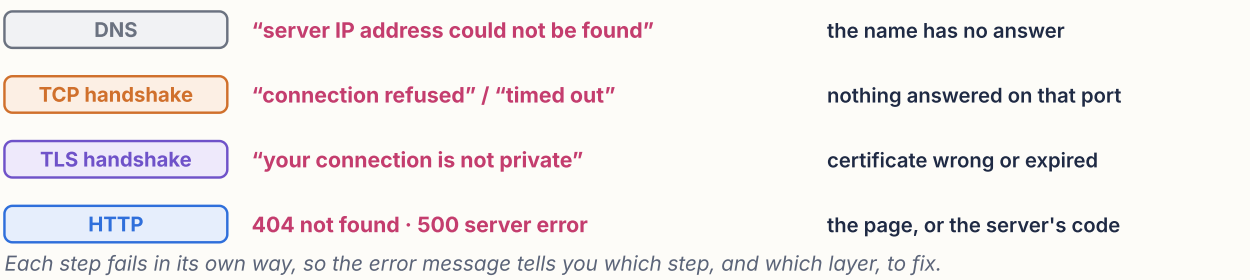


Figure 6. Counting round trips on a cold start: DNS, TCP, TLS 1.3 and HTTP each cost about one, so 200 ms of waiting before the first byte, plus the server's own work.

8 What goes wrong: the error names the step

- **DNS fails:** the browser says the server's IP address could not be found. The name has no answer.
- **The TCP handshake fails:** connection refused, or connection timed out. The address was found, but nothing answered on that port.
- **TLS fails:** your connection is not private. The certificate is wrong or expired.
- **HTTP fails:** everything else worked, and you get a page with a status code: 404 not found, or 500, a server error.

So reading the error tells you which step, and which layer, to fix.



Each step fails in its own way, so the error message tells you which step, and which layer, to fix.

Figure 7. What goes wrong: each of the four conversations has its own error, so the message tells you which step failed.

PAUSE AND THINK

One minute later, you open another page on the same site. It loads much faster. Which steps did the browser skip? Cover the answer and decide.

Answer. Usually three of the four. DNS is cached, so no lookup. And the connection is kept open, so no TCP or TLS handshake. Only the HTTP request remains: one round trip instead of four. That's why keeping connections open matters so much (lesson 29).

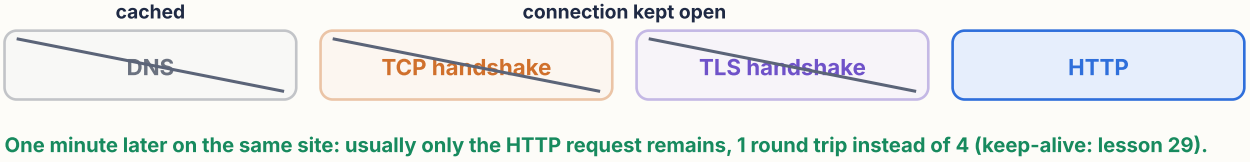


Figure 8. The pause-and-think answer: on a second page a minute later, DNS is cached and the connection is still open, so only the HTTP request remains.

TWO COMMON MISTAKES

- **"The browser goes straight to the server."** No. It first needs the address (DNS), then a connection (TCP), then encryption (TLS); only then can it ask for the page.
- **"A faster server fixes a slow page."** Often not. On a cold load most of the time is round trips; fewer and shorter round trips (caching, keep-alive, CDNs) usually help more.

RECAP IN THREE LINES

1. Typing a URL starts four conversations: DNS, the TCP handshake, the TLS handshake and HTTP.
2. Each costs about one round trip, so distance and caching decide most of the wait.
3. The error you see tells you which step failed.

9 Interview questions: what to say in 30 seconds

Q1. What happens when you type a URL into a browser and press Enter? BASIC

The browser parses the URL, resolves the name to an IP address with DNS (usually from a cache), opens a TCP connection with the three-way handshake, and for HTTPS does a TLS handshake that agrees keys and checks the certificate. Then it sends the HTTP request, the server responds with the HTML, and the browser fetches the other files, CSS, scripts and images, usually over the same connection, and renders the page.

Q2. Which protocols, at which layers, are involved? BASIC

DNS (application layer, normally over UDP port 53) to find the address; TCP (transport) for the connection; TLS for encryption, which runs above TCP inside the application; HTTP (application) for the request and response; IP (network) carries every packet; and Ethernet or Wi-Fi (link) moves them hop by hop.

Q3. Why is the first visit to a site slower than the second? SCENARIO

A cold start pays for DNS, the TCP handshake and the TLS handshake before the request, about four round trips in all. A minute later the DNS answer is cached and the connection is usually still open (keep-alive), so only the HTTP request and response remain, often one round trip. Browser caching of files saves even more.

Q4. A user sees “connection timed out”. Which step failed, and what would you check? SCENARIO

DNS worked, because there is an address; the TCP handshake failed, because nothing answered on that port. Check that the server is up and listening on 443, firewall and security-group rules, and the load balancer's health. If it said the IP address could not be found, it would be DNS; “not private” would be TLS; a 500 would be the application.

Q5. How long does a cold page load take, at minimum, if the round-trip time is 50 ms? TRICKY

About four round trips before the first byte: DNS (one or more if not cached), the TCP handshake (one), TLS 1.3 (one) and the HTTP request (one): roughly 200 ms, plus the server's processing time. TLS 1.2 costs two round trips, and HTTP/3 over QUIC combines the transport and TLS handshakes into one.

Q6. Most of that time isn't data. What does that mean for design? TRICKY

Latency, not bandwidth, dominates small requests, and latency is mostly distance. So reduce round trips (keep connections open, cache DNS and files, use HTTP/2 or 3) and shorten them (put content close to users with a CDN). Making the server faster helps only the small part of the time it owns.

Read more. RFC 3986 (URI syntax), RFC 1034/1035 (DNS), RFC 9293 (TCP, the three-way handshake), RFC 8446 (TLS 1.3, one round trip), RFC 9110 (HTTP semantics), RFC 9000 (QUIC). 198.51.100.10 is from a range reserved for documentation (RFC 5737). The full written lesson is at backendroadmap.com.

Next: SD-N-008 · TCP part 1: sequence numbers and ACKs backendroadmap.com

TCP part 1: sequence numbers and ACKs

how TCP never loses your data

Watch the 10-min video

In one line: TCP numbers every byte; the receiver answers with an ACK that names the next byte it expects; a lost segment is sent again after three duplicate ACKs or a timeout.

WORDS YOU'LL MEET TODAY

- **Packet**: a small piece of data on the internet
- **Byte**: a tiny unit of data, about one letter
- **Sequence number**: where that piece belongs
- **Duplicate ACK**: the same ACK again: "there's a gap"
- **Timeout**: "I waited too long"
- **Fast retransmit**: resend after 3 duplicate ACKs
- **Router**: a machine that forwards packets
- **Segment**: one piece of data TCP sends
- **ACK**: a reply that says "got it"
- **Cumulative ACK**: one ACK confirms all before it
- **Retransmit**: send the same piece again
- **Round trip**: data going there + the ACK back

1 The problem: the network makes no promises

You send a photo to a friend on WhatsApp. To travel, it is cut into small pieces called **packets**, and each one crosses the internet on its own. On the way, one packet is lost. One arrives twice. Two arrive in the wrong order. And yet, your friend sees a perfect photo. Who fixes this? A protocol called **TCP**, the Transmission Control Protocol.

- **Lost.** A busy **router**, the machine that forwards packets, can simply throw one away.
- **Reordered.** Packets can take different roads, so a later one can overtake an earlier one.
- **Duplicated.** Sometimes a packet is sent twice.

TCP runs on the sender and the receiver only. It can't fix the network; it works around these three problems.

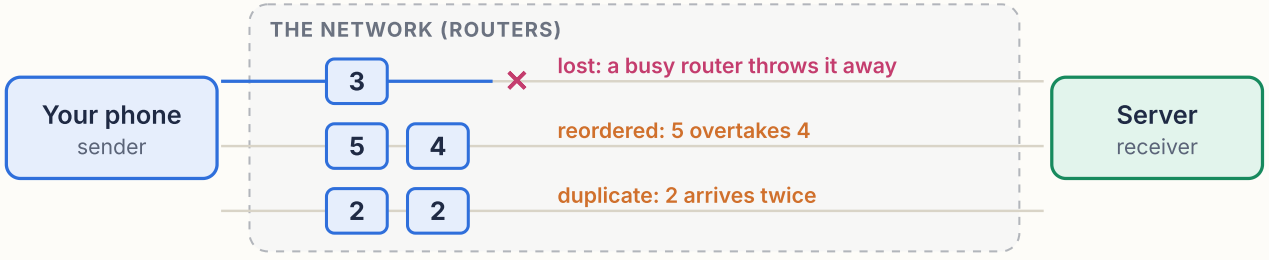


Figure 1. The network makes no promises. TCP runs only on the two ends and works around all three problems.

2 An everyday picture first: a letter by post

You post a five-page letter to a friend, each page in its own envelope. The post office usually works, but sometimes an envelope goes missing, and the envelopes arrive in a random order.

- **You numbered the pages**, so your friend sorts them. If page 2 arrives twice, they throw one copy away.
- **Page 4 never arrives.** Your friend sees the gap: 5 is here, 4 is not. So they send a postcard: *"I have pages 1 to 3. Send page 4 next."* That postcard is the idea of an ACK.
- **No postcard comes?** After a few days, you post the page again. Waiting too long is a **timeout**. Posting again is a **retransmit**.

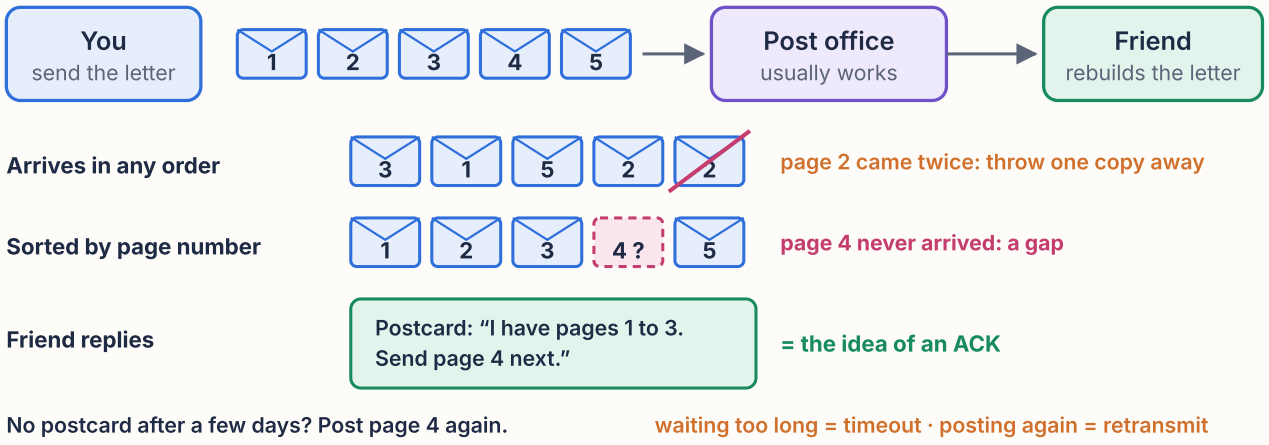


Figure 2. The everyday picture: page numbers to sort, a gap to ask about, silence to act on.

3 Now the real thing

In the letter story	In TCP
You, posting the letter	the **sender**: your phone
Your friend	the **receiver**: the server
One envelope	a **segment**: your data plus a small TCP header, inside an IP packet
The page number	the **sequence number**
The postcard "send page 4 next"	the **ACK**
Waiting a few days / posting again	**timeout** / **retransmit**

One twist: TCP doesn't number pages, it numbers bytes. A byte is a tiny unit of data, about one letter of text, and every byte gets its own number. Real numbers: your phone sends 3,000 bytes as three segments of 1,000 bytes. The first holds bytes 1 to 1,000, the second 1,001 to 2,000, the third 2,001 to 3,000. Each segment carries one number in its header, the number of its **first** byte, so the sequence numbers are **1, 1,001 and 2,001**.

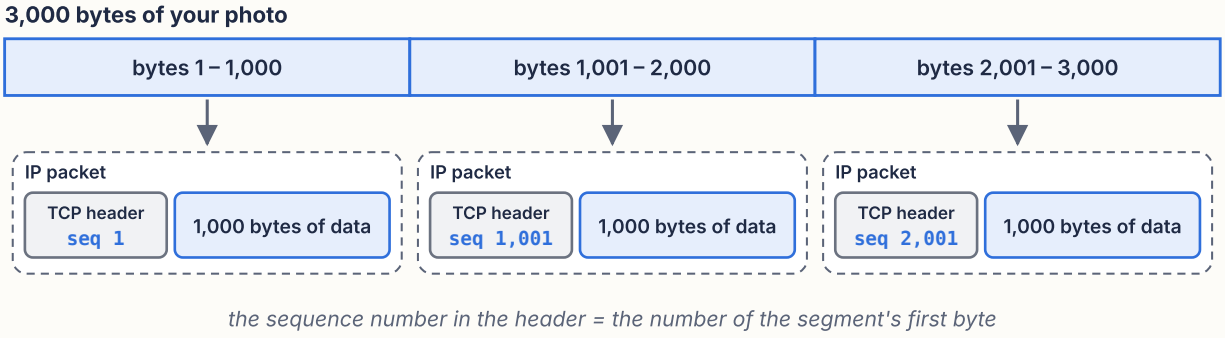


Figure 3. TCP numbers bytes, not pages: each segment carries the number of its first byte.

One honest detail. Real connections start counting from a random number (agreed in the handshake, lesson 9), but tools such as Wireshark show them from 1, and so do these notes.

4 The ACK: the most important idea today

ACK is short for **acknowledgement**: "I confirm I received it." The receiver sends ACKs back to the sender. Here is the rule, slowly:

```
ACK number = the next byte the receiver expects
```

The server receives bytes 1 to 1,000 and replies **ACK 1,001**. In plain words: *"I have everything up to 1,000. Send me 1,001 next."* Like a parcel tracking page, it shows one line: everything up to here has arrived. After the second segment the ACK becomes 2,001; after the third, 3,001. No gaps.

This is a **cumulative acknowledgement**: one ACK confirms everything before it. So if an ACK is lost on the way back, the next one covers it.

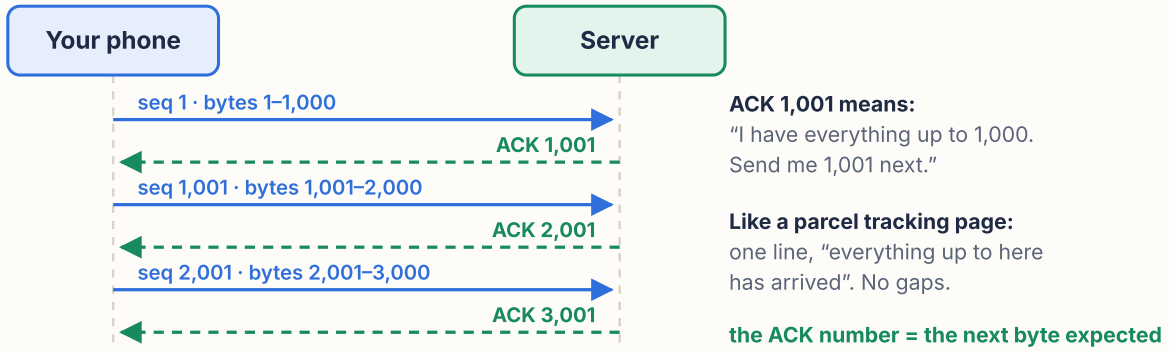


Figure 4. The ACK number is the next byte the receiver expects. One cumulative ACK confirms everything before it, so if one ACK is lost on the way back, the next one covers it.

5 Step by step: one segment is lost

The phone sends five segments, with sequence numbers 1, 1,001, 2,001, 3,001 and 4,001.

1. **Segment 1 arrives.** The server replies ACK 1,001. All good.
2. **Segment 1,001 is lost.** A busy router dropped it. Nothing arrives, so the server says nothing.
3. **Segment 2,001 arrives**, but bytes 1,001 to 2,000 are still missing. The server keeps the new segment in memory and repeats ACK 1,001: "I still need 1,001." An ACK that repeats the same number is a **duplicate ACK**. It means: there is a gap.
4. **Segment 3,001 arrives**, and the server sends ACK 1,001 again.
5. **Segment 4,001 arrives**, ACK 1,001 again. The phone now has **three duplicate ACKs**. It reasons: later segments get through, so one is missing. It resends segment 1,001 right away, without waiting. This is **fast retransmit**.
6. **The missing segment arrives.** The server now has every byte, 1 to 5,000, so the ACK jumps straight to **5,001**.

6 The real numbers, as a table

Step	Phone sends	Arrives?	Server now holds	ACK sent back
1	seq 1 (bytes 1–1,000)	yes	1–1,000	**1,001**
2	seq 1,001 (1,001–2,000)	**lost**	1–1,000	nothing
3	seq 2,001 (2,001–3,000)	yes	1–1,000, plus 2,001–3,000 kept aside	**1,001 (dup 1)**
4	seq 3,001 (3,001–4,000)	yes	1–1,000, plus 2,001–4,000 kept aside	**1,001 (dup 2)**
5	seq 4,001 (4,001–5,000)	yes	1–1,000, plus 2,001–5,000 kept aside	**1,001 (dup 3)**
	three duplicate ACKs → fast retransmit of seq 1,001			
6	seq 1,001 again	yes	1–5,000, no gaps	**5,001**

Read the last column: 1,001, then nothing, then 1,001 three times, then 5,001. The repeated number points straight at the gap.

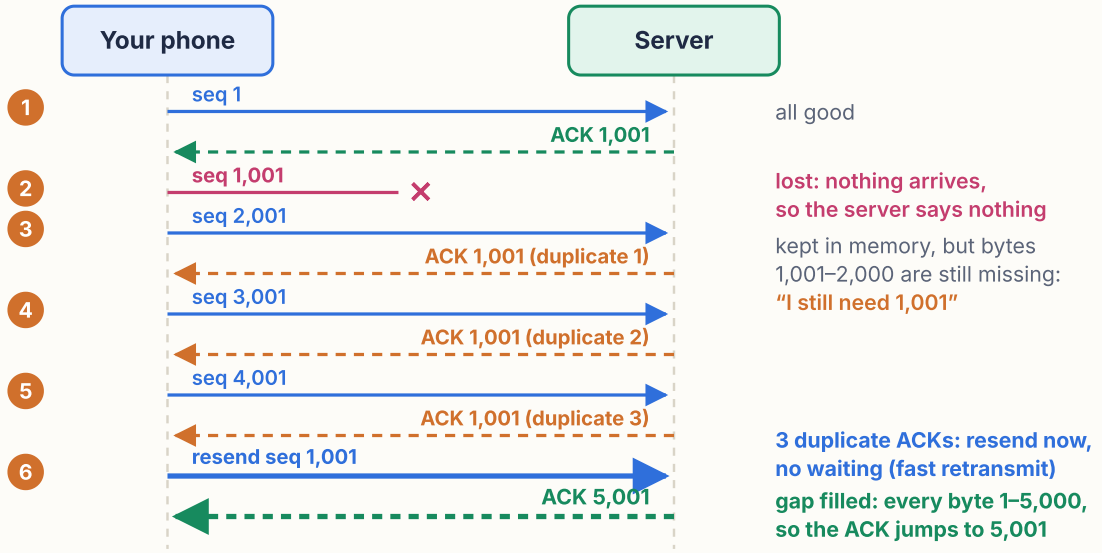


Figure 5. One lost segment: three duplicate ACK 1,001s, a fast retransmit, then ACK 5,001.

7 What goes wrong: the last segment is lost

If the lost segment was the last one, no later segments arrive, so there are no duplicate ACKs. Then the second safety net works: a **timer**. With every segment it sends, the sender starts a timer a little longer than one **round trip** (data going, and the ACK coming back). If it runs out with no ACK, that is a timeout: the sender retransmits, and **doubles the timer**, in case the network is struggling.

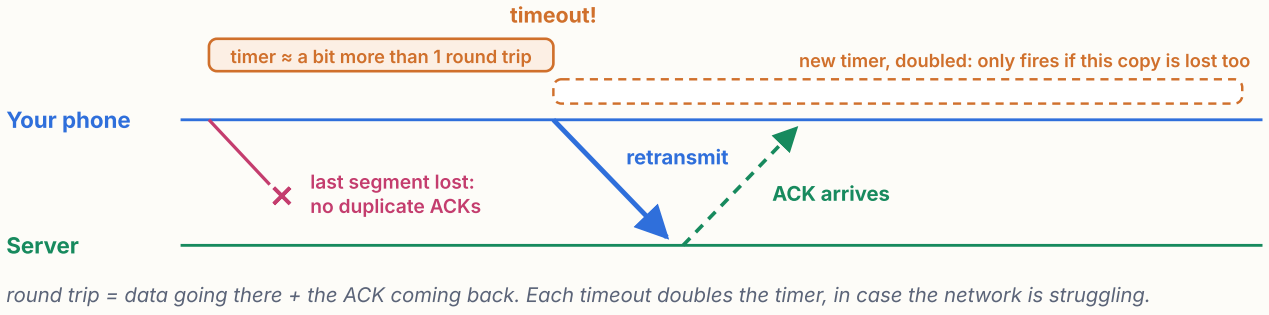


Figure 6. The second safety net. If no ACK comes before the timer runs out, the sender retransmits and doubles the timer.

For the curious. The timer is called the retransmission timeout (RTO). It is computed from measured round trips, with a floor (200 ms on Linux; the standard, RFC 6298, suggests 1 s). Real stacks run one timer for the oldest unacknowledged data rather than one per segment, and receivers often send one ACK for every two segments. None of this changes the logic: ACKs are cumulative.

PAUSE AND THINK

Four segments, starting at 1, 1,001, 2,001 and 3,001. The one at 2,001 is lost; the others arrive. Which ACK does the server keep sending? And after the resend arrives? Cover the answer and decide.

Answer. The server has bytes 1 to 2,000, so it keeps sending **ACK 2,001**. After the resend it has everything up to 4,000, so: **ACK 4,001**. If you got that, you understand ACKs.

Bonus: only one segment comes after the lost one, so there is only one duplicate ACK, not three. With the classic rule, this resend comes from the timer.

THREE COMMON MISTAKES

- **“Sequence numbers count segments.”** No, they count bytes. That's why they jump by 1,000.
- **“TCP guarantees fast delivery.”** No. It guarantees your data arrives complete and in order, or you get an error. Resending takes time, so on a weak mobile signal a page loads slowly, but nothing is lost.
- **“An ACK means the app handled the data.”** No. The receiver's operating system sends the ACK when the bytes reach its memory; the app may not have read them yet. That's why apps add their own confirmations, like WhatsApp's ticks, or a UPI success screen.

RECAP IN THREE LINES

1. TCP numbers every byte, and each segment carries the number of its first byte.
2. The receiver ACKs the next byte it expects.
3. A lost segment is resent after three duplicate ACKs, or a timeout.

8 Interview questions: what to say in 30 seconds

Q1. How does TCP make sure data is never lost? BASIC
TCP numbers every byte. The receiver sends cumulative ACKs with the next byte it expects, and uses sequence numbers to reorder data and drop duplicates. A lost segment is retransmitted after three duplicate ACKs, called fast retransmit, or when the retransmission timer expires.

Q2. The server sends “ACK 1,001”. What exactly does that mean? BASIC
“I have every byte up to 1,000; send me 1,001 next.” It is the next byte expected, not the last one received, and it is cumulative: it confirms everything before it, so a lost ACK is covered by the next.

Q3. Five 1,000-byte segments, the second is lost. Walk through the ACKs. SCENARIO
ACK 1,001; then nothing for the lost segment; then ACK 1,001 three more times as 2,001, 3,001 and 4,001 arrive and are kept aside. Three duplicates trigger fast retransmit of 1,001; the server then has bytes 1 to 5,000 and answers ACK 5,001.

Q4. What if the lost segment is the last one? SCENARIO
No later segments, so no duplicate ACKs. The retransmission timer covers it: a little longer than one measured round trip. When it runs out with no ACK, the sender retransmits and doubles the timer (exponential backoff).

Q5. Why wait for three duplicate ACKs, not one? TRICKY
Reordering also causes a duplicate ACK or two with nothing lost. Three duplicates are strong evidence of a real loss while later data still gets through, so the sender resends without waiting for the slower timeout.

Q6. The server's TCP ACKed your request. Has the app processed it? TRICKY
Not necessarily. The receiver's operating system sends the ACK when the bytes reach its memory; the app may not have read them yet, or may crash first. That's why apps add their own confirmations, like WhatsApp's ticks or a UPI success screen.

Read more. RFC 9293 (TCP), RFC 5681 (fast retransmit, duplicate ACKs), RFC 6298 (the retransmission timer). The full written lesson is at backendroadmap.com.

Next: SD-N-009 · TCP part 2: the three-way handshake backendroadmap.com

TCP part 2: the three-way handshake

SYN, SYN-ACK, ACK

Watch the 9-min video

In one line: before any data, TCP sends three messages: SYN, SYN-ACK and ACK. Each side sends a random starting number and the other confirms it with that number plus one; it costs one round trip, and a lost SYN waits a full second or more.

WORDS YOU'LL MEET TODAY

Connection: two programs agreeing to talk, with a counter on each side

ISN: initial sequence number: that random starting number

Flag: one bit in the TCP header that marks the kind of message

MSS: maximum segment size: the most data in one segment, often 1,460 bytes

SYN: synchronise: the message that says here's my starting number

SYN-ACK: the server's reply: its own SYN and an ACK in one segment

ESTABLISHED: the state once the handshake is done and data can flow

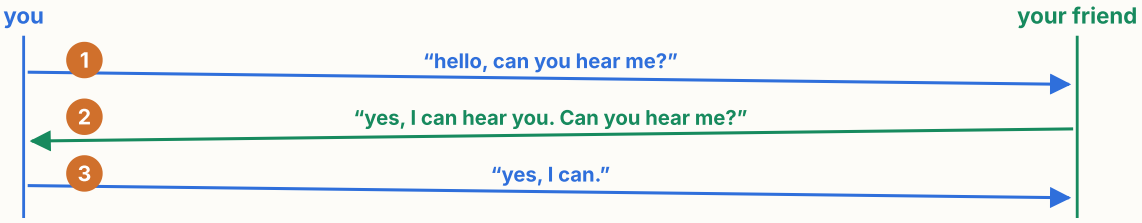
RST: reset: a flag meaning no, or stop, this connection

1 The question: three tiny messages, no data

Before your phone sends a single byte of your photo, it sends three tiny messages to the server. No data at all. In lesson 8 we saw that TCP numbers every byte, but each side starts counting from a random number. So before any data moves, both sides must tell each other where they start. That's the three-way handshake.

2 An everyday picture first: a call on a bad line

You call a friend on a bad line. You say: hello, can you hear me? Your friend says: yes, I can hear you. Can you hear me? You say: yes, I can. Now both of you know that your voice reaches them and theirs reaches you. Only then does the real conversation start. Three messages; two would not be enough, because your friend wouldn't know that you can hear them.



After three messages both know: my voice reaches you, and your voice reaches me. With only two, your friend still doesn't know that you can hear them.

Figure 1. The analogy: a call on a bad line takes three messages before both people know that each voice reaches the other.

3 The real thing, one message at a time

The server is waiting on port 443: in TCP's words, it is **listening**. Your phone picks a random starting number, say 4,000.

1. **SYN.** Your phone sends a segment with the SYN flag set and sequence number 4,000: I want to talk, and my bytes start at 4,000.

2. **SYN-ACK.** The server picks its own random number, say 9,000. Its reply has both flags set: SYN with sequence number 9,000, and ACK 4,001. Got your number, and my bytes start at 9,000.
3. **ACK.** Your phone replies ACK 9,001: got your number too. Both sides are now **ESTABLISHED**.

Why 4,001 and not 4,000? The rule from lesson 8: the ACK is the next number the receiver expects, and the SYN itself uses up one number. So the first byte of your photo carries sequence number 4,001, and the first byte of the server's reply 9,001. Two directions, two counters, both confirmed.



Figure 2. The three-way handshake with real numbers: each side sends its random starting number, and the other confirms it with that number plus one.

#	From → to	Flags	seq	ack
1	phone → server	SYN	4,000	-
2	server → phone	SYN + ACK	9,000	4,001
3	phone → server	ACK	4,001	9,001

Read the numbers: each ACK is the other side's number, plus one.

4 Why three messages, and why random numbers?

Each direction needs its own number, sent and confirmed. That's four jobs. But the server sends its SYN and its ACK in the same segment: four jobs, three messages.

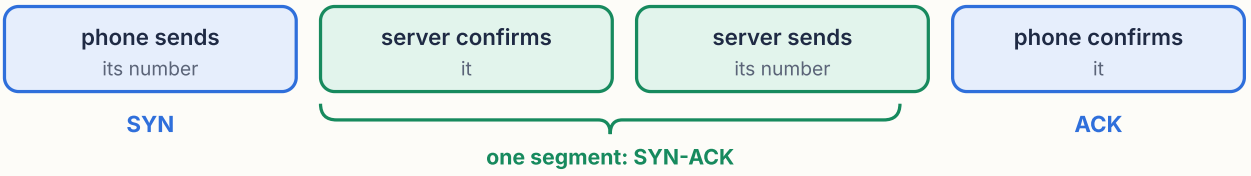


Figure 3. Why three messages: four jobs (each side sends its number, the other confirms it), and the server combines its confirmation and its own number in one SYN-ACK.

Random starting numbers for two reasons. A late packet from an old connection on the same ports won't be mistaken for new data. And an attacker can't easily guess the numbers to fake a connection.

5 Options, and real numbers for time

The SYN also carries options. The most important is the **maximum segment size** (MSS): how much data fits in one segment. On normal Ethernet or Wi-Fi it's 1,460 bytes: the 1,500-byte packet from lesson 6, minus 20 bytes of IP header and 20 bytes of TCP header.

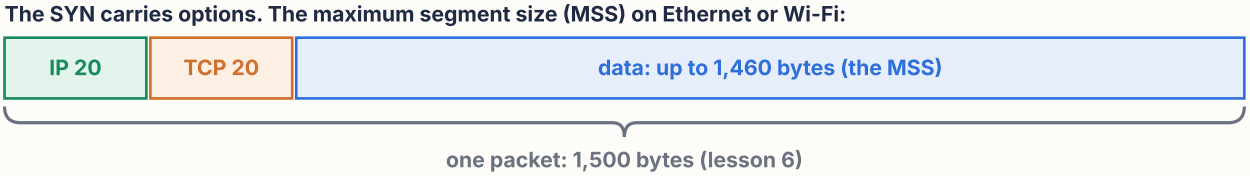
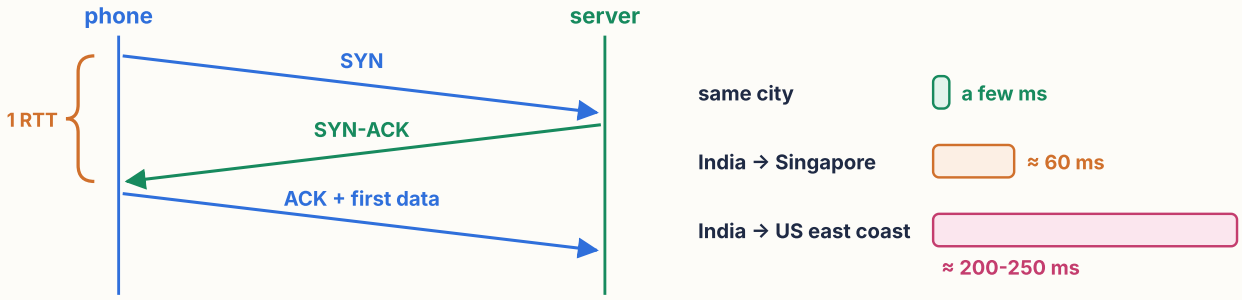


Figure 4. The SYN's most important option, the maximum segment size: 1,500 bytes per packet minus 20 bytes of IP header and 20 of TCP header leaves 1,460 bytes of data.

The handshake costs **one round trip** before any data; your phone can send its first data together with the final ACK. In the same city a round trip takes a few milliseconds, India to Singapore about 60, India to the US east coast about 200 to 250. You pay it once per connection, which is why opening connections is expensive and reusing them matters (lesson 29).



The handshake costs one round trip, paid once per connection: that's why reusing connections matters (lesson 29).

Figure 5. The handshake costs one round trip before any data (the first data can ride with the final ACK), and how long one round trip is depends on distance.

6 What goes wrong

- **No reply.** The server is down, or a firewall silently drops the SYN. Your phone waits, then resends; on Linux the first wait is 1 second and then it doubles: 2, 4, 8 and so on. After six retries, about two minutes in all, you see *connection timed out*.
- **Nobody listening.** The server is up, but no program is on that port. It answers straight away with a reset, a segment with the **RST** flag: *connection refused*. Fast, and honest.

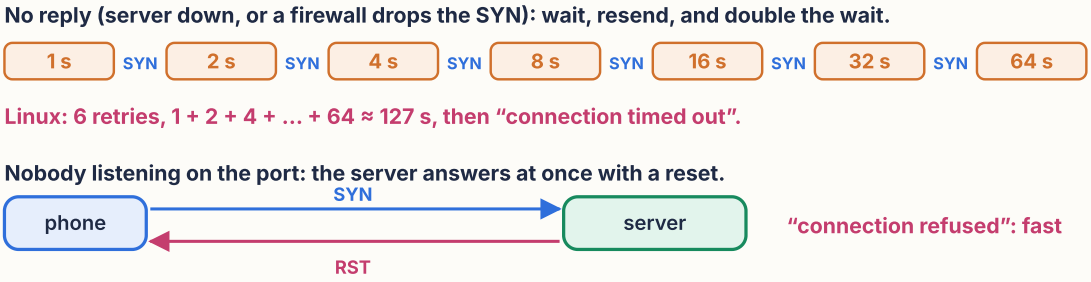


Figure 6. What goes wrong: with no reply the client resends the SYN, doubling the wait each time, until "connection timed out"; with no program on the port the server sends RST and you see "connection refused".

A SYN flood. An attacker sends millions of SYNs and never the final ACK. The server keeps a slot for each half-open connection, the queue fills, and real users can't get in. The usual fix is **SYN cookies**: the server keeps no slot, and hides what it needs inside its own starting number.

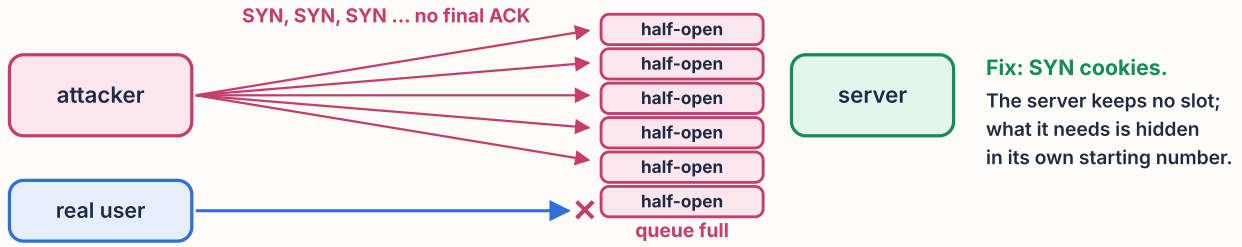


Figure 7. A SYN flood: an attacker sends SYNs and never the final ACK, filling the server's queue of half-open connections so real users can't connect. SYN cookies keep no slot until the ACK arrives.

PAUSE AND THINK

The round trip to a server is 100 ms. Your phone's first SYN is lost. How long until the connection is established? Cover the answer and decide.

Answer. About 1.1 seconds. Your phone waits one full second for a reply that never comes, then resends the SYN, and the handshake takes one round trip, 100 ms. One lost packet, and the page feels stuck for a second.

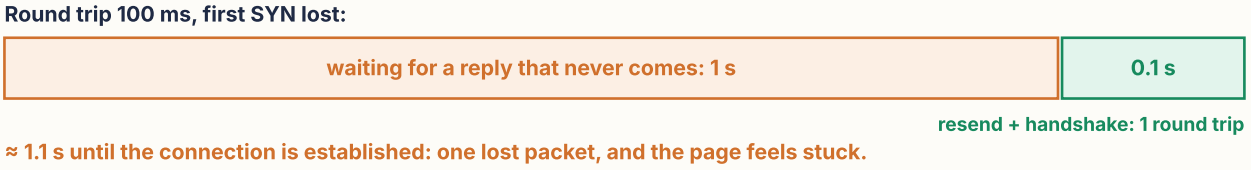


Figure 8. The pause-and-think answer: a lost first SYN costs the one-second retransmission wait, then one round trip, about 1.1 seconds in all.

THREE COMMON MISTAKES

- **"There's a handshake for every request."** No. It happens once per connection, and many requests can reuse it (lesson 29).
- **"The handshake makes the connection secure."** No. It only agrees numbers. Encryption is TLS, which starts after it (lesson 21).
- **"ACK 4,001 means 4,001 bytes were received."** No. It means: the next number I expect is 4,001.

RECAP IN THREE LINES

1. Before any data, TCP sends three messages: SYN, SYN-ACK and ACK.
2. Each side sends a random starting number, and the other confirms it with that number plus one.
3. It costs one round trip, and a lost SYN waits a full second, or more.

7 Interview questions: what to say in 30 seconds

Q1. Explain the TCP three-way handshake. BASIC

The client sends a SYN with its initial sequence number. The server replies with a SYN-ACK: its own initial sequence number, and an acknowledgement of the client's number plus one. The client acknowledges the server's number plus one, and both sides are established. Data can start with, or right after, that final ACK.

Q2. Why three messages and not two? BASIC

Each direction needs its own starting number sent and confirmed, which is four jobs. The server combines its acknowledgement and its own SYN in one segment, so it takes three. With only two, the server would never learn that the client received its number, so the server-to-client direction would be unconfirmed.

Q3. A client sees “connection refused” on one server and “connection timed out” on another. What's the difference? SCENARIO

Refused means the host answered with an RST: it is up, but no program is listening on that port (or a firewall actively rejects). Timed out means the SYN got no reply at all: the host is down, unreachable, or a firewall silently drops the packet, so the client kept retrying with doubling waits until it gave up.

Q4. What is a SYN flood, and how do you defend against it? SCENARIO

An attacker sends many SYNs, often from fake addresses, and never completes the handshake, so the server's queue of half-open connections fills and real clients can't connect. Defences: SYN cookies (the server keeps no state and encodes it in its initial sequence number), larger backlog queues, rate limiting, and absorbing it upstream at a load balancer or DDoS protection service.

Q5. Why are initial sequence numbers random instead of starting at 0? TRICKY

So a delayed segment from an earlier connection on the same address and ports doesn't fall inside the new connection's numbers and get accepted as data, and so an off-path attacker can't guess the numbers to inject data or fake a connection (blind spoofing).

Q6. How does the handshake affect latency, and what reduces it? TRICKY

It costs one round trip before the request can be sent, paid for every new connection, plus TLS on top. Reuse connections (keep-alive, connection pools), put servers closer (CDNs), and use TCP Fast Open or QUIC (HTTP/3), which can send data in the first flight on repeat connections.

Read more. RFC 9293 (TCP: the three-way handshake, states, RST), RFC 6528 (defending against sequence number attacks), RFC 4987 (SYN flooding and SYN cookies), RFC 6298 (the 1-second initial retransmission timeout), RFC 7413 (TCP Fast Open), and Linux tcp(7) (tcp_syn_retries, 6 by default). The full written lesson is at backendroadmap.com.

Next: SD-N-010 · TCP part 3: flow and congestion control backendroadmap.com

TCP part 3: flow and congestion control

how fast should TCP send?

Watch the 9-min video

In one line: the receive window stops TCP drowning the receiver, the congestion window stops it drowning the network; TCP keeps the smaller in flight, starts small, doubles every round trip, and cuts back on loss, so speed is at most window ÷ round trip.

WORDS YOU'LL MEET TODAY

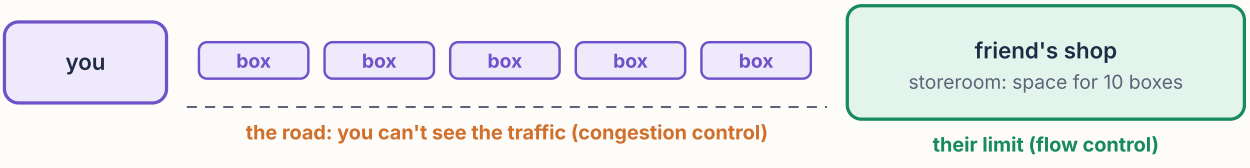
- **Window**: how many bytes may be on the way, not yet acknowledged
- **Congestion window (cwnd)**: the sender's own guess of what the network can take
- **Congestion control**: not drowning the network
- **Throughput**: how many bytes arrive per second
- **Receive window (rwnd)**: the receiver's free buffer space, sent in every ACK
- **Flow control**: not drowning the receiver
- **Slow start**: begin with about 10 segments and double every round trip
- **Window scaling**: a SYN option that allows windows far bigger than 64 KB

1 The question: who decides how fast?

You start a big download. The speed starts low, then climbs. On a crowded café Wi-Fi it suddenly drops, then slowly recovers. Nobody is turning a knob: TCP is deciding, every round trip, how fast to send. It must never drown two things: the receiver, and the network.

2 An everyday picture first: boxes to a friend's shop

You send boxes to a friend's shop by van. Sending one box and waiting for a thank-you each time takes forever, so you send many at once. But two things limit you. The shop's storeroom: your friend says, I have space for ten boxes. And the road: you can't see the traffic, so you start with a few vans, add more while they arrive on time, and send fewer when one gets stuck in a jam. The storeroom is **flow control**; the road is **congestion control**. TCP obeys both.



Send many boxes at once, but never more than the storeroom holds, and fewer when the road jams.

Figure 1. The analogy: boxes to a friend's shop. The storeroom is the receiver's limit (flow control); the road you can't see is the network's limit (congestion control).

3 The window: many segments in flight

Why not wait for each ACK? With a 100 ms round trip, one 1,460-byte segment per round trip is about 15 KB a second. So TCP keeps many segments in flight before their ACKs come back. The **window** is how many bytes may be on the way at the same time.

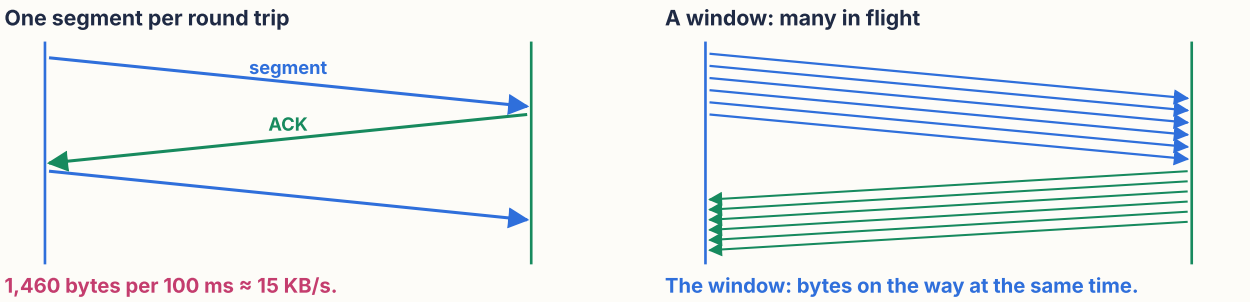


Figure 2. Waiting for each ACK sends one segment per round trip, about 15 KB/s at 100 ms; a window keeps many segments in flight before their ACKs return.

4 Flow control: the receive window

In every ACK, the receiver also writes the **receive window**: I have 64,000 bytes of free space, don't send more than that. If the app on the receiver reads slowly, its buffer fills and the window shrinks. At zero, the sender stops, and checks back now and then until there's space again.

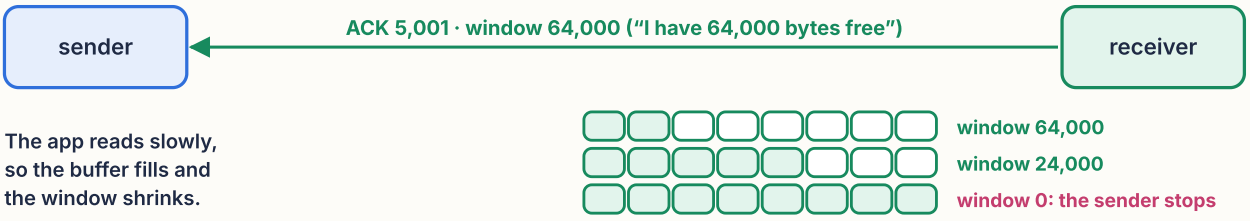
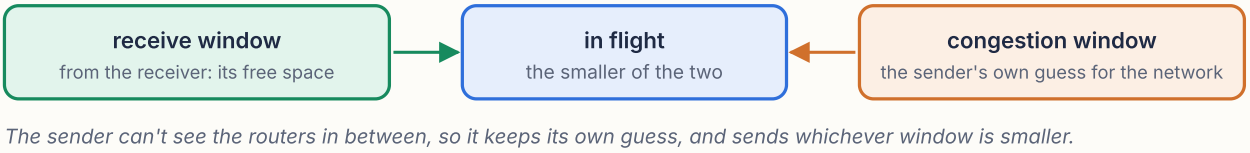


Figure 3. Flow control: every ACK carries the receive window, the receiver's free buffer space. A slow app lets the buffer fill and the window shrink, down to zero.

5 Congestion control: the sender's guess

The sender can't see the routers in between, so it keeps its own guess, the **congestion window**, and sends whichever is smaller: the receiver's window or its own guess.



The sender can't see the routers in between, so it keeps its own guess, and sends whichever window is smaller.

Figure 4. TCP's two limits: the receive window (flow control) and the congestion window (congestion control). The sender keeps the smaller of the two in flight.

Slow start. Modern TCP begins with ten segments, about 14 KB. Every round trip in which all the ACKs come back, it doubles: 10, 20, 40, 80. It isn't slow for long: it grows fast until a segment is lost, or it reaches a limit it learned before.

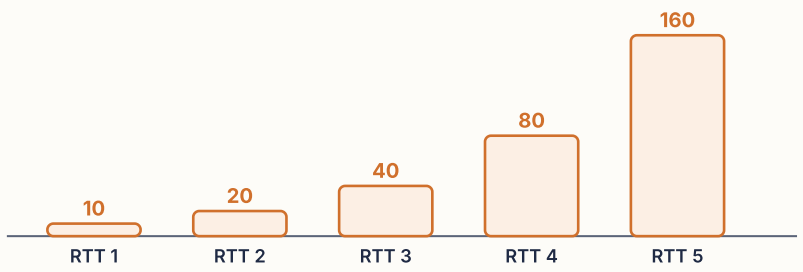


Figure 5. Slow start: the congestion window begins at 10 segments, about 14 KB, and doubles every round trip: 10, 20, 40, 80, 160.

Then it grows carefully, one more segment per round trip. When a loss shows up through three duplicate ACKs (lesson 8), it cuts the window; classic TCP cuts it in half, then grows again. Drawn over time, that's a sawtooth: TCP feeling for the limit of the road. After a timeout, with no ACKs at all, it drops to one segment and slow-starts again.

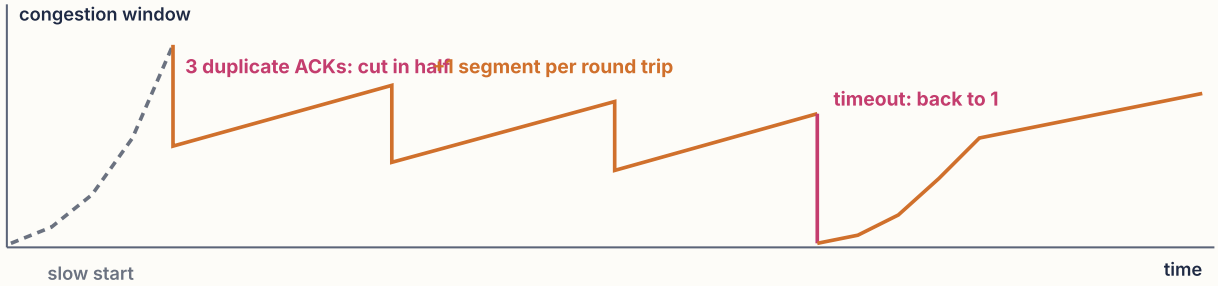


Figure 6. The window over time: slow start, then a sawtooth, growing by one segment per round trip and cut in half on loss; after a timeout it falls to one segment and slow-starts again.

6 Real numbers

The key formula: **speed ≤ window ÷ round trip**. A 64 KB window over a 100 ms round trip gives 640 KB a second, about 5 megabits, even on a gigabit link. To fill a 100-megabit link at 100 ms you need 1.25 MB on the way at once, which is why a SYN option called **window scaling** allows windows far bigger than 64 KB. And at the start of a connection slow start decides: a 100 KB page needs about three round trips, 14, then 29, then 58 KB. That's why the first 14 KB of a page matter so much.

Today's systems refine this: Linux uses **CUBIC** by default, and Google's **BBR** measures the bandwidth and round trip directly instead of waiting for losses. The two limits are the same.

speed ≤ window ÷ round trip

64 KB ÷ 100 ms	**640 KB/s ≈ 5 Mbit/s, even on a gigabit link**
100 Mbit/s × 100 ms	**1.25 MB must be in flight: window scaling**
a 100 KB page, cold start	**14 + 29 + 58 KB: about 3 round trips**

Most short transfers never reach full speed: the window and the round trip decide.

Figure 7. Real numbers: throughput is at most the window divided by the round-trip time, so long paths need big windows, and short transfers are limited by slow start.

7 What goes wrong

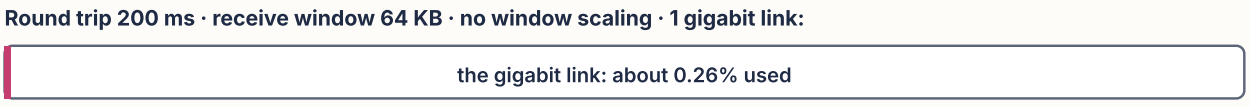
- **Wi-Fi loss.** A packet lost to radio noise looks like congestion, so TCP cuts its window even though the road was empty. On bad Wi-Fi, speed drops for no good reason.

- **A slow receiver.** The app doesn't read fast enough, the receive window falls to zero and the sender sits idle; a packet capture shows *zero window*. The network is fine; the app is the bottleneck.
- **Too small a window on a long path.** Fast links, far away, still slow: the window, not the link, is the limit.

PAUSE AND THINK

A server in the US, a user in India: a 200 ms round trip. The receive window is 64 KB, window scaling is off, and the link is one gigabit. What's the fastest this transfer can go? Cover the answer and decide.

Answer. 64 KB per 200 ms: about 320 KB a second, roughly 2.6 megabits, a quarter of one percent of the gigabit link. The fix: a bigger window, or a server closer to the user.



64 KB ÷ 0.2 s ≈ 320 KB/s ≈ 2.6 megabits.

Fix: a bigger window (window scaling), or a server closer to the user.

Figure 8. The pause-and-think answer: with a 64 KB window and a 200 ms round trip, the transfer tops out near 2.6 megabits, a tiny slice of a gigabit link.

THREE COMMON MISTAKES

- **"Flow control and congestion control are the same."** No. One protects the receiver, the other protects the network.
- **"TCP uses the full bandwidth straight away."** No. It starts small and has to grow, so short transfers rarely reach full speed.
- **"Losing packets means the network is broken."** No. A little loss is normal: it's the signal TCP uses to find the limit.

RECAP IN THREE LINES

1. The receiver's window stops TCP from drowning the receiver.
2. The congestion window stops it from drowning the network.
3. TCP sends the smaller, starts small, doubles, and cuts back on loss.

8 Interview questions: what to say in 30 seconds

Q1. What's the difference between flow control and congestion control? BASIC

Flow control protects the receiver: it advertises a receive window, its free buffer space, in every ACK, and the sender never has more unacknowledged data than that. Congestion control protects the network: the sender keeps its own congestion window, grows it while ACKs arrive and cuts it on loss. The sender is limited by the smaller of the two.

Q2. Explain slow start and congestion avoidance. BASIC

A new connection starts with a small congestion window, about 10 segments, and doubles it every round trip (slow start). After a threshold or a loss it grows by about one segment per round trip (congestion avoidance). On three duplicate ACKs classic TCP halves the window; on a timeout it drops to one segment and slow-starts again, which gives the sawtooth.

Q3. A transfer between India and the US is slow, though both ends have fast links. Why? SCENARIO

Throughput is at most window ÷ round-trip time. At 200 ms, a 64 KB window allows about 2.6 Mbit/s whatever the link speed. Check window scaling and buffer sizes, packet loss on the path (which keeps cutting the window), and consider moving the data closer (a CDN or a nearer region) or using parallel connections.

Q4. A capture shows the receiver advertising a zero window. What does that tell you? SCENARIO

The receiver's buffer is full because the application isn't reading fast enough, so the sender is paused and probing. The network is fine; the bottleneck is the receiving app: CPU, a slow disk, a blocked thread or too small a buffer.

Q5. What is the bandwidth-delay product, and why does it matter? TRICKY

Bandwidth × round-trip time: the bytes that must be in flight to keep the link full. 100 Mbit/s × 100 ms = 1.25 MB, far more than the 64 KB a window can say without window scaling, so long, fast paths need scaled windows and large buffers.

Q6. Why can TCP perform badly on Wi-Fi or mobile links, and what helps? TRICKY

Loss-based congestion control treats every lost packet as congestion, but radio noise also loses packets, so TCP cuts its window although the path isn't full. Link-layer retransmission hides some loss; newer algorithms such as BBR estimate bandwidth and round trip directly instead of reacting to each loss.

Read more. RFC 9293 (TCP, the window), RFC 5681 (congestion control: slow start, congestion avoidance, fast retransmit), RFC 6928 (an initial window of 10 segments), RFC 7323 (window scaling), RFC 9438 (CUBIC), and the BBR paper (Cardwell et al., ACM Queue 2016). The full written lesson is at backendroadmap.com.

Next: SD-N-011 · TCP part 4: closing, RST, TIME_WAIT, running out of ports backendroadmap.com

TCP part 4: how connections end

FIN · RST · TIME_WAIT

Watch the 8-min video

In one line: a connection closes in both directions with FIN and ACK; the side that closes first waits in TIME_WAIT, 60 seconds on Linux; a reset ends a connection at once, and many short connections can use up all your client ports.

WORDS YOU'LL MEET TODAY

FIN: finish: I'm done sending

TIME_WAIT: the wait, after closing, before a connection is really gone (60 s on Linux)

Ephemeral port: the temporary port a client picks for each connection

Connection pool: a few open connections reused for many requests

RST: reset: stop this connection right now

CLOSE_WAIT: the other side said goodbye, and our app hasn't closed yet

Half-close: one direction closed while the other can still send

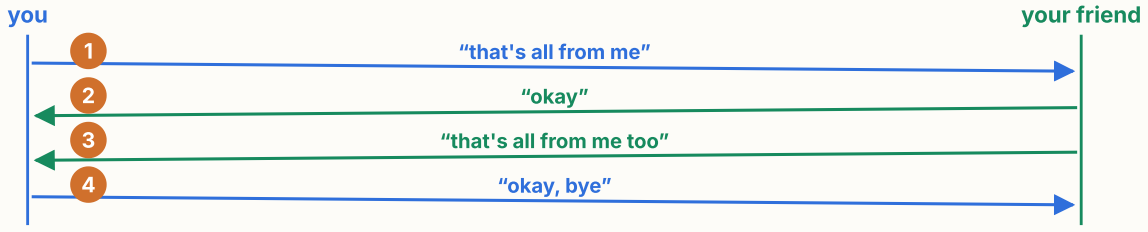
Keep-alive: keeping a connection open for the next request

1 The question: why did it run out of ports?

A busy service calls another service thousands of times a second. Suddenly every new call fails with *cannot assign requested address*. The CPU is fine, the memory is fine: it has run out of ports. To understand why, we need the end of a TCP connection, not the start.

2 An everyday picture first: ending a phone call

Ending a call politely: you say, that's all from me. Your friend says, okay. Then your friend says, that's all from me too, and you say, okay, bye. Each person says they're done and the other confirms: four messages, because there are two directions, just like the handshake in lesson 9. The rude way: you just hang up, and anything your friend was still saying is lost.



Each side says it's done and the other confirms: 4 messages, 2 directions. The rude way: just hang up (a reset).

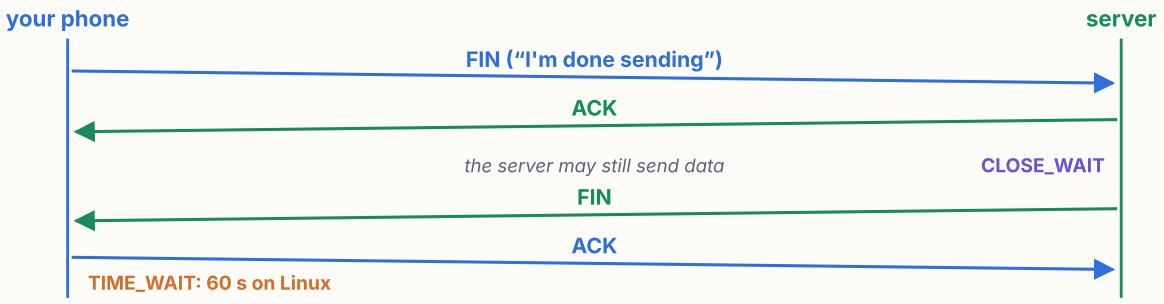
Figure 1. The analogy: ending a call politely takes four messages, because each person says they're done and the other confirms; hanging up is the reset.

3 The polite close: FIN and ACK, both ways

Your phone is done sending, so it sends a segment with the **FIN** flag. The server answers ACK, and one direction is closed. But the server may still have data to send. When it's done, it sends its own FIN, and your phone answers ACK. Both directions are closed. Often the server's ACK and FIN travel together, so you see three segments, not four.

Between the two halves the server is in **CLOSE_WAIT**: it got the goodbye and is waiting for its own app to close. If the app never closes, the connection stays there forever; thousands of them mean a bug in your

code, someone forgot to close.

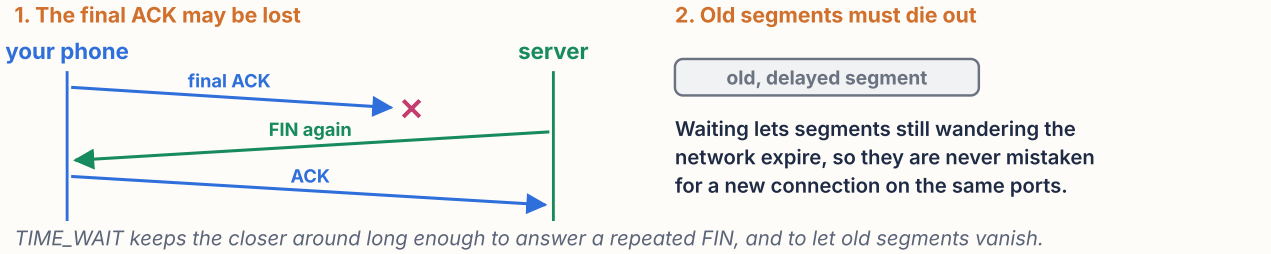


Each direction closes on its own. The server's ACK and FIN often travel together: 3 segments, not 4.

Figure 2. The polite close: FIN and ACK in each direction. The server sits in CLOSE_WAIT until its app closes; the side that closed first waits in TIME_WAIT.

4 TIME_WAIT: why the closer waits

The side that closed first, your phone, doesn't forget the connection right away. It enters **TIME_WAIT**; on Linux it waits 60 seconds. Two reasons. One: the final ACK might be lost; then the server sends its FIN again, and your phone must still be there to answer. Two: old, delayed segments from this connection might still be wandering the network; waiting lets them die out, so they are never mistaken for a new connection on the same ports.

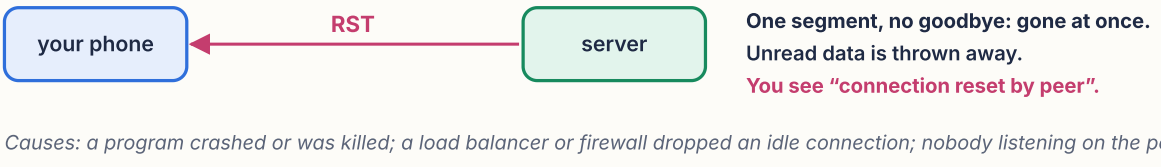


TIME_WAIT keeps the closer around long enough to answer a repeated FIN, and to let old segments vanish.

Figure 3. Why TIME_WAIT exists: a lost final ACK makes the server repeat its FIN, and the closer must still answer; and delayed old segments need time to die out.

5 The rude way: RST

A **reset** is one segment, no goodbye, and the connection is gone at once. Any data not yet read is thrown away, and you see *connection reset by peer*. Resets happen when a program crashes or is killed, when a load balancer or firewall drops an idle connection, or (lesson 9) when nobody listens on that port.



Causes: a program crashed or was killed; a load balancer or firewall dropped an idle connection; nobody listening on the port.

Figure 4. The rude way: a reset (RST) ends the connection at once and throws away unread data; you see "connection reset by peer".

6 Running out of ports

From lesson 4: a connection is named by four values, client address, client port, server address and server port. When your service calls one database at one address and one port, three of the four never change; only the client port can. So each connection to that database needs its own client port.



Calls to one database (one address, one port): each connection needs its own client port.

Figure 5. A connection is named by four values. To one database address and port, only the client port changes, so each connection needs its own.

Real numbers. Linux hands out client ports from 32,768 to 60,999: about 28,000. Every connection your service closes keeps its port busy, in TIME_WAIT, for 60 seconds. So at most about 28,000 ports every 60 seconds: roughly **470 new connections per second** to that one address and port. Open and close faster than that, and the ports run out.

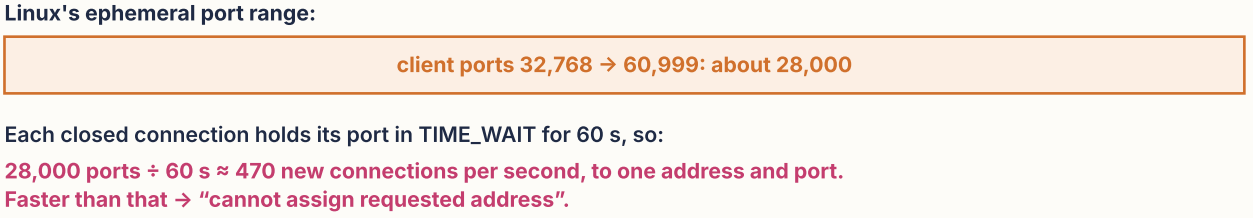


Figure 6. Running out of ports: about 28,000 client ports, each held for 60 seconds in TIME_WAIT, allow roughly 470 new connections per second to one server address and port.

The fixes, best first. Reuse connections: open a few and send many requests over each, with keep-alive and connection pools (lesson 29). Widen the port range. Let Linux safely reuse ports in TIME_WAIT for new outgoing connections, with `tcp_tw_reuse`. Spread the calls over more server addresses.

You see	It means	Do this
many TIME_WAIT on a client	too many short connections	pool them
many CLOSE_WAIT	your app forgets to close	fix the code
connection reset by peer	the other side crashed, timed out an idle connection, or refused it	check the other side

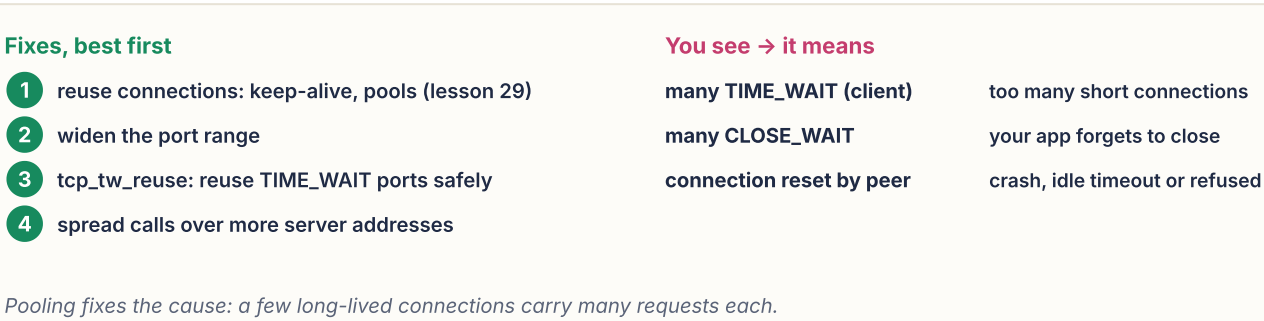


Figure 7. The fixes, best first, and a symptom checklist: TIME_WAIT piles mean short connections, CLOSE_WAIT piles mean an app that forgets to close.

PAUSE AND THINK

A service opens a new connection to its database for every query and closes it right after. It runs 1,000 queries per second; the port range is 28,000 and TIME_WAIT lasts 60 seconds. Does it run out of ports? Cover the answer and decide.

Answer. Yes. 1,000 a second, each holding a port for 60 seconds, needs 60,000 ports, and it only has 28,000. After about 28 seconds new connections start to fail. The fix: a connection pool with, say, 20 connections that stay open.



Figure 8. The pause-and-think answer: 1,000 connections a second for 60 seconds needs 60,000 ports against about 28,000, so new connections fail after about 28 seconds; a pool fixes it.

THREE COMMON MISTAKES

- **"Closing takes one message."** No. Each direction closes on its own: FIN and ACK, both ways.
- **"TIME_WAIT is a bug, so turn it off."** No. It protects you from lost ACKs and old segments. Reuse connections instead.
- **"A reset means an attack."** Usually not. Most resets are crashed programs, idle timeouts or closed ports.

RECAP IN THREE LINES

1. A connection closes in both directions: FIN, ACK, FIN, ACK.
2. The side that closes first waits in TIME_WAIT, for 60 seconds on Linux.
3. A reset ends a connection at once, and short connections can use up all your ports.

7 Interview questions: what to say in 30 seconds

Q1. How does a TCP connection close? BASIC

Each direction closes separately. The side that's done sends a FIN and the other ACKs it; later the other side sends its FIN and gets an ACK. That's four segments, often three because the second ACK and FIN travel together. The side that closed first then waits in TIME_WAIT.

Q2. What is TIME_WAIT and why does it exist? BASIC

The state the side that closed first stays in after the final ACK, 60 seconds on Linux (two maximum segment lifetimes in the standard). It lets the closer answer a repeated FIN if its last ACK was lost, and lets old delayed segments expire so they can't be mistaken for a new connection with the same addresses and ports.

Q3. A service starts failing with "cannot assign requested address". What's going on? SCENARIO

It's out of ephemeral ports. Each new connection to the same server address and port needs a distinct client port, and closed connections hold theirs in TIME_WAIT for 60 s: about 28,000 ports means roughly 470 new connections per second. Fix the cause with keep-alive and connection pools; widening the range, tcp_tw_reuse and more destination addresses help too.

Q4. You see thousands of connections in CLOSE_WAIT. What does it tell you? SCENARIO

The peer has closed, but your application never closed its side: a leak in your code, such as a missing close in an error path or a connection not returned to the pool. The kernel can't fix it; the app must close the socket.

Q5. What's the difference between FIN and RST? TRICKY

FIN is a graceful close of one direction: data already sent is delivered and acknowledged, and the other side can keep sending. RST aborts the whole connection immediately, discards data not yet read, and the other side sees “connection reset by peer”. Crashes, idle timeouts on load balancers and connections to a closed port cause resets.

Q6. Should you turn off TIME_WAIT to fix port exhaustion? TRICKY

No. It protects against lost ACKs and stray old segments. Reuse connections so you open fewer; on Linux, tcp_tw_reuse safely reuses TIME_WAIT ports for new outgoing connections using timestamps. The old tcp_tw_recycle was unsafe behind NAT and was removed.

Read more. RFC 9293 (TCP: closing, TIME_WAIT, RST), RFC 6056 (port randomisation and ephemeral ports), RFC 7323 (timestamps, used by tcp_tw_reuse), Linux ip-sysctl.txt (ip_local_port_range, tcp_tw_reuse) and tcp(7). The full written lesson is at backendroadmap.com.

Next: SD-N-012 · UDP vs TCP: when speed beats reliability backendroadmap.com

UDP vs TCP

when speed beats reliability

Watch the 4-min video

In one line: UDP sends independent datagrams with no handshake, ACKs, retransmission or ordering; choose it when fresh data beats complete data (voice, video, games), for tiny exchanges like DNS, or as the base for protocols like QUIC, and choose TCP when losing a byte would be a bug.

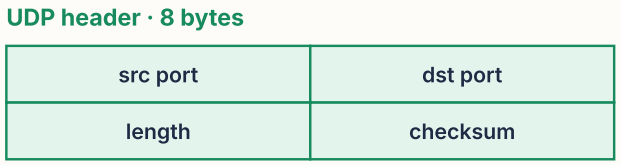
1 UDP: the simplest transport

You are on a video call and the connection hiccups for a split second. Do you want the call to freeze while every lost packet is resent? No. You would rather lose a few milliseconds of audio and keep talking. That is exactly what UDP gives you.

UDP sends independent messages called **datagrams**. There is no handshake, no connection, no acknowledgements, no retransmission and no ordering. Each datagram may arrive, arrive late, arrive out of order, arrive twice, or not arrive at all. Unlike TCP's byte stream, each datagram keeps its boundaries: one send is one receive.

2 What you get in return

- **Tiny overhead.** The UDP header is just **8 bytes**, against at least 20 for TCP.
- **No setup round trip.** The very first packet carries data.
- **No head-of-line blocking.** A lost datagram never holds up the next one.
- **No per-connection state** on the server, so one socket can serve huge numbers of clients.
- **Multicast and broadcast.** One packet to many receivers, which TCP cannot do.



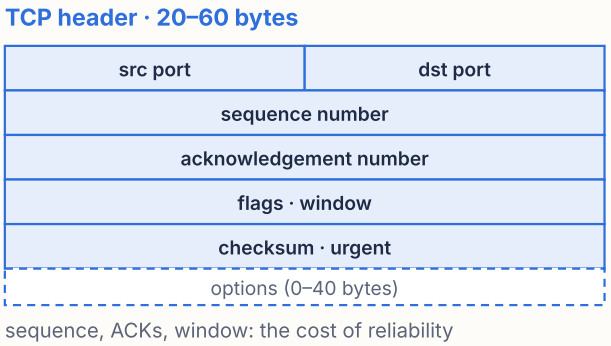


Figure 1. UDP adds only ports, a length and a checksum to IP. TCP's extra fields are what make reliability, ordering and flow control possible.

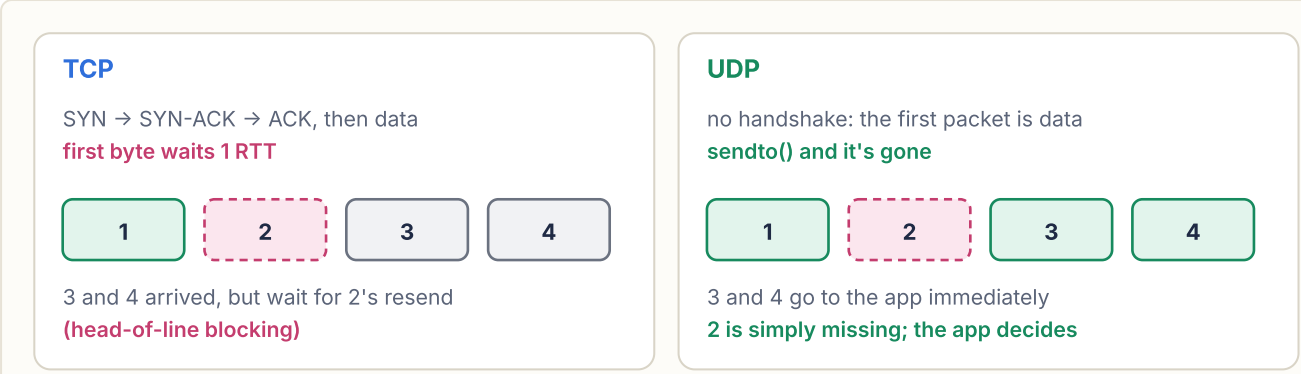


Figure 2. TCP waits a round trip before sending and holds later data behind a loss. UDP sends at once and hands every datagram that arrives straight to the app.

3 Real-time voice: why resending is pointless

Voice apps send audio in small packets, about one every **20 ms** (50 packets per second). If one is lost, resending it is pointless: by the time it arrives, at least one more round trip later, that moment of the conversation has already played.

So voice apps use UDP, keep a small **jitter buffer** (a few tens of milliseconds) to smooth out uneven arrival times, and simply **hide the gap** by repeating or blending the neighbouring audio. Losing 1–2% of 20 ms packets is barely noticeable; freezing for 150 ms on every loss is not.



Figure 3. A voice stream over UDP. The lost packet is concealed, not resent; the jitter buffer absorbs small variations in arrival time.

4 Choosing between them

Choose UDP when...	Choose TCP when...
Freshness beats completeness: voice and video calls, live broadcasts, online games	**Every byte must arrive, in order**: web pages and APIs, file transfers, database connections, payments
Tiny request/response: DNS (one packet out, one back), NTP, DHCP, metrics (StatsD)	**Large or long transfers** where you want congestion control and flow control for free
Building a new protocol with its own reliability: QUIC (HTTP/3), WebRTC media	**Losing data would be a bug**, and you don't want to rebuild reliability yourself

THE RULE OF THUMB

If losing data would be a bug, use TCP. If late data is worthless, use UDP. If you need reliability *and* UDP's freedoms (no head-of-line blocking, faster setup), use a protocol built on UDP such as QUIC rather than writing your own.

5 How a game handles lost UDP packets

A multiplayer game server sends the **whole relevant game state** many times per second (often 20–60 updates per second, up to 128 in competitive shooters). If one update is lost, the next one replaces it anyway, so there is nothing to resend.

The client **predicts** movement (its own player reacts instantly, other players are extrapolated) and **smooths** between updates, so a missing update is invisible. For the few things that must arrive, such as a purchase, a chat message or a match result, the game adds **its own acknowledgements and resends**, just for those messages, usually with an ID so a duplicate is ignored.

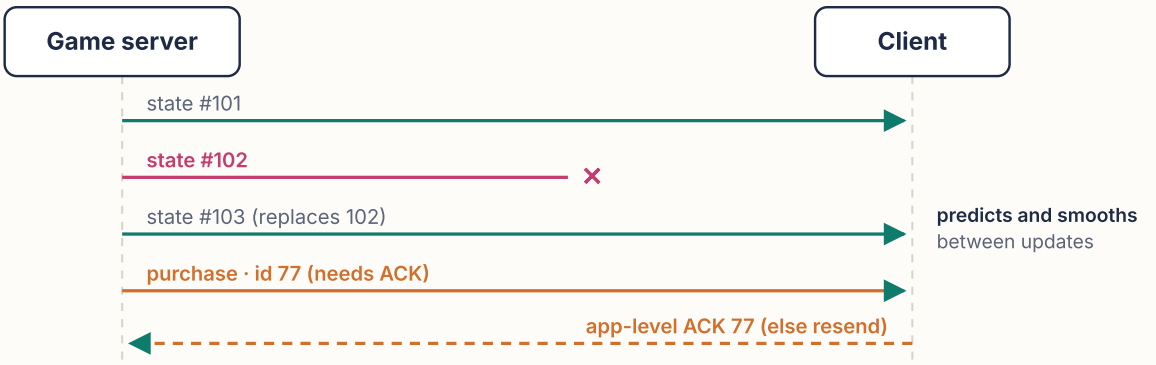


Figure 4. State updates are fire-and-forget: update 103 replaces the lost 102. Only the purchase is tracked with an application-level ACK and resent if needed.

WORKED EXAMPLE: WHAT DOES A LOSS COST?

Voice at 20 ms per packet, 150 ms round trip, 1% loss.

- **TCP:** each loss stalls the stream for at least one RTT plus detection time, about **150–300 ms** of silence, then a burst of stale audio. At 50 packets/s, that is roughly one stall every 2 seconds.
- **UDP:** each loss is a **20 ms** gap, concealed by the decoder. Total delay stays at the network delay plus a ~40 ms jitter buffer.

COMMON MISTAKES

- Saying UDP is “faster” in general. It is faster to start and never stalls on loss, but the bytes travel at the same speed.
- Sending large UDP payloads. Anything bigger than the path MTU (about 1,200–1,400 bytes to be safe) gets fragmented, and losing one fragment loses the whole datagram.
- Forgetting congestion: UDP has no congestion control, so a careless sender can flood a network. Rate-limit it yourself.
- Reinventing TCP on UDP for an API. Use TCP, or QUIC if you need what UDP offers.

6 Key terms

Datagram: one self-contained UDP message

Jitter buffer: short delay that evens out arrival

Tick rate: game state updates per second

Connectionless: no handshake, no shared state

Loss concealment: hiding a missing audio or video frame

Client prediction: simulating movement between updates

Jitter: variation in packet arrival times

Multicast: one packet delivered to many receivers

QUIC: reliable, encrypted transport on UDP

7 Interview questions

Q1. When would you choose UDP over TCP? BASIC

When low latency matters more than completeness (voice, video, games), for tiny exchanges like DNS where one packet each way is enough, or as the base for a protocol like QUIC that adds its own reliability. Otherwise, TCP.

Q2. What does UDP give up compared with TCP, and what does it gain? BASIC

It gives up the handshake, ACKs, retransmission, ordering, flow and congestion control. It gains an 8-byte header (vs 20+), no setup round trip, no head-of-line blocking, message boundaries and multicast.

Q3. How does a game handle lost UDP packets? BASIC

It sends frequent full state updates, so the next update replaces a lost one; the client predicts and smooths between updates; and only critical events (purchases, chat) get app-level ACKs and resends, with IDs to drop duplicates.

Q4. You're designing a video-calling service. TCP or UDP for the media, and what about signalling? SCENARIO

Media over UDP (RTP via WebRTC) with a jitter buffer and loss concealment, since late frames are worthless. Signalling (join, leave, chat) over a reliable channel such as WebSocket over TCP. Fall back to TURN over TCP or TLS only when UDP is blocked.

Q5. Your service pushes metrics to a collector 10,000 times a second. Would UDP be OK? SCENARIO

Yes, StatsD does exactly this: fire-and-forget over UDP, so a slow collector never blocks the app. Losing 0.1% of samples barely moves an aggregate. For billing-grade counts, use TCP or a queue instead.

Q6. Why does DNS use UDP, and when does it switch to TCP? SCENARIO

A query and its answer usually fit in one packet each, so UDP avoids a handshake and server state. If the answer is too large (truncated flag set), or for zone transfers, it retries over TCP.

Q7. If UDP is unreliable, why is HTTP/3 built on it? TRICKY

QUIC adds reliability, congestion control and TLS 1.3 on top of UDP, but tracks each stream separately, so one loss doesn't stall other streams. It also combines the transport and TLS handshakes into one round trip, and it runs in user space, so it can evolve without OS kernel changes.

Q8. A client sends one 6 KB UDP datagram and it often never arrives, while 1 KB datagrams do. Why? TRICKY

6 KB exceeds the typical 1,500-byte MTU, so IP fragments it into about 5 pieces. Lose any one and the whole datagram is dropped, and some firewalls drop fragments outright. Keep UDP payloads under about 1,200 bytes.

Next: SD-N-013 · Subnets and CIDR: neighbourhoods of addresses backendroadmap.com

IP Addresses, Subnets & CIDR

design the address space before you need it

Watch the 5-min video

In one line: an IPv4 address is 32 bits; CIDR (`/16`, `/24`) says how many of those bits are the fixed network part; you carve a VPC's block into subnets per zone and keep databases in private subnets with no route from the internet.

1 What an IP address is

Every machine on a network needs an address. In the cloud you design that address space yourself, and a bad plan is painful to change later, so it is worth getting right on day one.

An **IPv4** address is **32 bits**, written as four numbers from 0 to 255 (each one 8 bits, an *octet*), for example `10.0.1.25`. That gives 2^{32} ≈ **4.3 billion** addresses, and the internet ran out of fresh ones years ago. **IPv6** uses **128 bits**, written in hexadecimal (like `2001:db8::1`), so addresses are effectively unlimited.

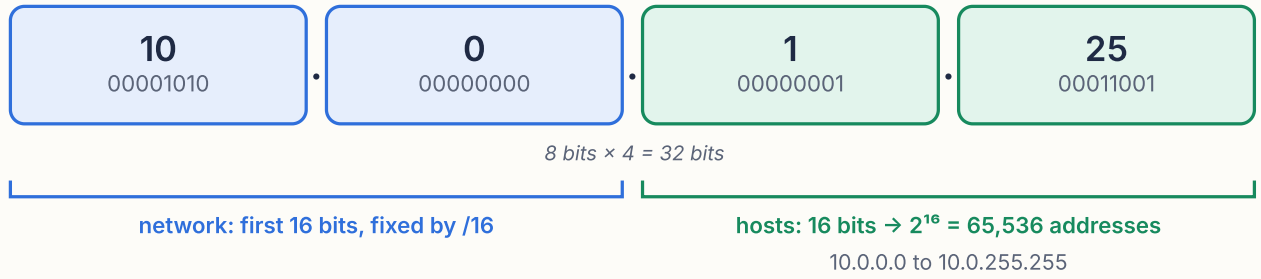


Figure 1. `10.0.1.25` is four 8-bit numbers. With `/16`, the first 16 bits are the network; the last 16 bits number the hosts.

2 Private ranges and NAT

Three ranges are reserved for internal networks (RFC 1918). They are not routed on the public internet, so every company can reuse them:

Range	CIDR	Size
`10.0.0.0 – 10.255.255.255`	`10.0.0.0/8`	16.7 million addresses: the usual choice for cloud VPCs
`172.16.0.0 – 172.31.255.255`	`172.16.0.0/12`	1 million. Only 172.**16** to 172.**31**; 172.15 and 172.32 are public
`192.168.0.0 – 192.168.255.255`	`192.168.0.0/16`	65,536: typical for home and office networks

To reach the internet, private machines go through **NAT** (network address translation). NAT swaps the private source address for a shared public one, remembers the mapping, and sends replies back to the right machine.

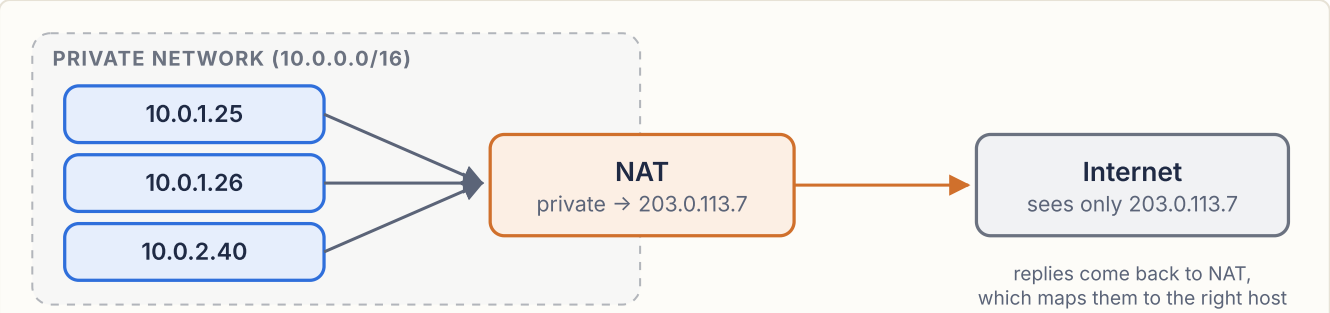


Figure 2. Many private hosts share one public address through NAT. The internet never sees the 10.x addresses.

3 CIDR: reading a block

CIDR notation (said "cider") describes a block of addresses as *start address / prefix length*. In `10.0.0.0/16`, the **/16** means the first 16 bits are the network part and are fixed. The other 32 − 16 = 16 bits are for hosts, so the block holds 2^{16} = **65,536** addresses, from `10.0.0.0` to `10.0.255.255`.

$$\text{addresses in a /n block} = 2^{(32 - n)}$$

A **bigger number after the slash means a smaller block**. Each extra bit halves it:

Prefix	Host bits	Addresses	Example	Usable on AWS (minus 5)
`/16`	16	65,536	`10.0.0.0 – 10.0.255.255`	a whole VPC
`/20`	12	4,096	`10.0.16.0 – 10.0.31.255`	4,091
`/24`	8	256	`10.0.1.0 – 10.0.1.255`	251
`/28`	4	16	`10.0.1.0 – 10.0.1.15`	11 (smallest AWS subnet)

4 Subnets: splitting the block

You split a big block into smaller **subnets**. Carve the `/16` into `/24`s and you get 2^8 = **256** of them: `10.0.0.0/24`, `10.0.1.0/24`, `10.0.2.0/24`, and so on up to `10.0.255.0/24`. Subnets don't have to be equal: you could use a few `/20`s for big app tiers and `/24`s for the rest, as long as they don't overlap.

In the cloud, **each subnet lives in exactly one availability zone**, and the provider reserves a few addresses in each. On AWS that is **five**: the network address, the next three (VPC router, DNS, reserved for future use) and the last (broadcast). So a `/24` gives 251 usable addresses.

5 Designing a real VPC

The example: a VPC with `10.0.0.0/16` across two availability zones, with three tiers of subnets in each zone.

- **Public subnets** hold only the load balancers (and the NAT gateways). Their route table sends `0.0.0.0/0` to the **internet gateway**.
- **Private app subnets** hold your services. Their default route goes to a **NAT gateway**: outbound only, for things like calling a payment API or pulling updates. Nobody on the internet can start a connection to them.
- **Private data subnets** hold the databases. Only the app tier can talk to them.

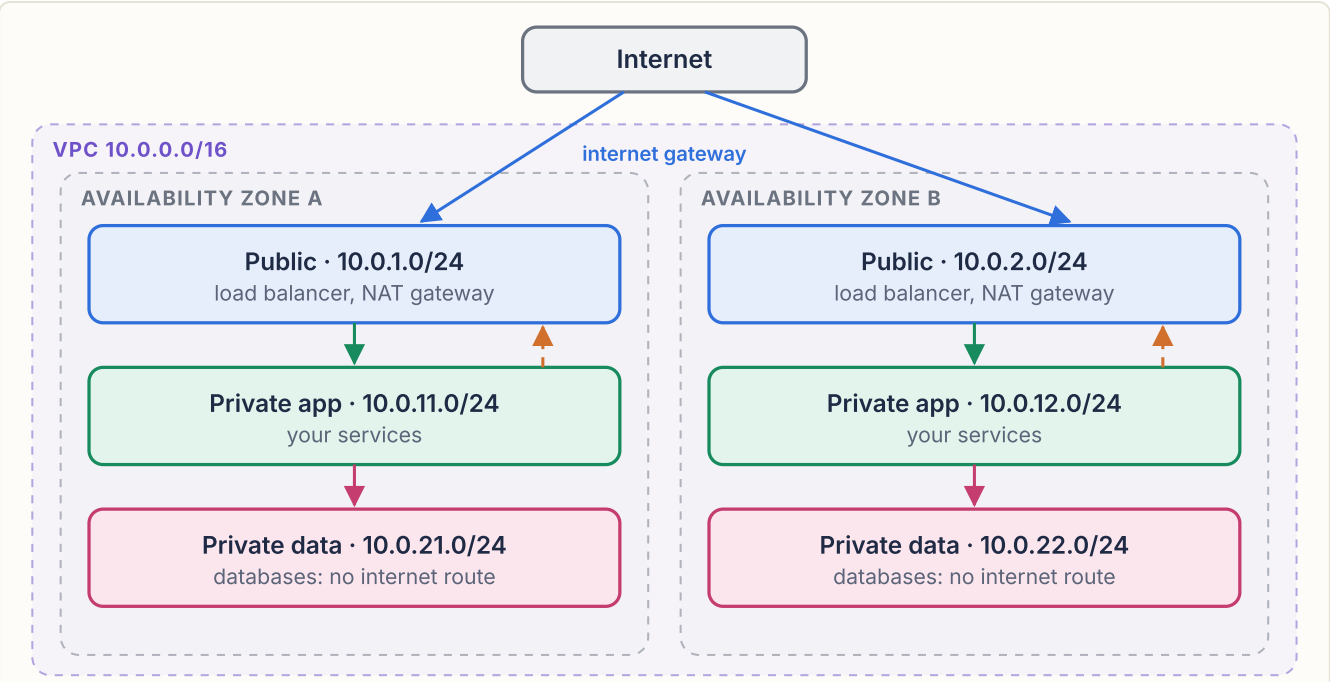


Figure 3. A two-zone VPC. Traffic enters only at the load balancers in the public subnets; apps go out through the NAT gateway (dashed orange arrows, outbound only); the data subnets have no internet route at all.

Why put databases in private subnets? Because there is simply no route from the internet to them. Even if a password leaks or a firewall rule is wrong, attackers cannot reach the database directly. That is **defense in depth**: routing is one layer, security groups are another, credentials a third.

WORKED EXAMPLE: THE ADDRESS PLAN

VPC `10.0.0.0/16` (65,536 addresses). Use `/24` subnets: public `10.0.1.0/24` and `10.0.2.0/24`, app `10.0.11.0/24` and `10.0.12.0/24`, data `10.0.21.0/24` and `10.0.22.0/24`. That is 6 × 256 = 1,536 addresses used, 6 × 251 = **1,506** usable on AWS, and over 97% of the block left for a third zone or new tiers. If the app tier will run thousands of containers that each take an IP, make the app subnets `/20` (4,091 usable each) instead.

PLAN AHEAD

Give the VPC a large block (a `/16`) and never overlap it with other VPCs or your office network. Peering, VPNs and transit gateways cannot route between two networks that both claim `10.0.0.0/16`, and renumbering a live VPC means rebuilding it. A simple fix: give each VPC its own slice, such as `10.0.0.0/16` for prod, `10.1.0.0/16` for staging, `10.2.0.0/16` for dev.

COMMON MISTAKES

- Reading `/24` as "24 hosts". It is 24 *network* bits, leaving 8 host bits: 256 addresses.
- Thinking all of 172.x is private. Only 172.16.0.0 to 172.31.255.255 is.
- Using the default small VPC or tiny subnets, then running out of IPs when containers or Lambda functions each take one.
- Overlapping VPC ranges with each other or with the office network.
- Putting databases in public subnets "just for now".

6 Key terms

IPv4 / IPv6: 32-bit / 128-bit addresses

Prefix length: number of fixed network bits

NAT: swaps private source IPs for a public one

Octet: one of the four 8-bit numbers in IPv4

Subnet: a slice of the VPC block, in one zone

Internet gateway: VPC route target for public subnets

CIDR: block = start address / prefix length

Private range: 10/8, 172.16/12, 192.168/16

Defense in depth: several independent layers of protection

7 Interview questions

Q1. What does `10.0.0.0/16` mean? BASIC

The first 16 bits (`10.0`) are fixed as the network, leaving 16 bits for hosts: 2^{16} = 65,536 addresses, from 10.0.0.0 to 10.0.255.255.

Q2. How many addresses are in a /24, and how many /24s fit in a /16? BASIC

A /24 has 32 − 24 = 8 host bits, so 256 addresses (251 usable on AWS, 254 on a classic network after network and broadcast). A /16 holds 2^8 = 256 of them.

Q3. What are the private IPv4 ranges, and why do they exist? BASIC

10.0.0.0/8, 172.16.0.0/12 and 192.168.0.0/16. They are not routed on the internet, so every organisation can reuse them internally; NAT translates them to a public address to go out.

Q4. Design the subnets for a three-tier app in two availability zones. SCENARIO

VPC 10.0.0.0/16. Per zone: a public /24 for load balancers and the NAT gateway (route 0.0.0.0/0 to the internet gateway), a private app subnet (default route to NAT), and a private data subnet with no internet route. Six subnets, each in one zone.

Q5. Why put databases in private subnets if they already have passwords and security groups? SCENARIO

Defense in depth. With no route from the internet, a leaked password or a wrong firewall rule is still not enough for an attacker to connect. Only the app tier can reach the database.

Q6. Your company wants to peer a new VPC with an existing one and the office VPN. What do you check first? SCENARIO

That the CIDR blocks don't overlap. If the new VPC uses 10.0.0.0/16 and so does the old one, routing between them is impossible without renumbering; pick, say, 10.1.0.0/16.

Q7. A /28 subnet on AWS: how many instances can you launch? TRICKY

A /28 has 2^4 = 16 addresses. AWS reserves 5, so 11 usable. It is the smallest subnet AWS allows, and it fills up fast.

Q8. Do instances in a private app subnet need public IPs to download updates? TRICKY

No. They send traffic to the NAT gateway in a public subnet, which uses its own public IP. Connections can go out and replies come back, but nothing on the internet can open a connection in.

Next: SD-N-014 · Routing and NAT: how packets find their way, and why your phone shares one public address backendroadmap.com

Ports, Sockets & Routing

how one port serves 100,000 clients, and how packets find it

Watch the 4-min video

In one line: the IP finds the machine and the port finds the process; every TCP connection gets its own socket, named by a unique 5-tuple, so one port can hold thousands of connections; and routers forward packets hop by hop using longest prefix match.

1 Ports: which process on the machine

One server, one port: 443. Yet it holds **100,000 live WebSocket connections** at the same time. How? The answer is in how ports and sockets really work.

An **IP address** finds the machine. A **port** finds the process on that machine. Ports are **16-bit** numbers, from 0 to 65,535. Servers *listen* on well-known ports: **443** for HTTPS, **22** for SSH, **5432** for Postgres, **6379** for Redis (and 80 for HTTP, 53 for DNS). Clients use a random, temporary port on their side, called an **ephemeral port**, picked by the OS for each outgoing connection (on Linux, from 32768 to 60999 by default).

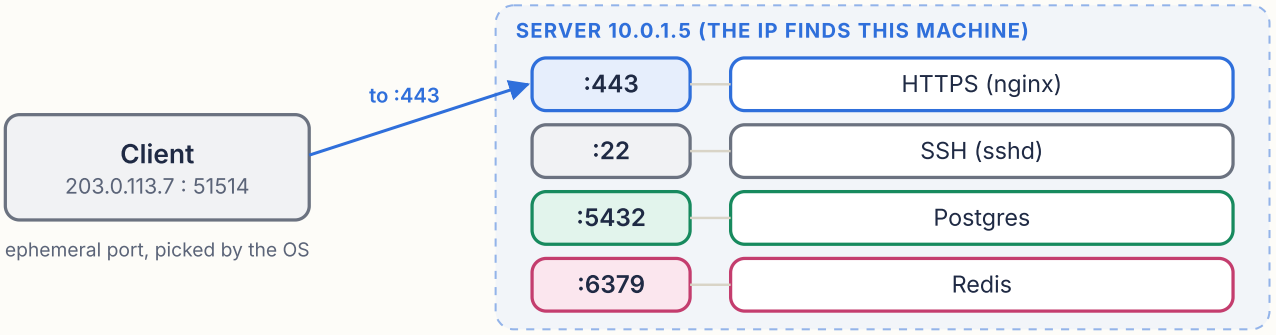


Figure 1. The client reaches 10.0.1.5 by IP; port 443 selects the web server process. The client's own port, 51514, was chosen by its OS.

2 Sockets and the 5-tuple

A **socket** is one endpoint of communication: an IP address, a port and a protocol, such as TCP. Your program asks the OS for a socket, then reads and writes through it, much like a file. On Linux it really is a file descriptor.

A TCP connection is identified by five values, the **5-tuple**:

```
( protocol, source IP, source port, destination IP, destination port )
```

Change any one of them and it is a different connection. That is why a TCP socket and a UDP socket can both use port 443 on the same machine, and why two tabs in your browser can open two separate connections to the same server: they get different ephemeral ports.

3 How one port holds 100,000 connections

The server has **one listening socket** on port 443. Each time a client connects, the OS creates a **brand new socket** for that connection and hands it to the program (the `accept()` call). All these sockets share local port 443, but each has a different client IP and port, so every 5-tuple is unique. Ports are not used up on the server side at all.

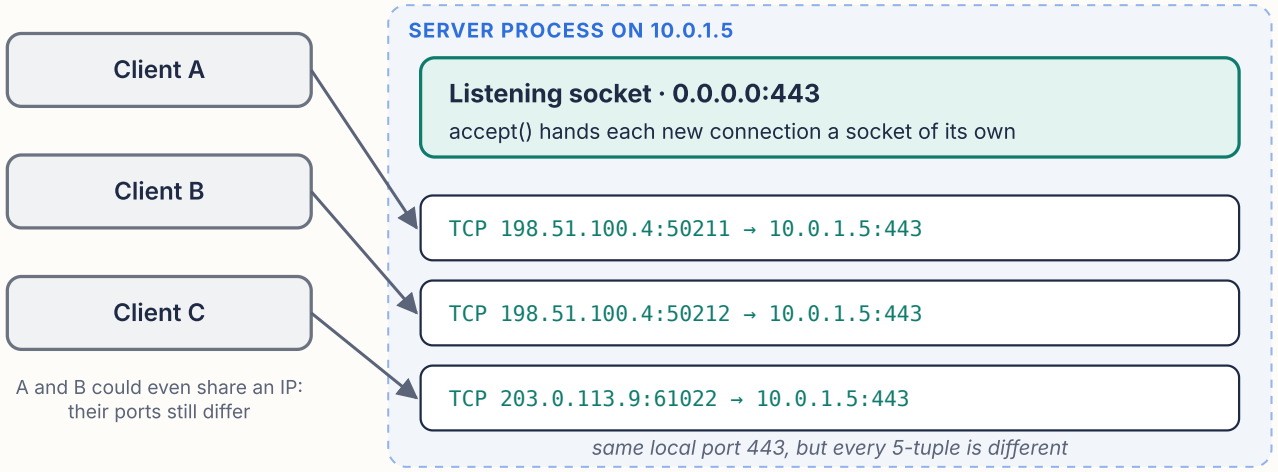


Figure 2. One listening socket on :443 accepts connections; each accepted connection becomes its own socket, told apart by the client's IP and port.

The real limits are **memory** and **file descriptors**, not ports. Every socket is a file descriptor, and many systems default to 1,024 open files per process. Every connection also costs kernel buffers plus your app's own state.

WORKED EXAMPLE: 100K WEBSOCKETS ON ONE BOX

File descriptors: 100,000 connections need 100,000 descriptors, plus some for logs and upstream connections. Raise the open-file limit (for example `ulimit -n` or systemd `LimitNOFILE`) to about 200,000.

Memory: budget per connection. If kernel buffers plus app state come to about 20 KB each, 100,000 × 20 KB ≈ **2 GB** of RAM. At 100 KB each it is 10 GB, which decides your instance size. Then load-test, because idle connections are cheap but busy ones also cost CPU.

4 Routing: how packets find the server

Every machine and router has a **route table**. For each packet, it picks the **most specific route** that matches the destination address. That rule is called **longest prefix match**: a `/24` beats a `/16`, which beats a `/0`. Addresses on the local network are delivered directly. Everything else goes to the **default gateway**, the `0.0.0.0/0` route, which matches any address.

Routers forward packets **hop by hop**. Each one looks only at the destination IP, checks its own table and passes the packet one step closer. No single router knows the full path. (Each hop also lowers the packet's TTL counter by one, so a routing loop can't circle forever; `traceroute` uses this to list the hops.)

Route table of a private app subnet

Destination	Target	Prefix bits
10.0.0.0/16	local (inside the VPC)	16
10.1.0.0/16	**peering link to VPC 2**	16
0.0.0.0/0	NAT gateway (default)	0

Packet to 10.1.4.20

matches /16 and /0: the /16 is more specific, so it goes over peering

8.8.8.8 matches only 0.0.0.0/0 → NAT gateway. 10.0.2.15 matches 10.0.0.0/16 → delivered locally. 0.0.0.0/0 matches every address, so it is the route of last resort: the default gateway.

Figure 3. Longest prefix match. The packet matches two routes; the one with more fixed bits wins.

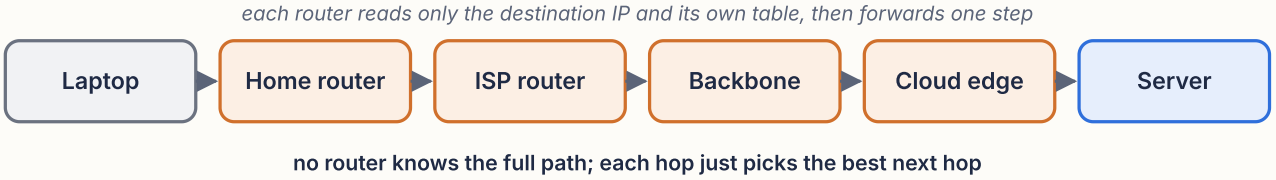


Figure 4. A request crosses several routers. Each makes a local decision; together they deliver the packet.

A **cloud VPC works the same way**. Each subnet's route table keeps local traffic (`10.0.0.0/16 → local`) inside the VPC, and sends `0.0.0.0/0` to an **internet gateway** (public subnets) or a **NAT gateway** (private subnets).

COMMON MISTAKES

- Thinking a server runs out of ports as clients connect. It doesn't: the client side picks the varying port.
- Leaving the default 1,024 open-file limit on a server meant for many connections: it fails with "too many open files".
- Sizing a WebSocket fleet by CPU only; memory per connection is usually the limit.
- Assuming the first matching route wins. The most specific one wins, whatever order the table is in.

5 Key terms

Port: 16-bit number that picks the process

5-tuple: protocol, src IP, src port, dst IP, dst port

Route table: destination prefix → next hop

Ephemeral port: temporary client-side port

Listening socket: waits on a port, accepts connections

Longest prefix match: the most specific route wins

Socket: endpoint: IP + port + protocol

File descriptor: OS handle for an open file or socket

Default gateway: the 0.0.0.0/0 route

6 Interview questions

Q1. What is a socket? BASIC

An endpoint defined by an IP address, a port and a protocol, which a program uses to send and receive data. The OS gives it to the program as a handle (a file descriptor) to read and write.

Q2. What's the difference between an IP address and a port? BASIC

The IP address finds the machine; the port (0 to 65,535) finds the process on it. 10.0.1.5:5432 is Postgres on that host, 10.0.1.5:6379 is Redis.

Q3. How can one server port accept thousands of connections? BASIC

Each connection gets its own socket, identified by the full 5-tuple. All share local port 443, but the client's IP and port make each one unique. The limits are memory and file descriptors.

Q4. You need one box to hold 100k WebSocket connections. What do you change? SCENARIO

Raise the open-file limit well above 100k (say 200k), budget memory per connection (100k × 20 KB ≈ 2 GB), tune kernel buffers if needed, and load-test. Ports are not the constraint.

Q5. A packet for 10.1.4.20 hits a table with 10.0.0.0/16 local, 10.1.0.0/16 peering, and 0.0.0.0/0 NAT. Where does it go? SCENARIO

Peering. It matches 10.1.0.0/16 and 0.0.0.0/0; longest prefix match picks the /16, the more specific route.

Q6. An app in a private subnet can't reach an external API. What do you check in routing? SCENARIO

That the subnet's route table has 0.0.0.0/0 pointing at a NAT gateway, that the NAT gateway sits in a public subnet whose 0.0.0.0/0 goes to the internet gateway, then security groups and network ACLs.

Q7. Where can ports actually run out? TRICKY

On the client side. One client IP talking to one server IP:port can only open as many connections as it has ephemeral ports, about 28,000 with Linux defaults. Proxies and load balancers hit this; fix it with more source IPs, more backend IPs, a wider port range, or connection reuse.

Q8. Do routers know the full path to a destination? TRICKY

No. Each router looks only at the destination IP, picks the best next hop from its own table and forwards the packet. The full path emerges from many local decisions, and it can change between packets.

Next: SD-N-015 · DNS part 1: the phone book of the internet, one lookup step by step backendroadmap.com

How DNS Resolution Works: Resolvers, Records, TTL

why half your users still hit the old server

Watch the 4-min video

In one line: your machine asks a recursive resolver, which walks root → TLD → authoritative on a cache miss; every answer is cached for its TTL, so lower the TTL before a migration and changes spread in minutes instead of hours.

1 The problem: DNS remembers

You move `api.example.com` to a new load balancer and update DNS. An hour later, half your users are still hitting the old one. Nothing is broken. It is DNS caching, doing exactly what you told it to. To control it, you need to know how a lookup really works.

DNS turns names into IP addresses. Your laptop does not search the internet itself. It asks a **recursive resolver**, usually run by your ISP, your cloud provider (inside a VPC, the provider's resolver), or a public service like `1.1.1.1` or `8.8.8.8`. The resolver does the hard work for you, and **caches** the answers so the next user gets them instantly.

2 The lookup, step by step

On a **cold cache**, the resolver walks down the naming tree, from right to left in the name:

1. It asks a **root server**: who handles `.com`? The root points it to the `.com` **TLD** (top-level domain) servers.
2. It asks a TLD server: who handles `example.com`? The TLD points it to example.com's **authoritative name servers**.
3. It asks the authoritative server, which gives the real answer: the IP address of `api.example.com`.

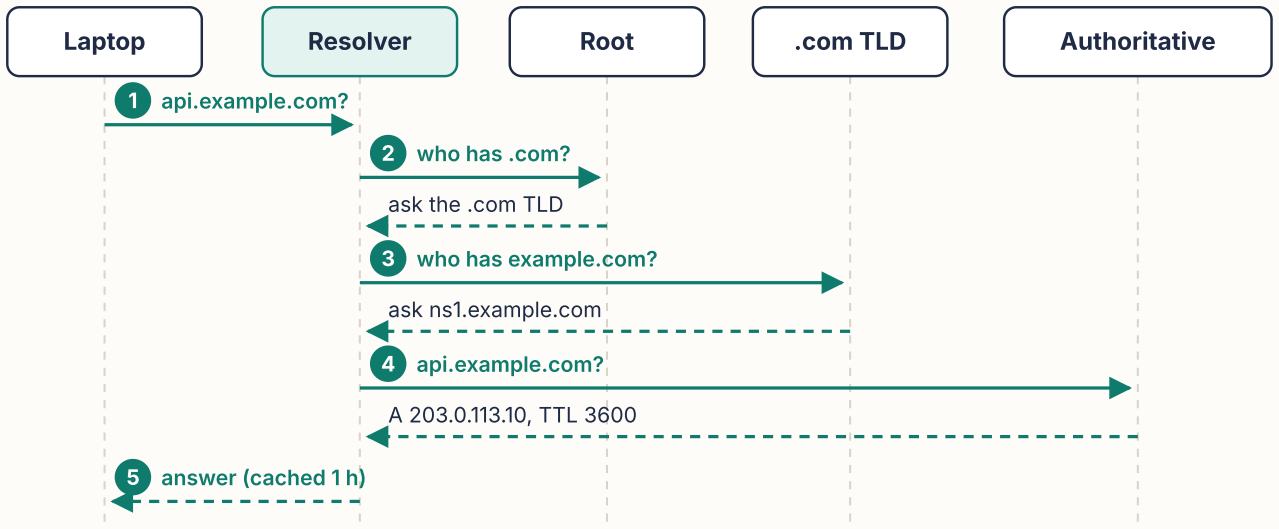


Figure 1. A cold-cache lookup. The laptop asks once; the resolver does every step and caches the answer for its TTL.

Root and TLD servers only *refer* the resolver onwards; only the authoritative server holds the records. On a warm cache most of this is skipped: the resolver already knows the `.com` and `example.com` servers, so a lookup is often one round trip, or zero.

3 Record types to know

Type	What it does	Example
A	Name → IPv4 address	`api.example.com A 203.0.113.10`
AAAA	Name → IPv6 address (“quad A”)	`api.example.com AAAA 2001:db8::10`
CNAME	Alias: points one name to another *name*, which is then resolved	`www CNAME my-lb.elb.amazonaws.com`
MX	Where email for the domain goes, with a priority	`example.com MX 10 mail.example.com`
TXT	Free text, often domain verification or email policy (SPF)	`example.com TXT "v=spf1 …"`
NS	Names the authoritative servers for a zone	`example.com NS ns1.example.com`

4 TTL and caching

Every record has a **TTL** (time to live) in **seconds**. Resolvers, operating systems and browsers all cache the answer until it expires. With a TTL of **3600**, a resolver may keep serving the old IP for **up to an hour** after you change it. (This DNS TTL is a cache lifetime; it has nothing to do with the hop counter called TTL in IP packets.)

Even failed lookups get cached. That is **negative caching**: if someone looked up a name before you created it, the “does not exist” answer is cached too (for a time set in the zone's SOA record), so the new name can keep failing for a while.

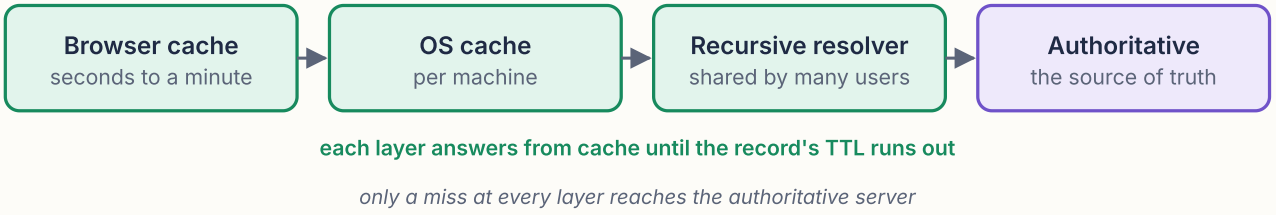


Figure 2. Answers are cached at several layers. A change reaches a user only when every layer between them and the authoritative server has expired its copy.

5 Migrating a record safely

Here is how to move `api.example.com` to a new load balancer with almost no stale traffic:

1. **A day or two ahead**, lower the TTL from 3600 to **60**.
2. **Wait at least one old TTL** (here, one hour), so every cache drops its 1-hour copy and picks up the short value.
3. **Switch the record** to the new load balancer. Within about a minute, most clients follow.
4. **Keep the old load balancer running** for a while (hours to days), because some clients and libraries ignore TTLs, and open connections stay where they are.
5. **Once stable, raise the TTL** again.

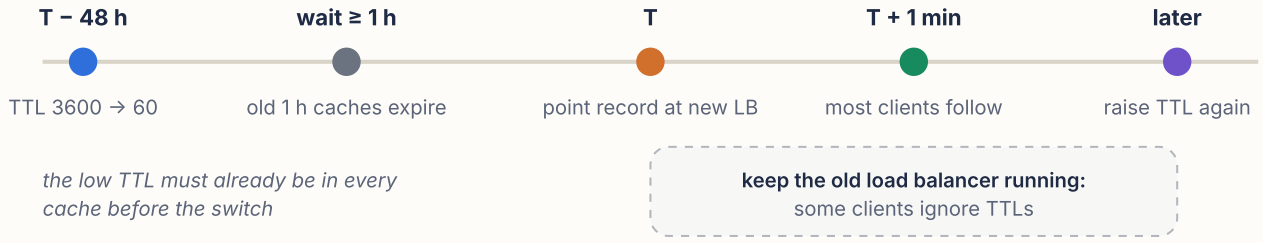


Figure 3. The safe migration. Lowering the TTL only helps once the old, long TTL has expired everywhere, so do it well before the switch.

WORKED EXAMPLE: WHAT GOES WRONG WITHOUT STEP 1

TTL is 3600. You switch the record at 10:00. A resolver that cached the answer at 09:59 keeps serving the old IP until **10:59**, and every user behind it hits the old load balancer for an hour. Lowering the TTL to 60 *at* 10:00 doesn't help them: they won't ask again until 10:59. Lower it to 60 the day before, and the worst case after the switch is about **60 seconds** for well-behaved caches.

WHY NOT KEEP TTLS LOW ALL THE TIME?

Short TTLs mean more lookups, more load (and cost) on DNS, and slightly slower first requests, because more of them miss the cache. Long TTLs are faster and cheaper, but changes take longer to spread. So use **long TTLs** (hours) for stable records and **short ones** (30–60 s) for records you may need to change quickly, such as a failover endpoint.

COMMON MISTAKES

- Lowering the TTL at the same moment as the change. The old, long TTL is still cached.
- Shutting down the old server as soon as DNS changes. Stale caches and long-lived connections still send traffic there.
- Testing from your own laptop and seeing a cached answer. Query the authoritative server directly (`dig @ns1.example.com api.example.com`).
- Treating a CNAME as an HTTP redirect. It is resolved inside DNS, before any connection is made.

6 Key terms

Recursive resolver: does the lookup for clients, caches answers

Root server: points to TLD servers

TLD server: points to a domain's name servers

Authoritative server: holds the real records for a zone

TTL: seconds an answer may be cached

Negative caching: caching "name does not exist"

A / AAAA: name → IPv4 / IPv6

CNAME: alias to another name

NS: which servers are authoritative

7 Interview questions

Q1. Walk through a DNS lookup. BASIC

The client asks a recursive resolver. On a cache miss, the resolver asks a root server (which refers it to the .com TLD), then the TLD server (which refers it to the domain's authoritative servers), then the authoritative server, which returns the record. The answer is cached for its TTL.

Q2. What's the difference between an A record and a CNAME? BASIC

An A record maps a name straight to an IPv4 address. A CNAME maps a name to another name, which must then be resolved; it's how you point `www` at a cloud load balancer whose IPs change.

Q3. What does a TTL of 3600 mean? BASIC

Any cache may keep serving that answer for up to 3,600 seconds (one hour) without asking again. A change can take up to an hour to reach users behind that cache, longer if a client ignores TTLs.

Q4. Why lower TTLs before a migration, and how far ahead? SCENARIO

So that when you switch the record, cached answers expire in minutes, not hours. Lower it at least one old TTL before the switch (a day or two ahead is common), or caches still hold the long value.

Q5. You updated DNS an hour ago and some users still hit the old server. Why? SCENARIO

Their resolver, OS or browser cached the old answer, either from a long TTL or because the client ignores TTLs; long-lived connections also stay put. Keep the old server up and draining until traffic to it stops.

Q6. You created a new subdomain, but it fails to resolve for some users for minutes. Why? SCENARIO

Negative caching. Someone queried it before it existed, and resolvers cached the “does not exist” answer for the time set in the zone's SOA record. It clears when that expires.

Q7. Why not set every TTL to 10 seconds? TRICKY

Far more lookups: more DNS load and cost, more cache misses, slower first requests, and more exposure if the DNS provider has an outage. Use short TTLs only for records you need to move fast.

Q8. Can you put a CNAME at the zone apex (example.com)? TRICKY

No. The apex must hold SOA and NS records, and a CNAME can't coexist with other records. Providers offer ALIAS or ANAME records (Route 53 alias records) that resolve the target and answer with A records instead.

Next: SD-N-016 · DNS part 2: records, TTL and safe migrations backendroadmap.com

DNS part 2: records and TTL

and moving a site with no downtime

Watch the 9-min video

In one line: DNS stores records (A, AAAA, CNAME, MX, TXT, NS); every answer is cached for its TTL; to change safely, lower the TTL early, wait out the old TTL, switch while both servers run, then raise the TTL.

WORDS YOU'LL MEET TODAY

- **A / AAAA record**: a name pointing to an IPv4 / IPv6 address
- **CNAME**: an alias: this name is really that other name
- **MX record**: where email for the domain goes, with a priority
- **TXT record**: free text, used to prove ownership and for email security
- **NS record**: which servers are authoritative for the domain
- **TTL**: time to live: how long an answer may be cached, in seconds
- **Authoritative server**: the server that holds the real records for a domain
- **Negative caching**: caching a “no such name” answer too

1 The question: why do some users still see the old site?

You move your website to a new server and update DNS. Half your users see the new site; the other half still reach the old one, for hours. Nothing is broken. In lesson 15 we followed one DNS lookup. Today: what DNS actually stores, the records, and the one number that decides how long the world remembers them, the TTL. Then, how to change DNS without anyone noticing.

2 An everyday picture first: a number on sticky notes

Your friend's phone number is written on sticky notes all over town, and each note also says: check again in one hour. Your friend changes their number. People who check their note within the hour still call the old number; then they check again and get the new one. If the note had said one week, some people would call the old number for a week. Nothing is broken: they're using what they wrote down. The sticky note is a DNS cache, and “check again in one hour” is the TTL.



Figure 1. The analogy: a phone number copied onto sticky notes that say “check again in 1 hour”. The note is a DNS cache and the reminder is the TTL.

3 DNS records

The records for backendroadmap.com live on its **authoritative servers**. Each record has four parts: a name, a type, a TTL and a value.

- **A**: backendroadmap.com, type A, TTL 3,600, value 198.51.100.10, the IPv4 address. **AAAA** does the same for an IPv6 address.

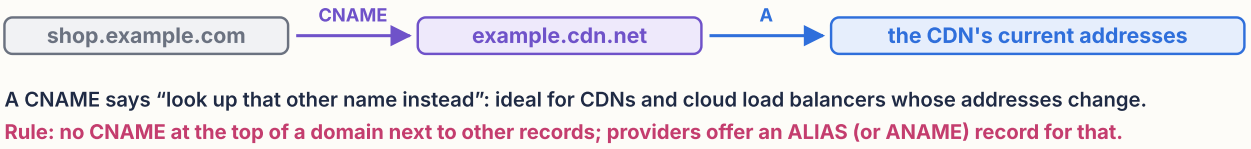
- **CNAME**: an alias. www.backendroadmap.com CNAME backendroadmap.com means: look up that other name instead. It's how you point a name at a CDN or a cloud load balancer whose addresses change.
- **MX**: where email for the domain goes, with a priority number; lower goes first.
- **TXT**: free text, used to prove you own a domain and for email security.
- **NS**: which servers are authoritative for the domain.

name	type	TTL	value
backendroadmap.com	A	3600	198.51.100.10
backendroadmap.com	AAAA	3600	2001:db8::10
www.backendroadmap.com	CNAME	3600	backendroadmap.com
backendroadmap.com	MX	3600	10 mail.backendroadmap.com
backendroadmap.com	TXT	3600	site-verification=...
backendroadmap.com	NS	86400	ns1.provider.net

A and AAAA give an address; CNAME gives another name; MX, TXT and NS carry email, proof and delegation.

Figure 2. The six records you'll see most: A and AAAA give an address, CNAME gives another name, and MX, TXT and NS carry email, proof of ownership and delegation. Each record has a name, a type, a TTL and a value.

One rule: a CNAME can't sit at the top of a domain next to other records. Many DNS providers offer a special alias record for that case.

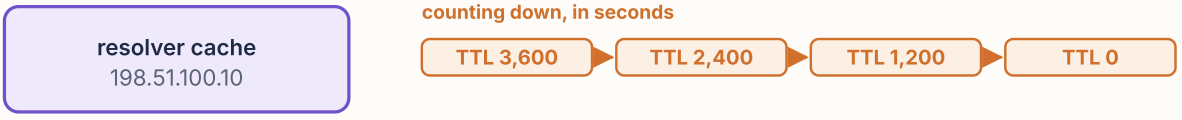


A CNAME says "look up that other name instead": ideal for CDNs and cloud load balancers whose addresses change.
Rule: no CNAME at the top of a domain next to other records; providers offer an ALIAS (or ANAME) record for that.

Figure 3. A CNAME is an alias: the resolver follows it to another name and looks that up. It can't sit at the top of a domain next to other records.

4 The TTL

Every answer carries a **TTL**, in seconds; 3,600 means one hour. The resolver caches the answer and counts down, and until it reaches zero it won't ask again.



Every answer carries a TTL. The resolver keeps the answer and won't ask again until the TTL reaches zero.

Figure 4. The TTL: every DNS answer carries a time to live in seconds; a resolver caches the answer and only asks again when the countdown reaches zero.

Real numbers. A TTL of 300 is five minutes; 86,400 is one day. A short TTL means changes spread fast, but more lookups, and each lookup can add 20 to 100 ms. A long TTL means fewer lookups, but a change can take a whole day to reach everyone.

So the famous line, "DNS changes take 48 hours to propagate", isn't quite right. The new record is live on the authoritative server at once. What you're waiting for is old copies, in caches, running out their TTL.

TTL	lasts	changes reach users	lookups
300	5 minutes	fast	more (each can add 20-100 ms)
86,400	1 day	up to a day	few

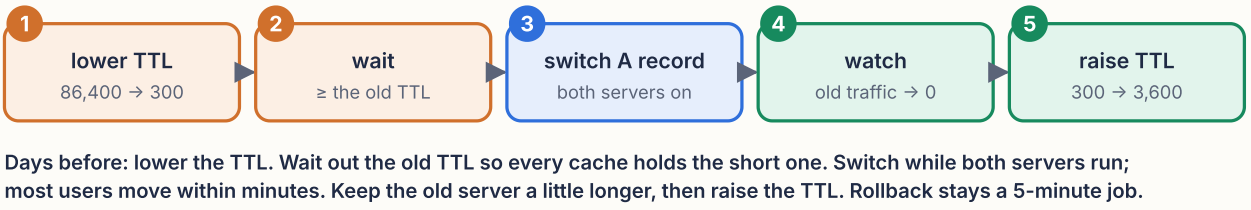
Keep TTLs short around a change, longer the rest of the time.

Figure 5. Short versus long TTLs: short ones spread changes fast but cost more lookups; long ones save lookups but make a change take up to a day.

5 A safe migration, step by step

1. **Days before the move, lower the TTL**, from one day to five minutes.
2. **Wait at least the old TTL**, a full day, so every cache now holds the short one.
3. **Keep the old server running and change the A record** to the new address. Within about five minutes, most users move.
4. **Watch traffic on the old server fall to almost zero**, and keep it alive a little longer for slow caches.
5. **When everything is stable, raise the TTL again**, say to one hour.

And the bonus: for the whole move, rolling back takes only five minutes.



Days before: lower the TTL. Wait out the old TTL so every cache holds the short one. Switch while both servers run; most users move within minutes. Keep the old server a little longer, then raise the TTL. Rollback stays a 5-minute job.

Figure 6. A safe DNS migration: lower the TTL, wait at least the old TTL, switch the record while both servers run, watch traffic move, then raise the TTL again.

6 What goes wrong

- **Skipping step 2.** You lower the TTL and change the address at the same time; caches still hold the old answer with its old TTL, up to a full day.
- **Switching the old server off too early.** Users with the old answer cached get errors until their cache expires.
- **A typo in a record with a long TTL.** The mistake is cached everywhere for a day. Short TTLs during changes make mistakes cheap.
- **Negative caching.** Even "not found" is cached: look up a name before you create it, and the "no such name" answer is remembered too.

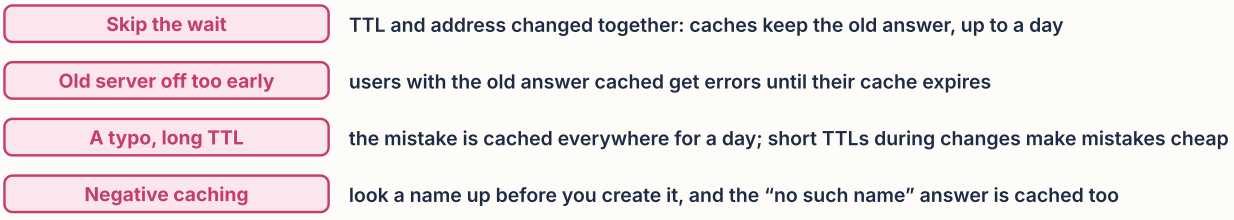


Figure 7. What goes wrong with DNS changes: skipping the wait, switching the old server off too early, a typo cached under a long TTL, and negative caching of a name that didn't exist yet.

PAUSE AND THINK

Your A record has a TTL of one day. You lower it to five minutes and change the address in the same minute. How long until every user reaches the new server? Cover the answer and decide.

Answer. Up to a day. A resolver that cached the old record just before your change keeps it for its full 86,400 seconds; it never saw the new five-minute TTL. Lowering the TTL only helps once the old TTL has run out.

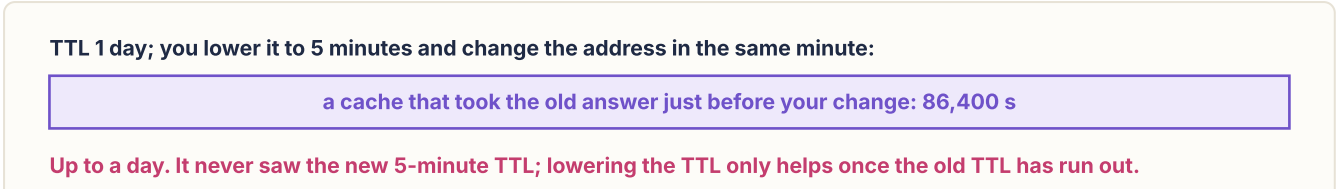


Figure 8. The pause-and-think answer: changing the address in the same minute as lowering the TTL still leaves caches holding the old answer for up to a day.

THREE COMMON MISTAKES

- **"DNS changes take 48 hours."** No. Caches expire by TTL; with planning, the move takes minutes.
- **"A shorter TTL is always better."** No. It means more lookups, more load and a little more latency. Keep it short around changes, longer the rest of the time.
- **"A CNAME is a redirect."** No. The address bar doesn't change; DNS just follows one name to another. An HTTP redirect is different (lesson 19).

RECAP IN THREE LINES

1. DNS stores records: A, AAAA, CNAME, MX, TXT and NS.
2. Every answer is cached for its TTL.
3. To change safely: lower the TTL early, wait out the old TTL, switch, keep the old server, then raise the TTL.

7 Interview questions: what to say in 30 seconds

Q1. Name the common DNS record types and what they do. BASIC

A maps a name to an IPv4 address and AAAA to an IPv6 address; CNAME makes a name an alias of another name; MX lists mail servers with priorities; TXT holds text such as ownership proofs and SPF/DKIM for email; NS names the authoritative servers for a domain. Each record also carries a TTL.

Q2. What is a TTL in DNS, and what's the trade-off in choosing it? BASIC

The time, in seconds, a resolver may cache an answer before asking again. A short TTL (like 300) makes changes reach users fast but causes more lookups and a little more latency; a long TTL (like 86,400) saves lookups but makes changes take up to that long.

Q3. How do you move a website to a new IP address with no downtime? SCENARIO

Days before, lower the record's TTL, say from a day to five minutes, and wait at least the old TTL so every cache holds the short one. Switch the A record while both old and new servers run, watch traffic move to the new one, keep the old server a little longer for slow caches, then raise the TTL again. Rollback is quick throughout.

Q4. Users still reach the old server hours after a DNS change. Why, and what can you do? SCENARIO

Resolvers cached the old answer with its old TTL and will keep it until that expires; some resolvers and clients also hold answers longer. You can't flush other people's caches, so keep the old server running (or proxying to the new one) until traffic drains, and lower TTLs ahead of time next time.

Q5. What's the difference between a CNAME and an HTTP redirect? TRICKY

A CNAME is resolved inside DNS: the resolver follows the alias to another name and returns its address, and the browser's URL doesn't change. An HTTP redirect (301 or 302) is the web server telling the browser to request a different URL, so the address bar changes.

Q6. Why can't you put a CNAME at the root of a domain? TRICKY

A CNAME means the name has no other data of its own, but the root (apex) must have NS and SOA records, and usually MX and TXT. So DNS forbids a CNAME there. Providers offer ALIAS or ANAME records, or CNAME flattening, which resolve the target themselves and return its addresses as A and AAAA records.

Read more. RFC 1034 and RFC 1035 (DNS concepts, records and TTL), RFC 2308 (negative caching), RFC 3596 (AAAA), RFC 7208 (SPF in TXT records), RFC 3849 and RFC 5737 (the documentation addresses used here). The full written lesson is at backendroadmap.com.

Next: SD-N-017 · DNS traffic management backendroadmap.com

DNS Traffic Management: GeoDNS, Weighted, Failover & Anycast

send each user to the right region, and where DNS falls short

Watch the 4-min video

In one line: an authoritative DNS server can give different answers to different users (by location, measured latency, weights or health), which routes traffic between regions; but failover through DNS is slow because of health-check time and caching, so use an anycast global load balancer when you need seconds.

1 One name, different answers

Your app runs in two regions: **eu-west** in Ireland and **us-east** in Virginia. Users in Paris should land in Europe, users in New York in the US, and if Europe goes down, everyone should end up in us-east. DNS can do all of this.

The trick is that an authoritative DNS server doesn't have to give everyone the same answer. With **GeoDNS** (geolocation routing), it looks at where the query comes from and returns the IP of the closest region. A query from Paris gets the eu-west load balancer; a query from New York gets us-east.

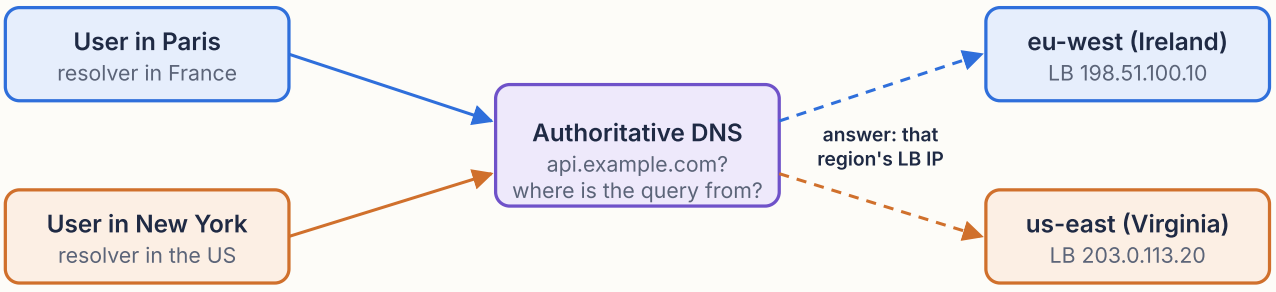


Figure 1. GeoDNS: the same question gets a different answer depending on where it comes from.

Latency-based routing goes one step further. Instead of guessing from geography, the DNS provider measures network latency from different networks to each region and returns the region that is actually fastest for that network. That matters where geography misleads: a user in one country may have a faster path to a region further away. **AWS Route 53** offers both, as well as the policies below.

2 Weighted routing

Weighted routing splits traffic by percentage. Give the current version weight 90 and a new one weight 10, and about 10% of answers point at the new one. It is handy for **canary releases**, **gradual migrations** (move a region or a provider in steps) and **balancing load** between regions of different size.

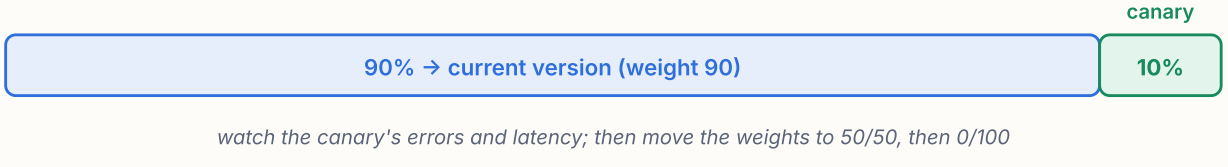


Figure 2. Weighted records send roughly 90% of lookups to the current stack and 10% to the canary.

The split is approximate. It applies to *DNS answers*, not to requests: one big resolver can serve thousands of users the same cached answer for a whole TTL. Over many resolvers it averages out, but don't expect an exact 10%.

3 Failover with health checks

The DNS provider runs **health checks** against each region every few seconds (for example an HTTP request to `/health` every 10 or 30 seconds, from several locations). If eu-west fails several checks in a row, it is marked **unhealthy**, and DNS starts answering with us-east instead. When eu-west recovers and passes its checks again, traffic moves back.

Policies combine: geolocation or latency routing picks the preferred region, and health checks remove a region from the answers when it fails.

4 Anycast: one IP in many places

Anycast is a different tool. The **same IP address** is announced from many locations around the world, and internet routing (BGP) delivers each packet to the nearest one in network terms. Nothing about the DNS answer changes: everyone gets the same IP.

DNS providers use anycast for their own name servers. That makes lookups fast (you reach a nearby server) and spreads out attacks: a DDoS flood is split across every location instead of hitting one. Global load balancers and CDNs use it too, which is the fix for slow DNS failover.

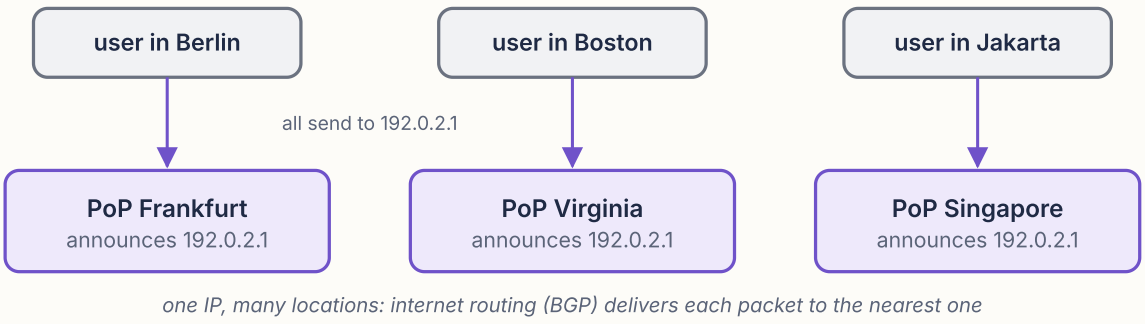
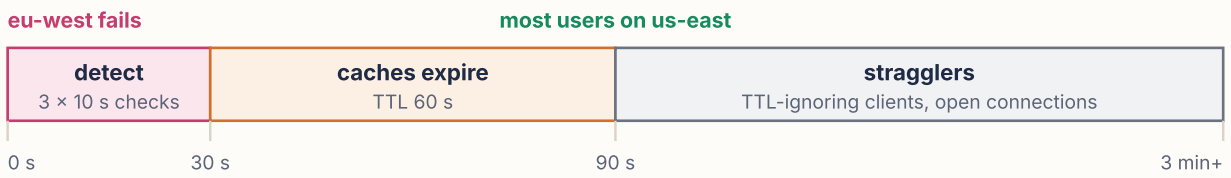


Figure 3. Anycast: three points of presence announce 192.0.2.1. Each user's packets reach the closest one; if one location fails, routing sends them to the next.

5 Why DNS failover is slow

- **Detection.** Several failed health checks are needed to avoid flapping: 3 checks at 10 s is 30 s; at 30 s it is a minute and a half.
- **Caching.** Resolvers keep the old answer until its TTL expires, and some ignore TTLs completely.
- **Connections.** Apps keep their existing connections open (keep-alive, connection pools, WebSockets) and won't look up DNS again until they reconnect.
- **Wrong location.** GeoDNS sees the *resolver's* location, not the user's. A user in Paris whose resolver is far away may land in the wrong region. (Some resolvers forward part of the client's subnet, called EDNS Client Subnet, which helps.)



a global load balancer with an anycast IP shifts traffic in seconds: no DNS answer has to change

Figure 4. A rough DNS failover timeline with fast checks and a 60 s TTL: about a minute and a half for most users, minutes or more for the rest.

WORKED EXAMPLE: THE FAILOVER BUDGET

Health checks every **10 s**, failure after **3** misses: detection takes about **30 s**. The record's TTL is **60 s**, so caches may serve the dead region for up to another minute: about **90 s** total for most users. With 30 s checks and a 300 s TTL it becomes 90 + 300 ≈ **6.5 minutes**. If the target is "under 10 seconds", DNS can't meet it. Put a **global load balancer with an anycast IP** in front (for example AWS Global Accelerator or Cloudflare): its health checks move traffic between regions behind the same IP, so no cache has to expire.

COMMON MISTAKES

- Promising near-instant failover with DNS alone.
- A long TTL on a failover record. The failover can only be as fast as the TTL allows.
- Assuming GeoDNS knows where the user is. It knows where the resolver is.
- Expecting weights to split traffic exactly.
- Health checks that only test "is the web server up", not whether the region can actually serve requests (database, dependencies).

6 Key terms

GeoDNS: answer by the query's location

Health check: probe that marks a target healthy or not

BGP: internet routing between networks

Latency-based routing: answer with the fastest measured region

DNS failover: stop answering with an unhealthy region

Canary: small share of traffic on a new version

Weighted routing: split answers by percentage

Anycast: one IP announced from many places

Global load balancer: anycast front door across regions

7 Interview questions

Q1. How can DNS route users to the nearest region? BASIC

GeoDNS answers based on where the query comes from; latency-based routing returns the region with the lowest measured latency from the user's network. Both are features of the authoritative DNS provider, such as Route 53.

Q2. What is anycast? BASIC

The same IP address announced from many locations. Internet routing delivers each packet to the nearest one. DNS providers use it for their name servers: lookups are fast and DDoS traffic is spread across every site.

Q3. Why is DNS failover slow? BASIC

Health checks need time to detect the failure (often 30–90 s), then cached answers live until their TTL expires, sometimes longer when clients ignore TTLs, and open connections stay on the old region.

Q4. Roll out a new version to 5% of users at the DNS level. How? SCENARIO

Weighted records: 95 for the current stack, 5 for the new one. Watch error rate and latency, then step up to 25, 50, 100. Keep the TTL short (60 s) so you can roll back quickly, and remember the split is approximate.

Q5. Users in Paris sometimes land in us-east. Why? SCENARIO

GeoDNS sees the resolver's location, not the user's. If they use a resolver located in the US, or one without EDNS Client Subnet, they get the US answer. Latency-based routing or an anycast front door reduces this.

Q6. The requirement is failover between regions in under 10 seconds. Design it. SCENARIO

Don't rely on DNS. Put a global load balancer with an anycast IP in front of both regions; it health-checks the regional backends and shifts traffic within seconds behind the same IP. Keep DNS as a slower backup layer.

Q7. You set TTL to 60 s, but after failover some traffic hits the dead region for 30 minutes. Why? TRICKY

Some resolvers and client runtimes cache longer than the TTL, and long-lived connections (pools, WebSockets, HTTP keep-alive) never re-resolve until they break. Close idle connections periodically and have clients retry against a fresh lookup.

Q8. Weighted 90/10, but the canary gets 30% of traffic. How? TRICKY

Weights apply to DNS answers, not requests. A few large resolvers (a big ISP, a corporate network) that happened to get the canary answer serve it to all their users for the whole TTL. It evens out over time, not per minute.

Next: SD-N-018 · HTTP part 1: requests, responses and methods backendroadmap.com

HTTP Fundamentals: Methods, Status Codes, 401 vs 403 & ETags

the details interviewers love

Watch the 5-min video

> **In one line:** HTTP is stateless request → response; idempotent methods are safe to retry, 401 means “who are you?” and 403 “not allowed”, and ETags turn a re-download into a tiny 304.

1 Requests and responses

Every API call you make is HTTP: a request goes out and a response comes back. A **request** has a **method**, a **path**, **headers** and an optional **body**. For example `GET /products/42` with headers like `Host`, `Authorization` and `Accept`. The **response** has a **status code**, headers and a body, like `200 OK` with some JSON.

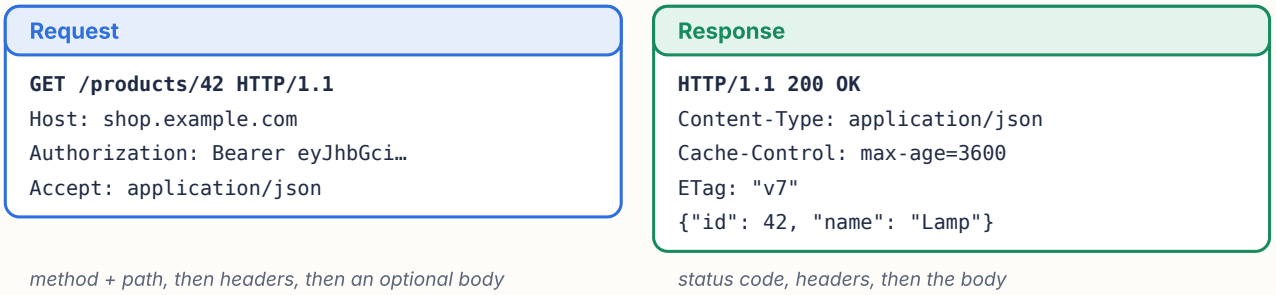


Figure 1. One request and its response. Headers say who is calling, what format is wanted and how long the answer may be cached.

HTTP itself is **stateless**: each request stands alone. Cookies or tokens, sent on every request, carry the session, so any server behind a load balancer can handle the next one.

2 Methods: safe and idempotent

Safe means the request doesn't change anything on the server. **Idempotent** means repeating it has the same effect as sending it once. Idempotency is what makes a retry safe after a timeout, when you don't know if the first try worked.

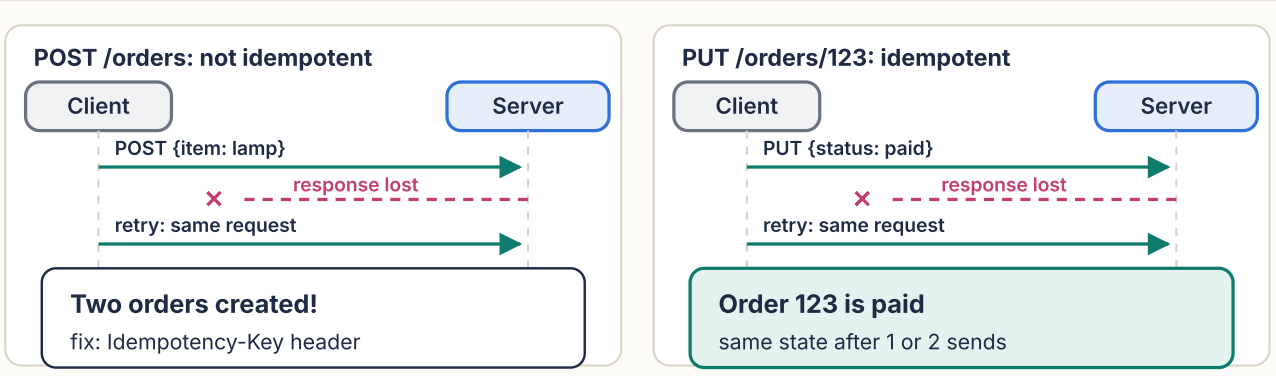


Figure 2. The response to the first send is lost, so the client retries. A second POST creates a second order; a second PUT leaves the same state.

Method	Meaning	Safe	Idempotent
GET	Read a resource	yes	yes
HEAD	Like GET, headers only (check size, ETag, existence)	yes	yes
OPTIONS	Which methods are allowed (used by CORS preflight)	yes	yes
PUT	Replace the resource with this full version	no	yes
DELETE	Remove the resource	no	yes
POST	Create a resource, or run an action	no	**no**
PATCH	Update part of a resource	no	not guaranteed

So a client can safely retry a `PUT`, but retrying a `POST` might create a duplicate order. `PATCH` depends on the change: "set status to paid" is idempotent, "add 1 to quantity" is not.

3 Status codes come in families

Family	Meaning	Common codes
2xx	Success	`200 OK`, `201 Created` (after a POST, with a `Location` header), `204 No Content`
3xx	Redirect, or use your cache	`301` / `302` redirects, `304 Not Modified`
4xx	The client made a mistake	`400 Bad Request`, `401`, `403`, `404 Not Found`, `429 Too Many Requests` (rate limited)
5xx	The server failed	`500 Internal Server Error`, `502 Bad Gateway` (a proxy got a bad answer from upstream), `503 Service Unavailable` (overloaded or down)

The family tells a client what to do. A 4xx won't succeed if repeated unchanged (except 429, after waiting). A 5xx or 429 may be worth retrying with backoff, if the method is idempotent.

4 401 vs 403

A classic interview question. **401 Unauthorized** really means **unauthenticated**: we don't know who you are, so log in or send a valid token (the response includes a `WWW-Authenticate` header saying how). **403 Forbidden** means we know exactly who you are, but you're **not allowed** to do this. A normal user opening an admin page gets a 403. Logging in again won't help.

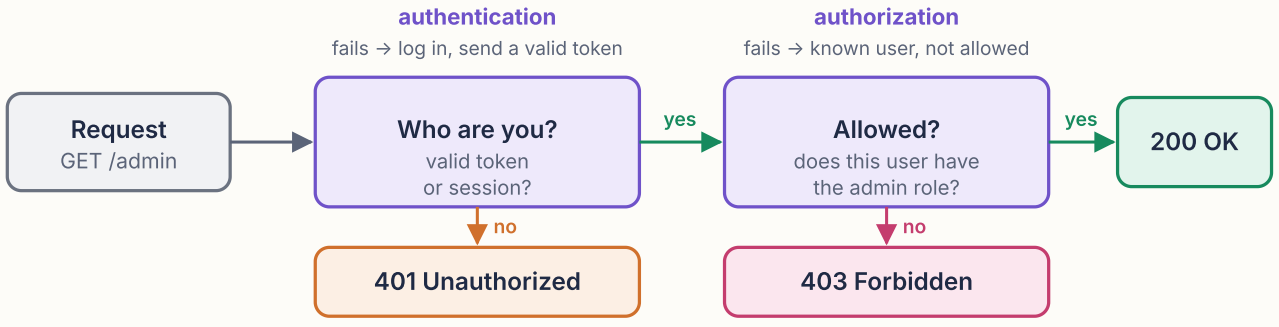


Figure 3. Authentication comes first (fails with 401), then authorization (fails with 403).

5 Caching headers and ETags

Caching headers let browsers and CDNs reuse responses. `Cache-Control` says how long a response stays fresh, like `max-age=3600` (one hour), or `no-store` for private data that must never be cached. **Content negotiation** uses headers too: the client sends `Accept: application/json` or `Accept-Encoding: gzip`, and the server picks the best format it can give and says which in `Content-Type` and `Content-Encoding`.

An **ETag** is a version tag for a response, like `ETag: "v7"`. Next time, the client sends `If-None-Match: "v7"`. If nothing changed, the server replies **304 Not Modified** with no body at all, and the client reuses its cached copy. If it did change, the server sends 200 with the new body and a new ETag.

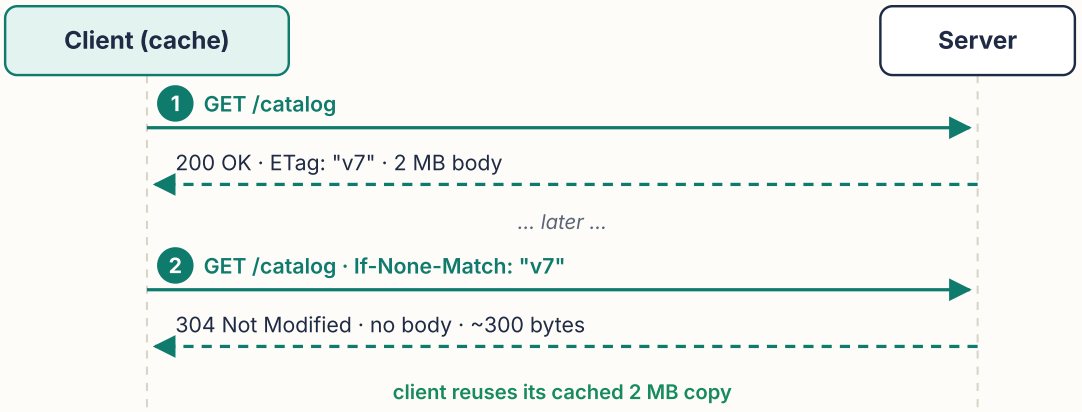


Figure 4. Revalidating with an ETag. The second exchange costs a few hundred bytes instead of 2 MB.

WORKED EXAMPLE: A PRODUCT CATALOG

The catalog response is **2 MB** and changes a few times a day. 100,000 app opens a day, each fetching it: **200 GB** of transfer. With an ETag, unchanged fetches become 304s of about **300 bytes**: 100,000 × 300 B ≈ **30 MB**, plus a few full downloads after each change. Add `Cache-Control: max-age=300` and most opens don't even ask for 5 minutes.

COMMON MISTAKES

- Automatically retrying POST on timeout without an idempotency key: duplicate orders and double charges.
- Returning 401 when the user is logged in but lacks permission (that's 403), or 200 with an error message in the body.
- Changing data in a GET handler. Crawlers, prefetchers and caches assume GET is safe.
- Using `no-cache` when you mean `no-store`. `no-cache` still stores the response, it just revalidates before every use.

6 Key terms

Stateless: each request stands alone

Safe method: doesn't change server state

Idempotent: repeating = same effect as once

401: not authenticated: who are you?

403: authenticated, but not allowed

Cache-Control: how long a response stays fresh

Content negotiation: Accept headers pick the format

ETag: version tag of a response

304 Not Modified: your copy is current, no body

7 Interview questions

Q1. Which HTTP methods are idempotent? BASIC

GET, HEAD, PUT, DELETE and OPTIONS. POST is not, so retries can create duplicates unless you add an idempotency key; PATCH is not guaranteed to be.

Q2. What's the difference between 401 and 403? BASIC

401 means you're not authenticated: send valid credentials. 403 means you're authenticated but not allowed: a normal user on an admin page. Re-logging in fixes a 401, not a 403.

Q3. What does it mean that HTTP is stateless? BASIC

The server keeps no memory between requests; each one carries everything needed, with cookies or tokens identifying the session. That's what lets any server behind a load balancer handle any request.

Q4. A payment POST times out. Can the client retry? SCENARIO

Only safely with an idempotency key: the client sends a unique `Idempotency-Key` header, the server stores the result under that key, and a retry with the same key returns the stored result instead of charging twice.

Q5. Your mobile app downloads a 2 MB catalog on every launch. How do you cut bandwidth? SCENARIO

Return an ETag (and a short `max-age`). The app sends `If-None-Match`; when nothing changed, the server answers 304 with no body, about 300 bytes instead of 2 MB. Also serve it gzip-compressed via `Accept-Encoding`.

Q6. Users get 502 from the load balancer, and 503 at peak. What do they tell you? SCENARIO

502: the load balancer got an invalid response from a backend (it crashed or closed the connection). 503: the service is overloaded, has no healthy backends, or is in maintenance; clients should back off, ideally honouring `Retry-After`.

Q7. DELETE is idempotent, but the second DELETE returns 404. Is that a contradiction? TRICKY

No. Idempotency is about the effect on server state, not the response. After one or two DELETEs the resource is gone either way; the status code can differ.

Q8. Is PATCH idempotent? TRICKY

Not guaranteed. A PATCH that sets fields (`{"status": "paid"}`) is idempotent; one that applies a relative change ("add 1 to quantity") is not. Treat PATCH like POST for retries unless your API defines it otherwise.

Next: SD-N-019 · HTTP part 2: status codes, 401 vs 403, caching and ETags backendroadmap.com

HTTP part 2: status codes and caching

401 vs 403 · Cache-Control · ETags

Watch the 9-min video

In one line: the first digit names the family: 2xx success, 3xx redirect, 4xx the client's fault, 5xx the server's; 401 is “who are you?”, 403 is “not allowed”; Cache-Control says how long to reuse a response, and ETags with 304 make checking cheap.

WORDS YOU'LL MEET TODAY

Status code: the three-digit result of an HTTP request

2xx / 3xx: success / go somewhere else (a redirect)

4xx: a client error: the request was wrong

5xx: a server error: the server failed

Location header: where a redirect, or a newly created thing, is

Cache-Control: how long, and by whom, a response may be reused

ETag: a fingerprint of one version of the content

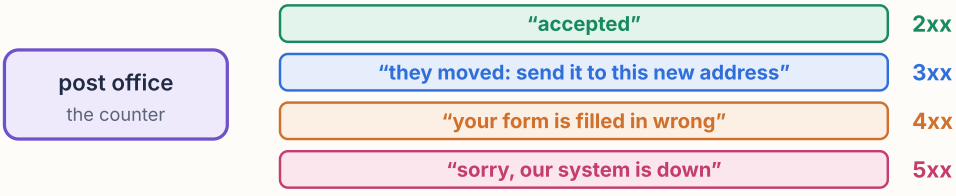
304 Not Modified: your cached copy is still good; no body sent

1 The question: what is the server telling you?

Your app shows “something went wrong”, but the server sent a number: 403. Another time, 503. Each number tells you exactly where to look, and one header can make a page load with almost no data at all. In lesson 18 we met HTTP requests and responses; today, the status codes and HTTP caching.

2 An everyday picture first: the post office counter

You hand a parcel over at a post office counter. The clerk can say: accepted. Or: they've moved, send it to this new address. Or: your form is filled in wrong. Or: sorry, our system is down. And if you ask “has my parcel changed since this morning?”, the clerk can just say “no, same as before”, without handing it over again. That's caching.



And “has it changed since this morning?” → “no, same as before”, with nothing handed over: that's caching.

Figure 1. The analogy: a post office clerk's four kinds of answer match the status code families, and “no, same as before” is caching.

3 Five families, by the first digit

1xx is information, and rare. 2xx is success. 3xx means go somewhere else. 4xx means the client made a mistake. 5xx means the server failed. The first digit tells you who to look at.

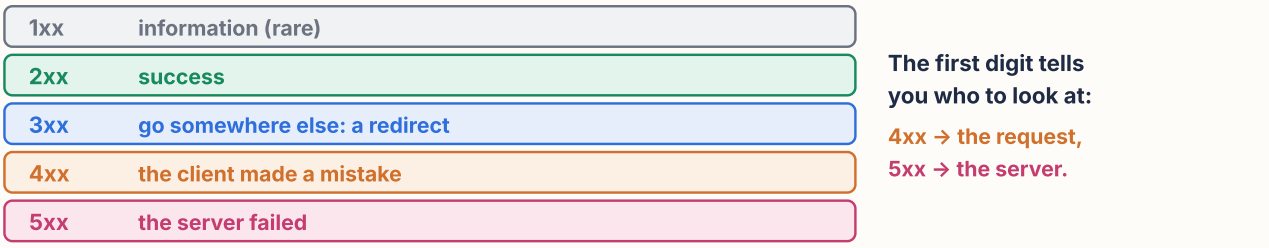


Figure 2. Status codes come in five families by their first digit: information, success, redirect, client error and server error.

- **200 OK**: here's what you asked for. **201 Created**: your POST made something new, and its address is in the Location header. **204 No Content**: it worked, nothing to send back, like after a delete.
- **301 Moved Permanently**: browsers and search engines remember the new address. **302**: moved, but only for now. The new URL is in the Location header and the address bar changes: that's the difference from a CNAME (lesson 16).

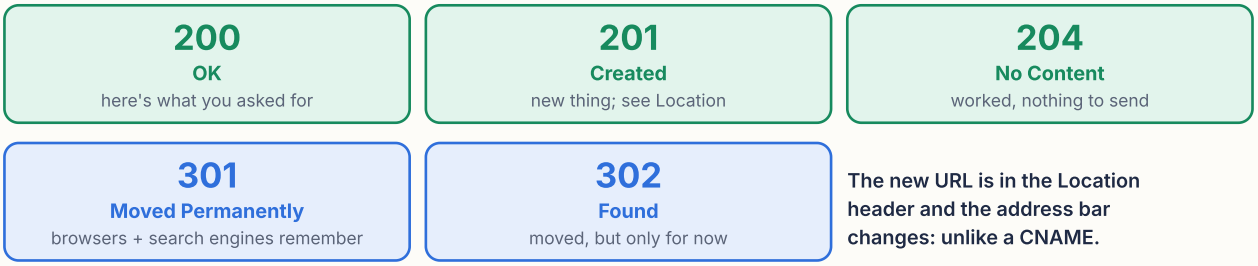


Figure 3. Success and redirects: 200, 201 and 204; 301 and 302 send the browser to the URL in the Location header.

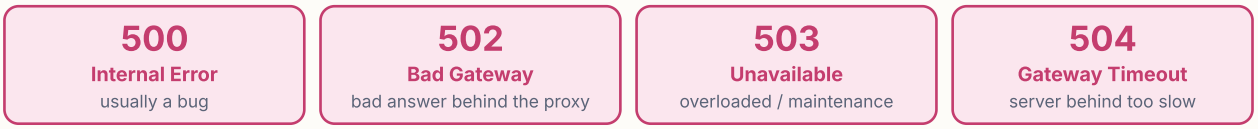
- **400 Bad Request**: the request itself is broken. **404 Not Found**. **409 Conflict**: it clashes with the current state, like a duplicate username. **429 Too Many Requests**: you hit a rate limit, so wait, often for the time in the Retry-After header.
- **401 versus 403.** 401, called Unauthorized, really means not authenticated: who are you? Send valid credentials. 403 Forbidden means: I know who you are, and you're not allowed. Logging in again won't help.
- **404 versus 410.** 404: not found, and maybe it never existed. 410 Gone: it existed, and was removed on purpose.



Figure 4. Client errors, and the pairs interviewers love: 401 means not authenticated, 403 means authenticated but not allowed; 404 may never have existed, 410 was removed on purpose.

Server errors. 500 Internal Server Error: usually a bug in the code. 502 Bad Gateway: a proxy or load balancer got a bad answer from the server behind it. 503 Service Unavailable: overloaded or down for maintenance, try later. 504 Gateway Timeout: the server behind didn't answer in time.

A rule for retries. 5xx and 429 may succeed if you try again, with a growing wait between tries. Most 4xx won't fix themselves: don't retry them, fix the request.



Retry with a growing wait (backoff): 5xx and 429.

Don't retry most 4xx: they won't fix themselves. Fix the request.

502 and 504 come from a proxy or load balancer: the problem is the server behind it.

Figure 5. Server errors, and the rule for retries: 5xx and 429 may succeed with backoff; most 4xx won't, so fix the request.

4 HTTP caching

A response can say `Cache-Control: max-age=3600`: the browser may reuse it for an hour without asking at all, the same idea as the DNS TTL in lesson 16. Other values: **no-store**, never keep a copy, like a bank statement; **no-cache**, you may keep it but check before every use; **private**, only the user's own browser may keep it; **public**, shared caches like CDNs may keep it too.

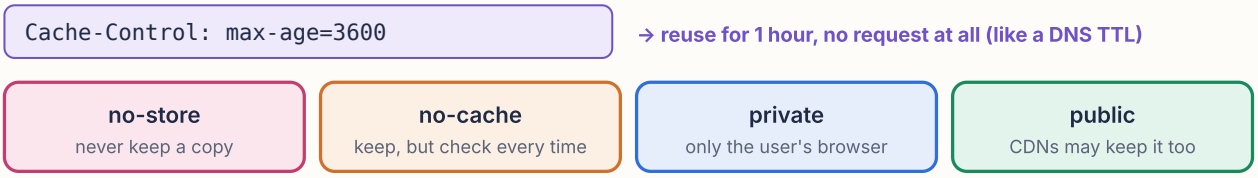
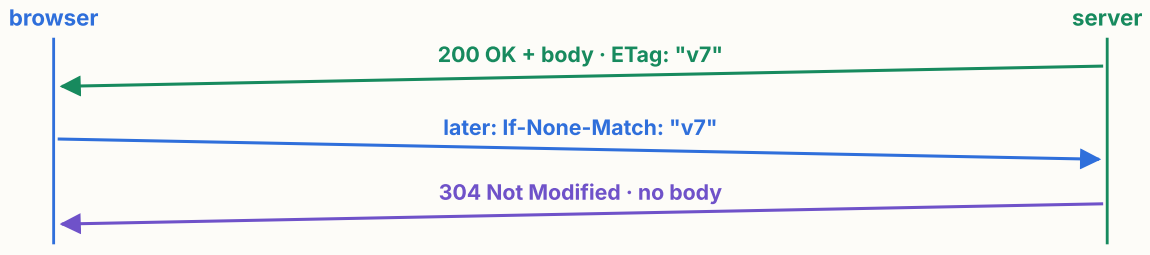


Figure 6. Cache-Control: max-age says how long a response may be reused without asking; no-store, no-cache, private and public say who may keep it and how.

When max-age runs out, the browser can check cheaply with an **ETag**. The first response says `ETag: "v7"`. Next time the browser asks `If-None-Match: "v7"`, and if nothing changed, the server replies **304 Not Modified**, with no body at all.

Real numbers. A 200 KB image: the first visit costs 200 KB; a check that returns 304 costs a few hundred bytes and one round trip; while max-age is still fresh, zero bytes and zero round trips.

A common pattern. Put a fingerprint in the file name, like `app.3f9a.js`, and cache it for a year. When you deploy, the name changes, so users get the new file at once. The HTML page itself gets a short max-age, or no-cache.



A 200 KB image: first visit 200 KB; a 304 check a few hundred bytes and one round trip; while fresh, 0 bytes and 0 round trips.

Figure 7. Cheap checks with an ETag: the browser sends If-None-Match with the version it has, and gets 304 Not Modified with no body if nothing changed.

5 What goes wrong

- **Errors sent as 200 OK**, with “error” written in the body: monitoring thinks everything is fine, and caches happily store the error page.

- **A personal page cached as public**: a CDN serves one user's account page to someone else. Mark it private, or no-store.
- **A long max-age on a file without a fingerprint**: after a deploy, users keep the old version until their cache runs out.

PAUSE AND THINK

A logged-in user opens an admin page they aren't allowed to see: which status code? A user whose login token has expired calls the API: which status code? Cover the answer and decide.

Answer. The first is 403: the server knows who they are, and the answer is no. The second is 401: the credentials are no longer valid, so log in again and try again.

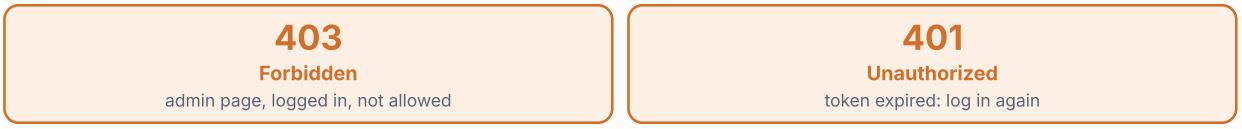


Figure 8. The pause-and-think answer: a logged-in user on a page they may not see gets 403; a request with an expired token gets 401.

THREE COMMON MISTAKES

- **"401 means no permission."** No. 401 means we don't know who you are. No permission is 403.
- **"no-cache means don't cache."** No. It means always check before using. The one that means never keep a copy is no-store.
- **"A 304 is an error."** No. It's a success: your copy is still good, so nothing needs to be sent.

RECAP IN THREE LINES

1. The first digit names the family: 2xx success, 3xx redirect, 4xx your fault, 5xx the server's.
2. 401 is "who are you?", 403 is "not allowed".
3. Cache-Control says how long to reuse a response, and ETags with 304 make checking cheap.

6 Interview questions: what to say in 30 seconds

Q1. What do the HTTP status code families mean? BASIC

1xx is informational; 2xx success, such as 200, 201 and 204; 3xx redirects, with the new URL in the Location header; 4xx client errors, such as 400, 404 and 429, where the request needs fixing; 5xx server errors, such as 500, 502, 503 and 504, where the server or something behind it failed.

Q2. What's the difference between 401 and 403? BASIC

401 (named Unauthorized) means not authenticated: the server doesn't know who you are, or your credentials are missing, invalid or expired, so authenticate and retry. 403 Forbidden means the server knows who you are and you're not allowed; logging in again won't help.

Q3. Which errors should a client retry? SCENARIO

5xx errors such as 502, 503 and 504, and 429 Too Many Requests, may succeed later: retry with exponential backoff and jitter, honour Retry-After, and only retry requests that are safe to repeat (idempotent, or protected by an idempotency key). Most 4xx errors won't fix themselves, so don't retry them.

Q4. How would you cache static files and HTML for a web app? SCENARIO

Put a content hash in static file names (app.3f9a.js) and serve them with a long max-age, even a year, marked immutable; a deploy changes the name, so users get the new file at once. Serve HTML with a short max-age or no-cache plus an ETag, so browsers revalidate cheaply. Mark personal pages private or no-store.

Q5. What's the difference between no-cache and no-store? TRICKY

no-cache allows storing a copy but requires revalidation with the server before every use, typically with an ETag and If-None-Match. no-store forbids keeping a copy at all, for sensitive responses like bank statements.

Q6. How do ETags and 304 work, and what do they save? TRICKY

The server labels a response with an ETag, a version fingerprint. When the cached copy is stale, the browser sends If-None-Match with that ETag; if the content hasn't changed the server replies 304 Not Modified with no body. That saves the bytes of the response, though it still costs a round trip; only a fresh max-age saves the round trip too.

Read more. RFC 9110 (HTTP semantics: status codes, Location, ETag and conditional requests), RFC 9111 (HTTP caching: Cache-Control), RFC 6585 (429 Too Many Requests), and MDN's HTTP status code and caching guides. The full written lesson is at backendroadmap.com.

Next: SD-N-020 · Why encryption: public and private keys backendroadmap.com

Why encryption: public and private keys

the lockbox · the padlock · why HTTPS uses both

Watch the 9-min video

In one line: without encryption every packet is a postcard; symmetric encryption (one key) is fast but the key must be shared, public and private keys solve the sharing but are slow, so HTTPS uses both: agree one secret, then encrypt fast with AES.

WORDS YOU'LL MEET TODAY

- **Encryption**: scrambling data so only the right person can read it
- **Symmetric**: the same key locks and unlocks (AES)
- **Private key**: the other half, kept only by the owner
- **Signature**: made with a private key, checked with the public key
- **Key**: the secret value that locks and unlocks
- **Public key**: half of a key pair that anyone may have
- **Key exchange**: both sides compute the same secret without sending it
- **Forward secrecy**: fresh keys per connection, so an old leak can't unlock old traffic

1 The question: what can someone on the café Wi-Fi see?

You're on café Wi-Fi and log in to a site over plain HTTP. Anyone on the same network can read your password. Worse, they can change the page before it reaches you. In lesson 19 we met status codes and caching; today, why encryption matters and how public and private keys make it work.

2 An everyday picture first: a postcard

Without encryption, every packet is a postcard. It passes the café router, your internet provider and many routers after that, and anyone holding it on the way can read it.

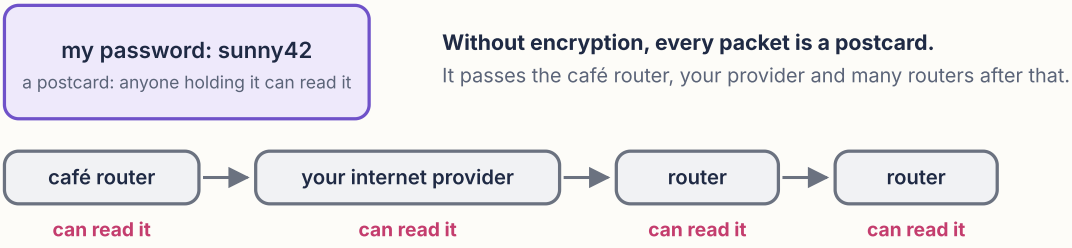


Figure 1. Without encryption every packet is a postcard: the café router, your internet provider and every router after that can read it.

Three things can go wrong. Someone **reads** it: privacy. Someone **changes** it: integrity. Someone **pretends** to be your bank: authenticity. Today we fix the first one; lessons 21 and 22 finish the rest.



Figure 2. Three things can go wrong on the wire: someone reads it (privacy), changes it (integrity), or pretends to be the other side (authenticity).

3 The simplest fix: a lockbox with one key

The same key locks and unlocks. This is **symmetric** encryption, and the common one is called **AES**. It's very fast: a laptop can encrypt gigabytes every second.

But there's a catch. Both sides need the same key. If you send it over the café Wi-Fi, the snooper copies the key too. This is the **key distribution problem**.



Symmetric encryption (AES): one key locks and unlocks. Very fast, but how do you share the key? The key distribution problem.

Figure 3. Symmetric encryption uses one key to lock and unlock, and is very fast; the catch is getting that key to the other side without anyone copying it.

4 Public and private keys: the padlock picture

A bank hands out open padlocks, free, to anyone. Only the bank has the key that opens them. You put your message in a box and snap a padlock shut. Now nobody can open it, not even you. Only the bank.

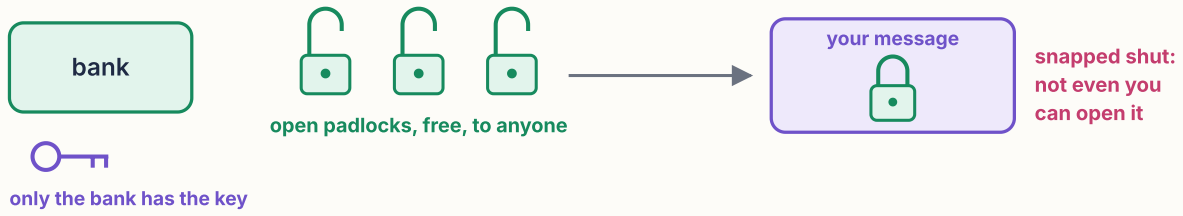
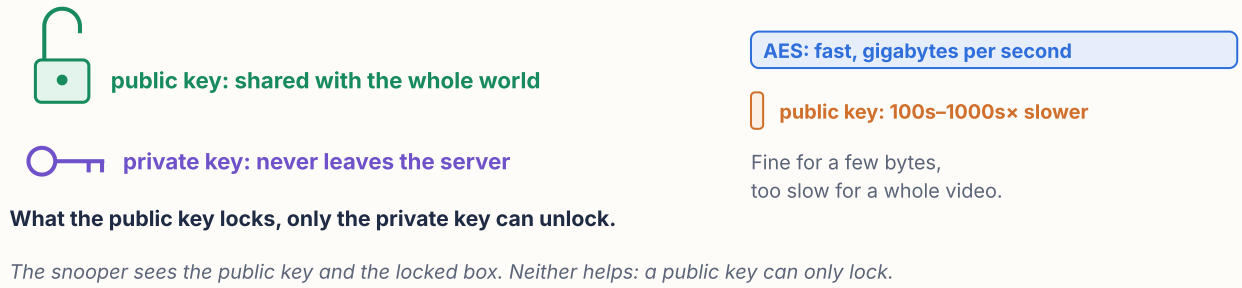


Figure 4. The padlock picture: the bank hands out open padlocks to anyone but keeps the only key; once you snap one shut on your box, only the bank can open it.

The real thing is a **key pair**. The **public key**, like the open padlock, is shared with the whole world. The **private key** never leaves the server. What the public key locks, only the private key can unlock. So a snooper who sees the public key and the locked box learns nothing: a public key can only lock.

Then why not use public keys for everything? Because they're slow: public-key operations are hundreds to thousands of times slower than AES. Fine for a few bytes, too slow for a whole video.



The snooper sees the public key and the locked box. Neither helps: a public key can only lock.

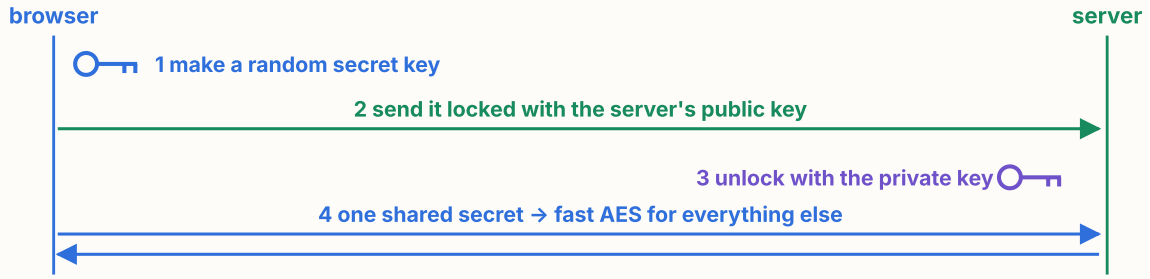
Figure 5. A key pair: the public key is shared with everyone, the private key never leaves the server; public-key operations are much slower than AES.

5 Why HTTPS uses both

1. The browser makes a random secret key.
2. It locks that key with the server's public key, and sends it.

3. The server unlocks it with its private key.
4. Now both sides share one secret, and switch to fast AES for everything else.

That's the classic idea. Modern **TLS 1.3** uses a cousin called a **key exchange**, or Diffie-Hellman: each side keeps a secret and sends only a public part, and both compute the same shared key, which is never sent at all. The snooper sees both public parts and still can't work it out.



TLS 1.3 uses a cousin, a Diffie-Hellman key exchange: each side sends only a public part, and both compute the same key, which is never sent at all.

Figure 6. HTTPS uses both: slow public-key crypto to agree one secret, then fast AES for the data. TLS 1.3 agrees it with a Diffie-Hellman key exchange.

The other way round: signatures. The server can sign something with its private key, like a wax seal. Anyone with the public key can check the seal is real. That proves who sent it, and it's the base of certificates, in lesson 22.

Real numbers. An AES key is 128 or 256 bits. An RSA key is usually 2,048 bits, and an elliptic curve key just 256. Guessing a 128-bit key means up to 2^{128} tries: even with every computer on Earth, that takes far longer than the age of the universe.

What HTTPS hides. The page content, the full URL path, your cookies and your password. What stays visible: your IP address, the server's, the site's domain name, and how much data moves, and when.

✓ hidden by HTTPS	× still visible
the page content	your IP address
the full URL path (/account/settings)	the server's IP address
cookies	the domain name (backendroadmap.com)
your password	how much data moves, and when

Figure 7. What HTTPS hides (content, URL path, cookies, passwords) and what stays visible (both IP addresses, the domain name, and how much data moves, and when).

6 What goes wrong

- **A leaked private key.** Anyone who has it can pretend to be your server. With old setups, they could even unlock traffic recorded months ago. That's why TLS 1.3 makes fresh keys for every connection: **forward secrecy**.
- **Home-made encryption.** Base64 is not encryption: anyone can decode it in a second. A clever scheme you invented is almost always broken. Use standard, well-tested libraries.
- **Encrypting to the wrong person.** If an attacker swaps in their own public key, you lock your secret for the attacker: a man in the middle. Encryption alone doesn't say who you're talking to. Certificates fix that, in lesson 22.

PAUSE AND THINK

An attacker on the café Wi-Fi copies the server's public key and your locked box. Can they read your message? And what if, before the key reached you, they swapped it for their own? Cover the answer and decide.

Answer. The first: no. A public key only locks, so copying it doesn't help. The second: yes. You locked your secret with the attacker's padlock. That's exactly why the browser must check who the key belongs to.



You lock your secret with the attacker's padlock: they read it, then pass it on. Encryption alone doesn't say who you're talking to. Certificates (lesson 22) let the browser check who the public key belongs to.

Figure 8. The pause-and-think answer: copying a public key doesn't help an attacker, but swapping in their own does. That's a man in the middle, and why certificates exist.

THREE COMMON MISTAKES

- **"The public key must be kept secret."** No. It's meant to be shared. Only the private key is secret.
- **"HTTPS hides which site I visit."** No. The domain name and IP addresses are still visible. The content and the URL path are hidden.
- **"Public keys are better, so use them for everything."** No. They're slow. They're used to agree on a key, then fast symmetric encryption does the work.

RECAP IN THREE LINES

1. Symmetric: one shared key, fast, but how do you share it?
2. Public and private keys: the public key locks, only the private key unlocks.
3. HTTPS uses both: agree a secret, then encrypt fast with AES.

7 Interview questions: what to say in 30 seconds

Q1. What's the difference between symmetric and asymmetric encryption? BASIC

Symmetric encryption, such as AES, uses one shared key to encrypt and decrypt; it's very fast, but both sides need the key, and getting it there safely is the key distribution problem. Asymmetric encryption uses a key pair: the public key is shared with anyone, the private key never leaves its owner. What the public key encrypts, only the private key can decrypt. It solves sharing but is far slower.

Q2. Why does HTTPS use both kinds? BASIC

Asymmetric crypto is too slow for bulk data, and symmetric crypto alone has no safe way to share the key. So TLS uses asymmetric crypto, today an ephemeral Diffie-Hellman key exchange, to agree a fresh session key, then encrypts all the data with fast symmetric AES (or ChaCha20). Certificates and signatures prove the server's public key really belongs to the site.

Q3. What does an attacker on public Wi-Fi see when you use HTTPS? SCENARIO

Your IP address and the server's, the domain name (from DNS, and the SNI field in the TLS handshake unless Encrypted Client Hello is used), and the timing and size of the traffic. They can't see the URL path, the page content, cookies or form data such as passwords, and they can't change them without being detected.

Q4. A server's private key leaks. What's the damage? SCENARIO

An attacker can impersonate the server, so the certificate must be revoked and the key replaced. With old RSA key transport, they could also decrypt any recorded past traffic. TLS 1.3 only uses ephemeral key exchanges, which give forward secrecy: each connection has fresh keys, so recordings stay safe.

Q5. Is Base64 or hashing encryption? TRICKY

No. Base64 is an encoding: anyone can reverse it, with no key. A hash is one-way: it can't be decrypted at all, which suits checking passwords (with a slow, salted hash) but not hiding data you need back. Encryption needs a key to reverse. And never invent your own scheme: use standard, well-tested libraries.

Q6. If public keys are public, how can an attacker in the middle break things? TRICKY

Copying the public key doesn't help, since it can only lock. But if the attacker swaps in their own public key, you encrypt for the attacker, who reads the data and passes it on: a man in the middle. Encryption alone doesn't say who you're talking to; certificates signed by a trusted authority let the browser check the key belongs to the site.

Read more. RFC 8446 (TLS 1.3), RFC 7748 (the X25519 elliptic curve key exchange), NIST FIPS 197 (AES), and Cloudflare's learning centre pages on encryption and public key cryptography. The full written lesson is at backendroadmap.com.

Next: SD-N-021 · HTTPS and the TLS handshake backendroadmap.com

HTTPS & TLS 1.3 Handshake

ECDHE, forward secrecy and 0-RTT

Watch the 5-min video

In one line: HTTPS is HTTP inside TLS; the TLS 1.3 handshake agrees on fresh secret keys with ECDHE and proves the server's identity with a certificate, all in one round trip, then switches to fast symmetric encryption.

1 What TLS guarantees

Every padlock in the browser means a TLS handshake just happened, usually in tens of milliseconds. **HTTPS** is plain HTTP carried inside a **TLS** connection (port 443). TLS adds three guarantees:

- **Confidentiality:** nobody on the network can read the traffic.
- **Integrity:** nobody can change a byte without the receiver noticing.
- **Authentication:** the server proves who it is with a certificate (the next lesson).

2 The TLS 1.3 handshake, step by step

One round trip both agrees on keys and proves the server's identity:

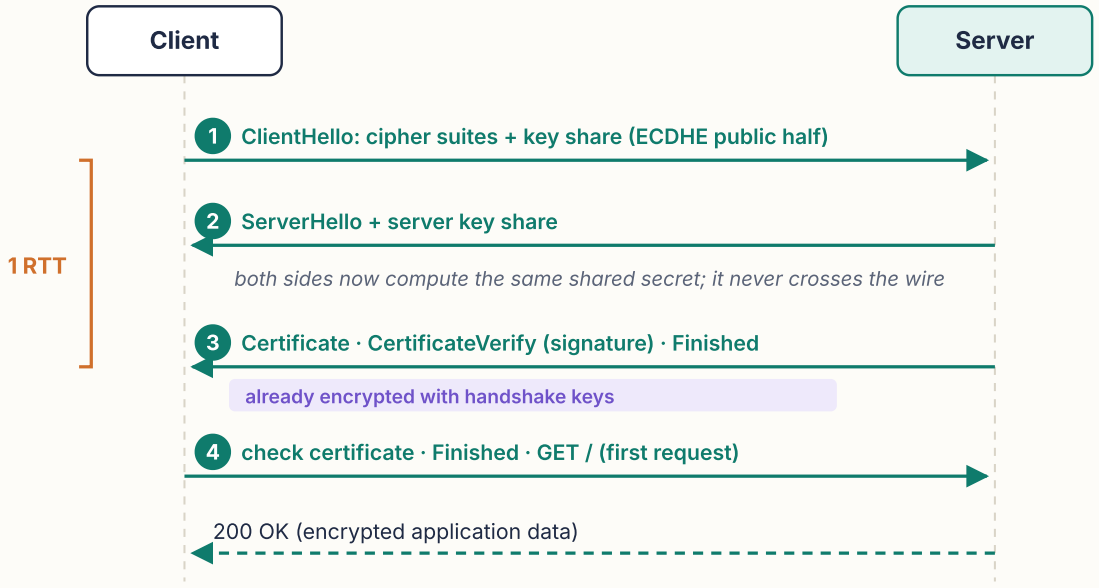


Figure 1. The TLS 1.3 handshake. The client can send its request after one round trip. Everything after the ServerHello is encrypted, including the certificate.

1. **ClientHello.** The client lists the cipher suites it supports and sends a **key share**: its public half of an **ECDHE** (elliptic-curve Diffie-Hellman, ephemeral) exchange, usually on curve X25519.
2. **ServerHello.** The server picks a suite and sends its own key share. From the two shares, each side computes the **same shared secret**. The secret itself is never sent.
3. **Server's encrypted flight.** Already encrypted, the server sends its **Certificate**, a **CertificateVerify** (a signature over the handshake made with its private key, proving it owns the certificate) and **Finished** (a MAC over the whole handshake, so nothing was tampered with).

4. **Client finishes.** The client checks the certificate and signature, sends its own Finished and, in the same flight, its first HTTP request.

3 Why TLS 1.3 is faster and safer than 1.2

TLS 1.2 needed **two round trips** before the first request, and still allowed old, weak options (RSA key exchange, CBC modes, RC4, SHA-1). TLS 1.3 needs **one**, removed the legacy algorithms (only five cipher suites remain, all AEAD, such as `TLS_AES_128_GCM_SHA256`), and made forward secrecy mandatory.



Figure 2. Time to the first response on a 100 ms round-trip link, counting the TCP handshake. TLS 1.3 saves 100 ms on every new connection; 0-RTT resumption saves another 100 ms.

Expensive once, cheap after. Public-key crypto (key exchange and signature) runs only during the handshake. Then both sides derive **session keys** from the shared secret and switch to fast **symmetric** encryption such as AES-GCM or ChaCha20-Poly1305, which modern CPUs do at gigabytes per second.

Resumption. After a handshake the server can hand the client a session ticket. Next time, the client offers it as a **pre-shared key (PSK)** and the server skips sending and checking the certificate. TLS 1.3 also allows **0-RTT**: the client sends *early data* (say, a GET) in its very first message. The catch: an attacker who copies that first message can **replay** it, and the server cannot tell. So accept 0-RTT only for **idempotent** requests such as GETs, never for "pay €50".

4 Forward secrecy

Suppose an attacker records all your encrypted traffic today and steals the server's private key next year. With the old **RSA key exchange**, the client encrypted the session secret with the server's long-term public key, so the stolen key unlocks every recording. With **ECDHE**, each session uses fresh **ephemeral** keys that are thrown away afterwards; the long-term private key only *signs* the handshake. Stealing it later decrypts nothing from the past. That property is **forward secrecy**.



Figure 3. With RSA key exchange one stolen key exposes years of recorded traffic. With ECDHE, a stolen key lets an attacker impersonate the server from then on, but not read the past.

5 Where TLS ends in your architecture

Most often TLS **terminates at the load balancer**. It holds the public certificate, decrypts, and can route by path, header or cookie, add headers like `X-Forwarded-Proto`, and run a WAF. It then **re-encrypts** to the backends over a second TLS connection, so traffic stays encrypted inside your network too, as zero-trust and compliance rules (PCI DSS, HIPAA) often require. In **passthrough** mode the load balancer forwards raw encrypted bytes (layer 4) and the backends terminate TLS themselves: true end-to-end encryption, but the load balancer cannot see HTTP, so no path routing.

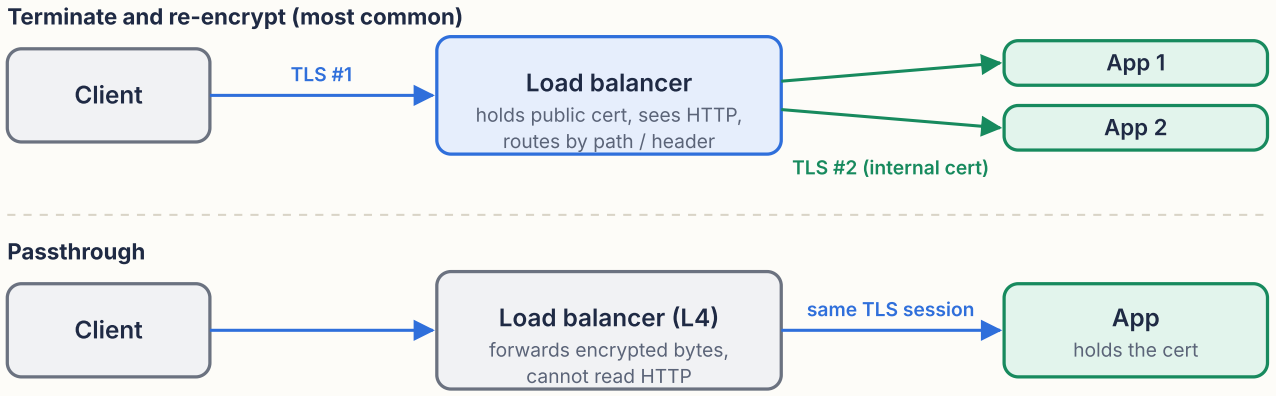


Figure 4. Top: two separate TLS connections, with the load balancer able to read and route HTTP. Bottom: one TLS session from client to app; the load balancer only sees bytes.

WORKED EXAMPLE

A mobile API: 100 ms round trip, users open a new connection on every app launch. TCP + TLS 1.2 + request = 4 round trips ≈ **400 ms** to the first byte. Moving to TLS 1.3: 3 round trips ≈ **300 ms**. Returning users with resumption skip certificate checks; enabling 0-RTT for the idempotent `GET /feed` brings it to ≈ **200 ms**. `POST /orders` still waits for the full handshake. (HTTP/3 folds the TCP step in too: lesson SD-N-023.)

COMMON MISTAKES

- Accepting 0-RTT early data for POSTs or payments: it can be replayed.
- Leaving TLS 1.0 / 1.1 or RSA key exchange enabled "for old clients": it removes forward secrecy for everyone who negotiates down.
- Terminating at the load balancer and sending plain HTTP inside, then assuming the internal network is safe.

6 Key terms

TLS: encrypts and authenticates a connection

CertificateVerify: signature proving the server holds the private key

Forward secrecy: a stolen long-term key can't decrypt past traffic

ECDHE: ephemeral elliptic-curve Diffie-Hellman key exchange

Finished: MAC over the handshake transcript

PSK / resumption: reuse a previous session, skip cert work

Key share: one side's public half of ECDHE

AES-GCM: fast symmetric AEAD cipher for bulk data

0-RTT: data in the first message; replayable

7 Interview questions

Q1. Walk through a TLS 1.3 handshake. BASIC

Client sends ClientHello with cipher suites and an ECDHE key share. Server replies with ServerHello and its key share; both derive the same secret. The server sends, encrypted, its certificate, a CertificateVerify signature and Finished. The client verifies, sends Finished and its first request: one round trip.

Q2. What is forward secrecy? BASIC

Each session uses ephemeral ECDHE keys that are deleted afterwards, and the long-term key only signs. So stealing the server's private key later cannot decrypt traffic recorded in the past. TLS 1.3 makes it mandatory.

Q3. Why use symmetric encryption after the handshake? BASIC

Public-key operations cost around a millisecond of CPU each; AES-GCM with hardware support encrypts several GB/s per core. So the handshake pays for asymmetric crypto once, then bulk data uses symmetric session keys.

Q4. Your app has 100 ms RTT and users complain about slow first loads. What does TLS 1.3 buy you? SCENARIO

One round trip less per new connection: about 400 ms to 300 ms counting TCP. Add session resumption, and 0-RTT for idempotent GETs, to reach about 200 ms. HTTP/3 would also merge the TCP handshake.

Q5. Security asks for encryption inside the data centre, but you route by URL path at the load balancer. What do you do? SCENARIO

Terminate TLS at the load balancer (it needs HTTP to route), then re-encrypt to the backends with internal certificates, or mTLS through a service mesh. Passthrough would block path routing.

Q6. Should you enable 0-RTT for your whole API? SCENARIO

No. Early data can be replayed by anyone who captured it, and the server can't detect it on its own. Allow it only for idempotent, safe requests (GETs) and reject or delay early data for POST, PUT and payments.

Q7. If TLS 1.3 encrypts the certificate, can a network observer still see which site you visit? TRICKY

Usually yes. The destination IP is visible, and the SNI hostname in the ClientHello is plaintext unless Encrypted Client Hello (ECH) is used. TLS 1.3 hides the certificate, not the destination.

Q8. An attacker steals your TLS private key today. What can they do with TLS 1.3? TRICKY

Impersonate your server to clients going forward (until the certificate is revoked and replaced), but not decrypt previously recorded sessions, thanks to ephemeral ECDHE. Rotate the key, revoke the certificate, and check for misuse.

Next: SD-N-022 · Certificates and PKI: why your browser trusts a site it has never met backendroadmap.com

Certificates & PKI

chain of trust, SNI, Let's Encrypt and mTLS

Watch the 4-min video

In one line: a certificate binds a name to a public key; browsers trust it by checking signatures up a chain to a root in their trust store, and short-lived certificates with automated renewal (ACME) end surprise expiry outages.

1 What a certificate is

Your HTTPS site works for 90 days. Then one morning every browser shows a red warning: the certificate expired and nobody renewed it. It is one of the most common outages there is.

A certificate is an **X.509** document that **binds a name to a public key**. It contains:

Field	What it holds
Subject Alternative Names (SAN)	The domain names it is valid for, e.g. `shop.example.com` or a wildcard `*.example.com` (one level only)
Public key	The key matching the server's private key, which never leaves the server
Issuer	The certificate authority (CA) that signed it
Validity	`notBefore` and `notAfter` dates
Signature	The issuer's digital signature over all of the above, so no field can be changed

2 The chain of trust

Who signs it? A **certificate authority (CA)**. Your server's **leaf** certificate is signed by an **intermediate CA**. The intermediate is signed by a **root CA**. Root certificates come pre-installed in the operating system and browser: the **trust store**. Roots are kept offline and only sign intermediates, so a compromised intermediate can be revoked without replacing roots on billions of devices.

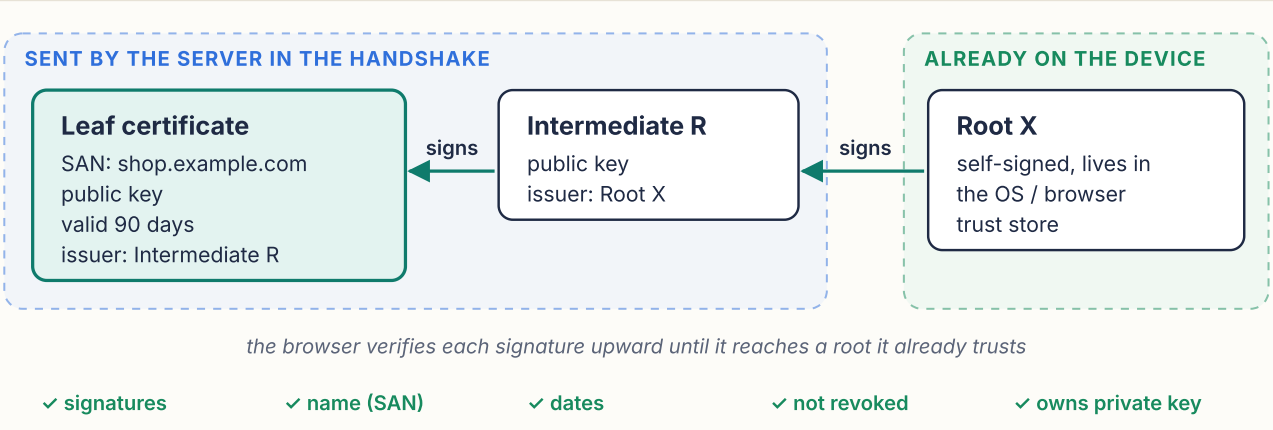


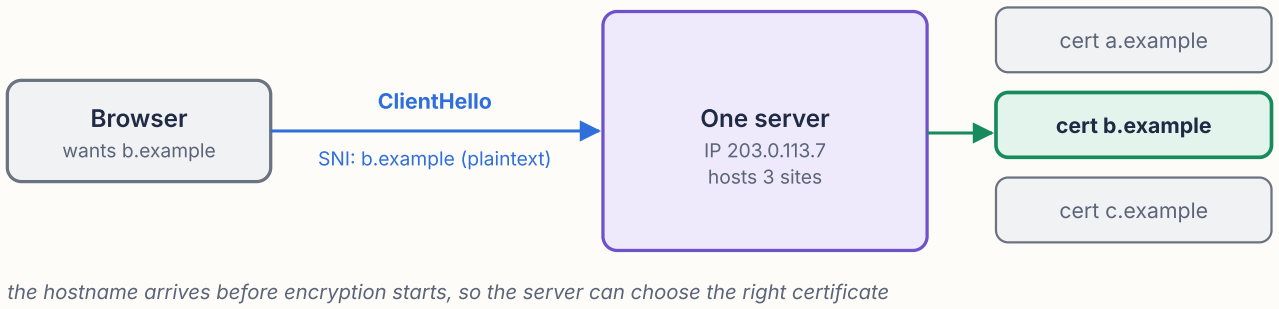
Figure 1. The server sends its leaf plus the intermediate. The root is never sent: the client must already have it. Every check must pass, or the browser shows the warning.

How a browser decides to trust a certificate:

1. Verify each signature up the chain until it reaches a root in its trust store.
2. Check that the site's hostname matches a name in the SAN.
3. Check that today is between the validity dates.
4. Check that the certificate has not been **revoked** (via CRLs, OCSP or the browser's own revocation lists).
5. In the handshake, the server **signs** with the matching private key (CertificateVerify), proving it owns the certificate and did not just copy it.

3 SNI: many sites, one IP address

One server or load balancer often hosts many domains on a single IP address. Which certificate should it send? **Server Name Indication (SNI)**: the client puts the hostname in its ClientHello, so the server picks the right certificate *before* encryption starts. Because it comes first, SNI is sent in plaintext (Encrypted Client Hello, ECH, is the newer fix).



the hostname arrives before encryption starts, so the server can choose the right certificate

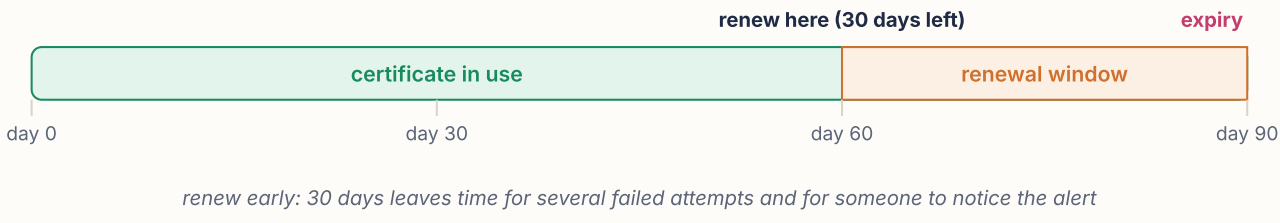
Figure 2. SNI lets one IP serve many HTTPS sites, each with its own certificate. Without SNI the server would have to guess, and usually sends a default certificate with the wrong name.

4 Rotation: Let's Encrypt, ACME and cert-manager

Let's Encrypt issues free certificates that last **90 days**, using the **ACME** protocol. To get one you prove you control the domain with a **challenge**:

- **HTTP-01:** serve a token file at `http://your-domain/.well-known/acme-challenge/…` on port 80.
- **DNS-01:** create a TXT record at `_acme-challenge.your-domain`. Needed for wildcard certificates, and works for servers not reachable from the internet.

In Kubernetes, **cert-manager** automates it: it requests certificates, stores them as Kubernetes **Secrets** that ingresses use, and renews them well before they expire (by default when a third of the lifetime is left). Short lifetimes plus automation mean no surprise expiries, and a leaked key is useful for weeks, not years. The industry is pushing lifetimes even shorter, so manual renewal is no longer realistic.



renew early: 30 days leaves time for several failed attempts and for someone to notice the alert

Figure 3. A 90-day certificate renewed at day 60. Alert if any certificate has fewer than ~14 days left: that means automation has been failing for two weeks.

5 Mutual TLS (mTLS)

Normally only the server shows a certificate. With **mutual TLS**, the client presents one too, and the server checks it against a trusted CA. This is ideal for **service-to-service authentication**: the payments service accepts calls only from services holding a valid internal certificate, issued by your own private CA. Service meshes like **Istio** and **Linkerd** run a sidecar proxy next to each service that issues, presents and rotates these certificates automatically (often every 24 hours), so application code does not change.

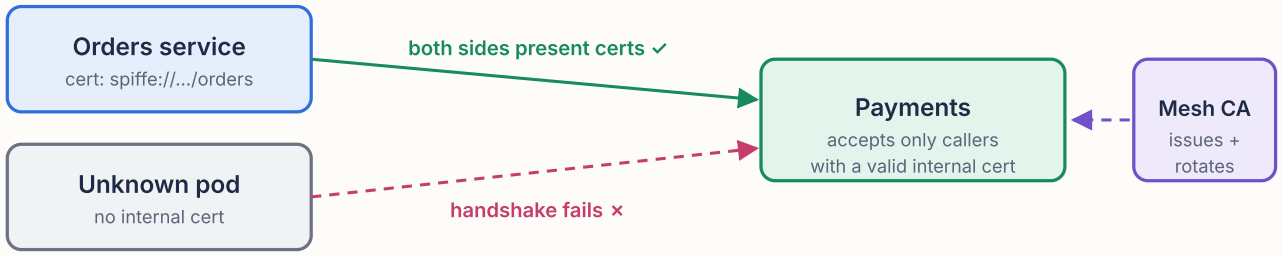


Figure 4. With mTLS the payments service knows which service is calling, from its certificate, before any request is read. A pod without a valid certificate cannot even finish the handshake.

WATCH OUT: CERTIFICATE PINNING

Some mobile apps **pin**: they hard-code which certificate or public key they will trust. If you rotate to a new key the app doesn't know, every installed copy stops connecting until users update. If you pin, pin a backup key too and plan rotation, or pin the CA's intermediate instead of your leaf.

WORKED EXAMPLE

You run 40 domains. With manual 1-year certificates that is about 40 renewals a year, each one a chance to forget. With ACME and 90-day certificates renewed at 30 days left, that is about 160 renewals a year, all automatic, each with a 30-day window to retry. Monitor `notAfter` for every endpoint, and page when any is under 14 days.

COMMON MISTAKES

- Renewing by hand and relying on someone's calendar.
- Serving the leaf without the intermediate: works in browsers that cached it, fails in curl, mobile apps and containers.
- Relying on the certificate's CN instead of SAN, or expecting `*.example.com` to cover `a.b.example.com`.
- Pinning a leaf key in a mobile app with no backup pin.

6 Key terms

X.509: the standard certificate format

CA: certificate authority, signs certificates

Leaf / intermediate / root: the three links of a chain

Trust store: roots pre-installed in OS and browser

SAN: the names a certificate is valid for

SNI: hostname in the ClientHello

ACME: protocol for automated issuance and renewal

cert-manager: automates ACME in Kubernetes

mTLS: both client and server present certificates

7 Interview questions

Q1. How does a browser trust a certificate? BASIC

It verifies the chain of signatures from the leaf, through intermediates, to a root CA in its trust store. It checks the hostname against the SAN, the validity dates and revocation. And in the handshake the server signs with the matching private key.

Q2. What is SNI and why is it needed? BASIC

Server Name Indication: the client sends the hostname in the ClientHello, so a server hosting many domains on one IP can present the matching certificate before encryption starts.

Q3. What is mTLS used for? BASIC

Mutual authentication, mostly between services: both sides present certificates, so the server knows exactly which service is calling. Meshes like Istio and Linkerd issue and rotate them automatically.

Q4. Your site went down because a certificate expired. How do you prevent it from happening again? SCENARIO

Automate with ACME (certbot, cert-manager, or managed certificates on the load balancer), renewing at 30 days left. Monitor notAfter for every endpoint and alert below 14 days, so failing automation is caught with two weeks to spare.

Q5. You need a certificate for *.internal.example.com on servers not reachable from the internet. Which ACME challenge? SCENARIO

DNS-01: create a TXT record at _acme-challenge.internal.example.com. HTTP-01 needs port 80 reachable from the internet and can't issue wildcards.

Q6. curl says 'unable to verify the first certificate' but the browser shows the padlock. Why? SCENARIO

The server isn't sending its intermediate certificate. The browser had cached it or fetched it on its own; curl and most libraries don't. Configure the server to send the full chain (leaf + intermediate).

Q7. Why do CAs use intermediates instead of signing leaves with the root? TRICKY

The root key stays offline in an HSM, so it's hard to steal. If an intermediate is compromised it can be revoked and replaced, without updating trust stores on billions of devices, which takes years.

Q8. Why are short certificate lifetimes considered more secure? TRICKY

Revocation checking is unreliable (browsers often soft-fail), so a stolen key stays usable until expiry. With 90-day or shorter certificates, the damage window is weeks, not years. It also forces automation, which removes human error.

Next: SD-N-023 · HTTP/1.1 vs HTTP/2 vs HTTP/3, and head-of-line blocking from scratch backendroadmap.com

HTTP/1.1 vs HTTP/2 vs HTTP/3

how requests share the network

Watch the 4-min video

In one line: HTTP/1.1 sends one request at a time per connection; HTTP/2 multiplexes many streams over one TCP connection but still stalls on a lost packet; HTTP/3 runs on QUIC over UDP, so streams are independent, setup is faster and connections survive network changes.

A modern web page can fetch a hundred files: scripts, styles, images and API calls. How those requests share the network decides how fast the page feels. The meaning of HTTP (methods, status codes, headers) is the same in all three versions; what changes is how messages travel on the wire.

1 HTTP/1.1: one request at a time per connection

HTTP/1.1 is **text-based**. **Keep-alive** lets one TCP connection carry many requests, but only one at a time: the next request waits until the current response has fully arrived. A slow response blocks everything queued behind it. That is **head-of-line (HOL) blocking** at the HTTP level. (Pipelining tried to fix this, but responses still had to come back in order, and browsers never enabled it.)

So browsers open about **six connections per host**, each with its own TCP and TLS handshake, and still send the same bulky headers (cookies, user agent) on every request, often 500 bytes to 2 KB each.

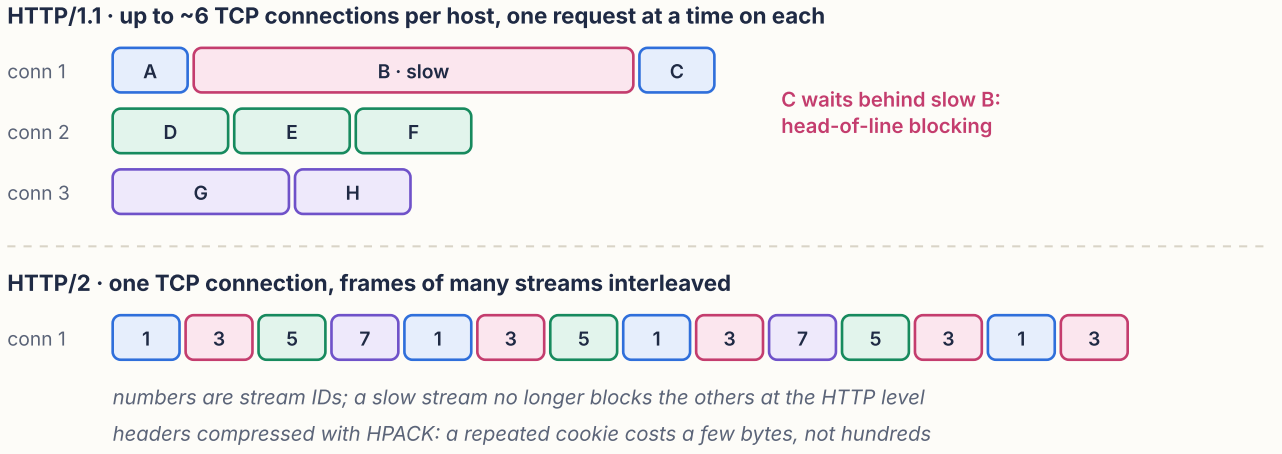


Figure 1. Top: six lanes at most, and a slow response blocks its lane. Bottom: HTTP/2 splits each request and response into frames and interleaves them on a single connection.

2 HTTP/2: binary frames and multiplexing

HTTP/2 switched to **binary framing**. Each request/response pair becomes a **stream** with an ID, cut into frames. Frames of many streams are **interleaved** on one TCP connection: that is **multiplexing**. No more waiting in line at the HTTP level, and no need for six connections (and six handshakes). Servers typically allow 100 or more concurrent streams per connection.

- **HPACK** header compression: both sides keep a table of headers already sent, so a repeated header becomes a small index of a few bytes.
- **Server push** let a server send files before they were asked for. It rarely helped and is now mostly abandoned (Chrome removed it in 2022).

- Browsers only use HTTP/2 over TLS; the version is agreed in the TLS handshake via **ALPN** (`h2`).

gRPC is built on HTTP/2. Every call is a stream, so one connection can carry thousands of concurrent calls, including long-lived **streaming calls in both directions**.

3 The catch: TCP head-of-line blocking

HTTP/2 still runs on **TCP**, and TCP delivers bytes strictly in order. If one packet is lost, every byte after it waits in the receiver's kernel buffer until the retransmission arrives, even bytes that belong to other streams. Lose a single packet and **every stream on that connection stalls**. On a lossy mobile network, one HTTP/2 connection can even be slower than six HTTP/1.1 connections, where a loss only stalls one of the six.

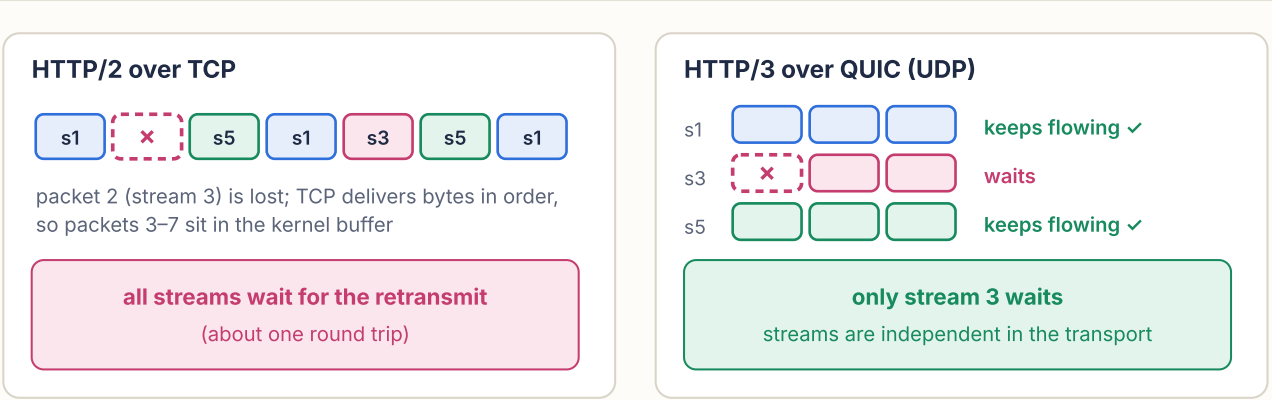


Figure 2. HTTP/2 moved head-of-line blocking from HTTP down to TCP. QUIC tracks ordering per stream, so a lost packet only delays the stream it belongs to.

4 HTTP/3: QUIC over UDP

HTTP/3 runs on **QUIC**, a transport protocol built on top of UDP. It keeps HTTP/2's ideas (streams, header compression, now called QPACK) and fixes the transport:

- **Independent streams.** QUIC handles ordering and retransmission per stream, so a lost packet only stalls its own stream.
- **Faster setup.** TLS 1.3 is built into QUIC, so the transport and encryption handshakes happen together in **one round trip**, or **zero** on resumption (0-RTT, idempotent requests only).
- **Connection migration.** A connection is identified by a **connection ID**, not by IP address and port. When your phone moves from Wi-Fi to 4G, the connection migrates instead of starting over.

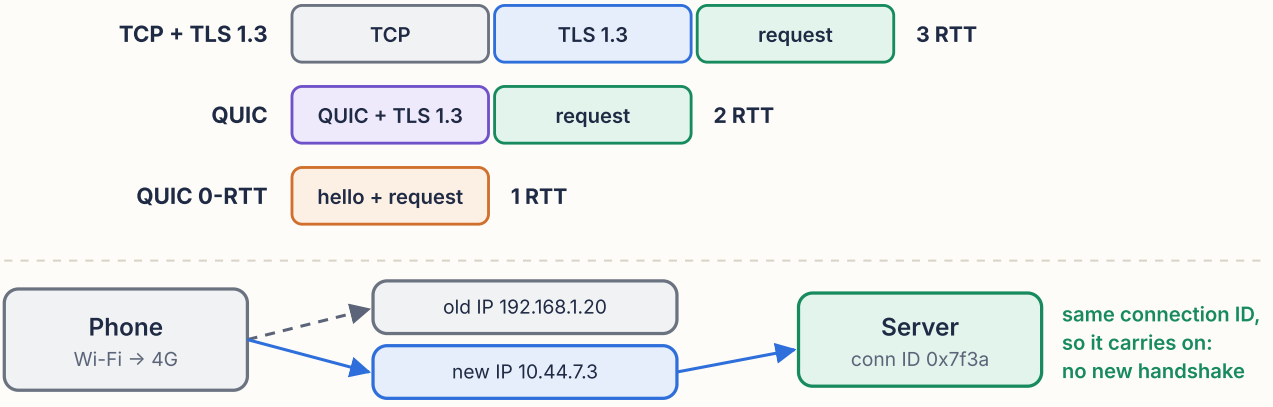


Figure 3. Top: round trips before the first response byte. QUIC merges the TCP and TLS handshakes. Bottom: the client's IP changes, but the connection ID doesn't, so the server keeps the same connection.

Why build on UDP instead of fixing TCP? TCP lives inside operating-system kernels, and countless routers, firewalls and middleboxes expect it to behave in a fixed way (it has **ossified**). Changing it would take decades. UDP passes through almost everywhere, and QUIC runs in **user space**, inside the browser or server, so it evolves as fast as they update. Some networks still block UDP, so browsers learn about HTTP/3 from an `Alt-Svc` header (or DNS) and fall back to HTTP/2 over TCP if QUIC fails.

	HTTP/1.1	HTTP/2	HTTP/3
Format	Text	Binary frames	Binary frames
Transport	TCP	TCP	QUIC over UDP
Requests per connection	One at a time	Many streams, multiplexed	Many independent streams
Head-of-line blocking	HTTP level	TCP level (packet loss)	Only within one stream
Headers	Repeated in full	HPACK	QPACK
Setup (with TLS 1.3)	2 RTT	2 RTT	1 RTT, or 0 on resume
Network change	Reconnect	Reconnect	Migrates (connection ID)

WORKED EXAMPLE

A page loads 60 files over a 100 ms round-trip mobile link with 2% packet loss. HTTP/1.1: 6 connections × (TCP + TLS) handshakes, and 60 requests queued about 10 deep per lane, so the page needs at least ~12 round trips (≈ 1.2 s) even with fast servers. HTTP/2: one handshake, all 60 requests sent at once, so far fewer round trips; but with 2% loss, each lost packet freezes all 60 streams for about a round trip. HTTP/3: one combined handshake (100 ms less), and a loss only delays the one file it belongs to.

COMMON MISTAKES

- Saying HTTP/2 removed head-of-line blocking completely: it moved it to TCP.
- Keeping HTTP/1.1 tricks on HTTP/2 (domain sharding, sprite sheets, giant bundles): extra domains mean extra connections and lose multiplexing.
- Assuming HTTP/3 always works: corporate firewalls may block UDP 443; keep the HTTP/2 fallback.
- Thinking a new version changes your API: methods, status codes and headers mean the same thing.

5 Key terms

Keep-alive: reuse one TCP connection for many requests

Multiplexing: interleaving frames of many streams

QUIC: UDP-based transport with built-in TLS 1.3

HOL blocking: one slow or lost item delays everything behind it

HPACK / QPACK: header compression for HTTP/2 / HTTP/3

Connection ID: identifies a QUIC connection, not IP + port

Stream: one request/response inside a connection

ALPN: TLS extension that picks h2 or http/1.1

Ossification: middleboxes freezing a protocol's behaviour

6 Interview questions

Q1. What problem does HTTP/2 solve? BASIC

HTTP/1.1 handles one request at a time per connection, so browsers open ~6 connections and a slow response blocks its lane. HTTP/2 multiplexes many streams over one connection and compresses headers with HPACK.

Q2. Why was HTTP/3 built on UDP? BASIC

To escape TCP's head-of-line blocking and its ossified, kernel-bound design. QUIC on UDP runs in user space, so it can offer independent streams, a combined 1-RTT handshake with TLS 1.3, and connection migration, and still evolve quickly.

Q3. What is head-of-line blocking? BASIC

When one slow or lost item delays everything queued behind it. In HTTP/1.1 it's a slow response on a connection; in HTTP/2 it's a lost TCP packet stalling all streams; HTTP/3 limits it to one stream.

Q4. Your mobile users on flaky networks see slow loads even after moving to HTTP/2. Why, and what helps? SCENARIO

Packet loss: every lost TCP packet stalls all streams on the single connection. Enable HTTP/3 at the CDN or load balancer (with HTTP/2 fallback); QUIC limits stalls to one stream and survives Wi-Fi/4G switches without reconnecting.

Q5. Why does gRPC use HTTP/2? SCENARIO

Streams: one connection carries thousands of concurrent calls, binary framing suits protobuf, and long-lived streams allow client, server and bidirectional streaming calls. It also gets HPACK and per-stream flow control.

Q6. You enabled HTTP/3 but some office users still use HTTP/2. Is something broken? SCENARIO

Probably not. Their firewall likely blocks UDP 443, so the browser falls back to TCP. Browsers discover HTTP/3 via Alt-Svc or DNS HTTPS records and use it only when QUIC actually connects.

Q7. Can HTTP/2 be slower than HTTP/1.1? TRICKY

Yes, on lossy links. With six HTTP/1.1 connections, a lost packet stalls one of them; with one HTTP/2 connection, it stalls every stream, and one connection also gets a single share of TCP congestion control.

Q8. Why can't we just fix head-of-line blocking in TCP? TRICKY

TCP is implemented in OS kernels and inspected by routers, firewalls and NATs that drop anything unexpected. Deploying a change takes many years. QUIC encrypts nearly all its headers, so middleboxes can't freeze its behaviour again.

Next: SD-N-024 · WebSockets backendroadmap.com

WebSockets

upgrade handshake, heartbeats and scaling

Watch the 4-min video

In one line: a WebSocket starts as an HTTP request, upgrades with `101 Switching Protocols`, and becomes one long-lived, full-duplex connection; the hard part is that every connection is state pinned to one server.

1 Why WebSockets

A chat app needs messages to appear instantly, in both directions. With plain HTTP the client must keep asking "anything new?", and the server can only answer, never start a conversation. A **WebSocket** fixes that: one long-lived connection where **either side can send at any time**. Use `wss://` (WebSocket over TLS) in production.

2 The upgrade handshake

A WebSocket starts as a normal HTTP/1.1 request that asks to switch protocols. After one exchange, the connection belongs to WebSocket:

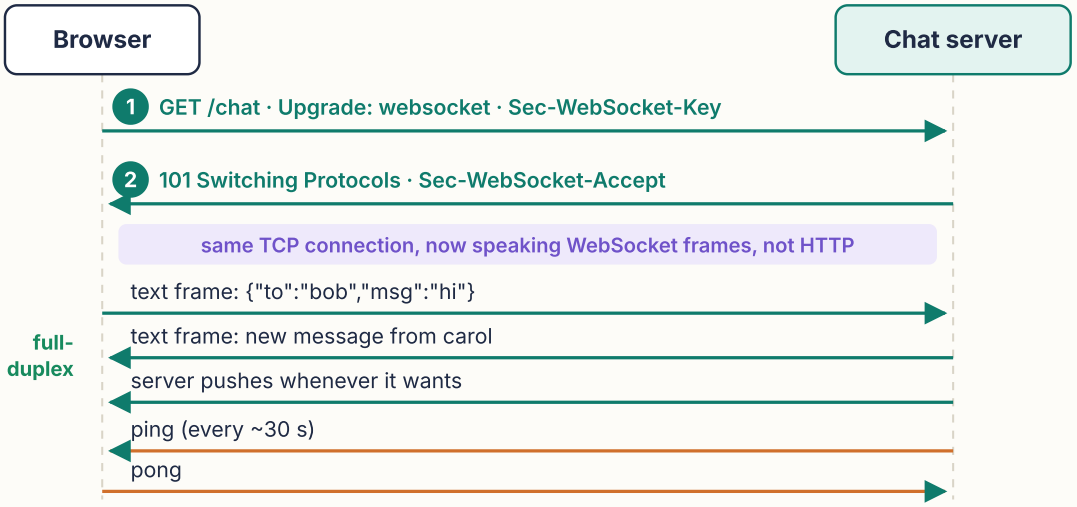


Figure 1. After the 101 response, both sides send frames whenever they like. Ping and pong are small control frames that keep the connection alive and prove the peer is still there.

On the wire, the client sends a random key and the server answers with a value derived from it:

```
GET /chat HTTP/1.1
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
```

The server proves it really speaks WebSocket by returning an **accept value derived from the key** (Base64 of SHA-1 of the key plus a fixed GUID). From then on, the **same TCP connection** carries WebSocket frames, not HTTP. Any other status (400, 401, 404, 200) means the upgrade failed and is debugged like any HTTP error.

3 Full-duplex frames and heartbeats

The connection is **full-duplex**: client and server both send whenever they want, without waiting for each other. Messages travel as **frames**, text (UTF-8) or binary, with just **2 to 14 bytes** of header each (client frames add a 4-byte mask), instead of hundreds of bytes of HTTP headers.

Connections live for hours, so you need **heartbeats**. **Ping** and **pong** control frames every ~30 seconds:

- prove the other side is still there (a phone that lost signal sends no close frame, so the socket looks open forever);
- stop load balancers and NAT gateways from closing a connection that looks idle (AWS ALB and nginx's `proxy_read_timeout` both default to 60 s);
- if pongs stop (say, two missed in a row), **close the connection and clean up**: remove it from the registry and free its memory. (Browsers answer pings automatically but JavaScript can't send them, so the server sends the pings.)

4 Example: a chat server pushing messages

Alice and Bob each hold a WebSocket to a **gateway** server, but not necessarily the same one. When Alice sends a message, her gateway stores it, looks up which gateway holds Bob's connection, and publishes it there; that gateway pushes it down Bob's socket. No polling at all.

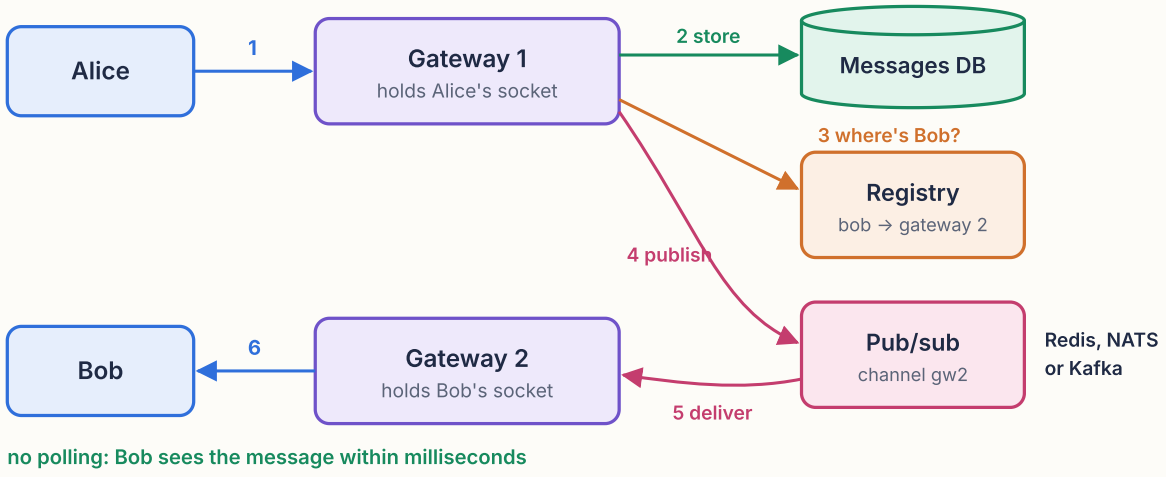


Figure 2. Gateways only hold sockets. A registry says who is connected where, and pub/sub carries messages between gateways. Storing first means Bob can still fetch the message if he is offline.

5 Scaling: every connection is state

An HTTP request lasts milliseconds; a WebSocket lasts hours, **pinned to one server**. That changes how you scale:

- **Load balancer** must support the upgrade (nginx needs `proxy_http_version 1.1` and the Upgrade/Connection headers set explicitly) and allow **long idle timeouts**, above the heartbeat interval.
- **Per-server limits** are memory and file descriptors, not CPU: each socket needs a file descriptor and some kilobytes of buffers. A tuned server (raised `ulimit -n`, async I/O) holds **hundreds of thousands** of mostly idle connections.

- **Scale out** with many gateway servers, a **registry** of who is connected where (e.g. Redis with a TTL refreshed by heartbeats), and **pub/sub** to route messages between servers.

WORKED EXAMPLE

Target: **2 million** concurrent users. At ~20 KB per idle connection, 2M × 20 KB ≈ **40 GB** of memory across the fleet. If one gateway safely holds 100,000 connections, you need 20 gateways; run **30** so that losing a server or draining one for a deploy doesn't overload the others. Heartbeats every 30 s mean 2M / 30 ≈ **67,000 pings per second** fleet-wide, tiny per server. Keep the load balancer idle timeout above 30 s, for example 120 s.

6 Plan for reconnection

When a server restarts, all its clients drop at once. If they all reconnect immediately, they **stampede** the next server (a thundering herd). So clients retry with **exponential backoff plus random jitter** (e.g. wait a random time up to 1 s, then up to 2 s, 4 s ... capped at 30 s), and **resume from the last message ID they saw** so nothing is lost. During deploys, **drain** servers gradually: stop sending new connections to a server, then close its sockets in batches with a "going away" close (code 1001), instead of killing them all at once.

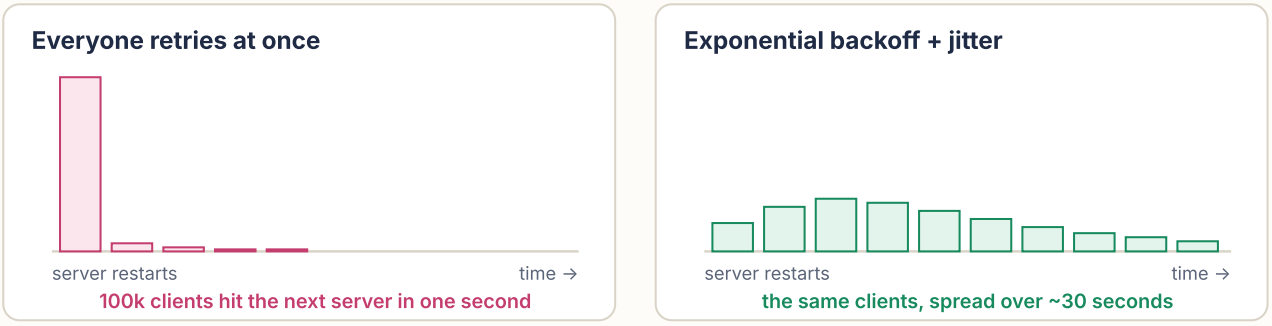


Figure 3. Left: a synchronized retry spike can knock over the server that was supposed to take the load. Right: random jitter spreads the same reconnects out.

COMMON MISTAKES

- No heartbeats: dead connections pile up, and idle-timeout proxies cut live ones at 60 s.
- Keeping the "who is connected" map only in one process's memory: messages for users on other servers vanish.
- Reconnecting instantly in a tight loop, with no backoff or jitter.
- Using WebSockets for one-way updates where server-sent events or plain HTTP would be simpler (next lesson).

7 Key terms

Upgrade: HTTP mechanism to switch protocols

Frame: unit of a WebSocket message (text, binary, control)

Connection registry: maps user → gateway

101 Switching Protocols: server agrees to switch

Ping / pong: control frames used as heartbeats

Backoff + jitter: growing, randomized retry delays

Full-duplex: both sides send at any time

Gateway: server that holds client sockets

Draining: closing connections gradually before shutdown

8 Interview questions

Q1. How does a WebSocket connection start? BASIC

As an HTTP/1.1 GET with Upgrade: websocket, Connection: Upgrade and a random Sec-WebSocket-Key. The server answers 101 Switching Protocols with a Sec-WebSocket-Accept derived from the key. Then the same TCP connection carries WebSocket frames.

Q2. Why do WebSockets need heartbeats? BASIC

To detect dead peers (a dropped phone sends no close frame) and to keep load balancers and NATs, which often cut idle connections after ~60 s, from closing healthy ones. Ping every ~30 s; close after missed pongs.

Q3. How do you scale to millions of WebSocket connections? SCENARIO

Many gateway servers (~100k connections each) behind a load balancer that supports upgrades and long idle timeouts; a registry of user → gateway; pub/sub between gateways; heartbeats; and clients that reconnect with backoff and jitter. For 2M users: about 20 gateways, plus spare capacity.

Q4. Users on gateway 1 don't receive messages sent by users on gateway 2. What's wrong? SCENARIO

Each gateway only knows its own sockets. Add pub/sub (Redis, NATS, Kafka) and a registry, so the sender's gateway can publish to the gateway that holds the recipient.

Q5. Every deploy causes a spike of errors and a CPU storm. Why, and how do you fix it? SCENARIO

Killing a server drops all its sockets and every client reconnects at once. Drain gradually (stop new connections, close in batches with code 1001), and make clients use exponential backoff with jitter and resume from their last message ID.

Q6. Connections drop exactly every 60 seconds when nobody is chatting. Why? TRICKY

A proxy or load balancer idle timeout (e.g. nginx proxy_read_timeout or ALB idle timeout, both 60 s by default). Send heartbeats more often than the timeout, e.g. every 30 s, and raise the timeout.

Q7. Can you use sticky sessions instead of pub/sub? TRICKY

Stickiness only decides where one client lands. Alice and Bob can still land on different servers, so messages still need to cross servers. Use a registry and pub/sub, or route everyone in a room to the same server by consistent hashing.

Next: SD-N-025 · SSE vs long polling vs WebSockets backendroadmap.com

SSE vs Long Polling vs WebSockets

choosing how the server pushes updates

Watch the 4-min video

> **In one line:** short polling asks on a timer, long polling holds each request until there is news, server-sent events keep one HTTP response open and stream events down it, and WebSockets give a two-way channel. Pick the simplest one that fits the direction of your data.

When you chat with an AI assistant, the answer appears word by word. That is usually **server-sent events (SSE)**: one HTTP response that stays open while the server keeps pushing tokens. Here is how it compares with polling, long polling and WebSockets.

1 Short polling and long polling

Short polling is the simplest: the client asks every few seconds, “anything new?”. Most answers are empty, which wastes requests, and an update can wait up to a full interval before anyone sees it (on average half of it).

Long polling is smarter. The client sends a request and the server **holds it open** until there is news, or until a timeout (say 30 seconds), then answers. As soon as a response arrives, the client sends the next request. Updates arrive almost instantly, and it works through any HTTP infrastructure, which makes it a great **fallback**. The cost: every message needs a full request and response, with headers, and there is a small gap while the next request is on its way.

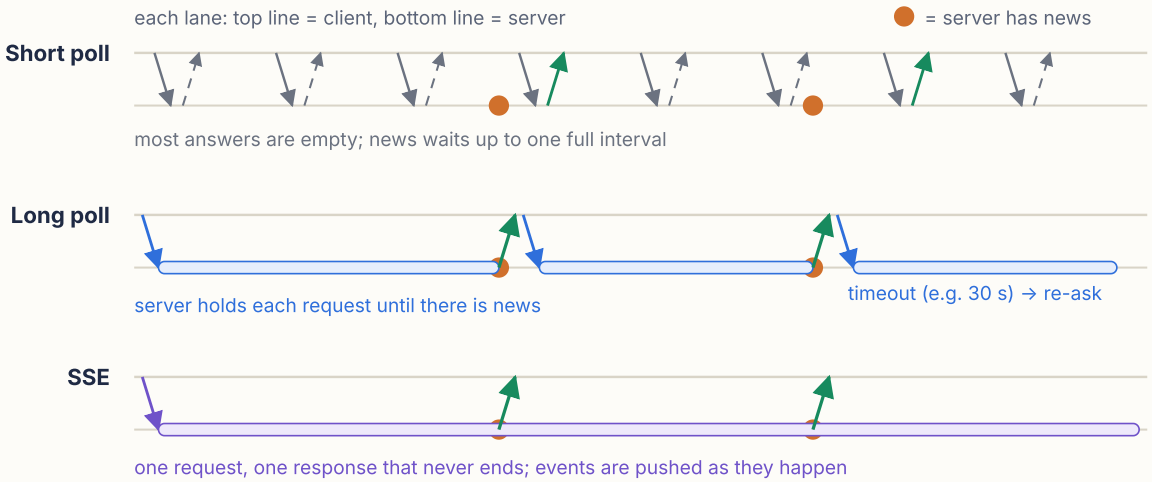


Figure 1. Three ways to get the two news events (orange) to the client. Short polling finds them late and asks many empty questions; long polling and SSE deliver them almost at once.

2 Server-sent events

With **SSE** the browser opens one request with the `EventSource` API. The server replies with `Content-Type: text/event-stream` and **never closes** the response. It writes events as they happen, as plain text: lines starting with `data:`, with a **blank line** ending each event.

```
id: 42
event: score
data: {"home": 2, "away": 0}

: keep-alive (a comment line, ignored)
```

Each event can carry an `id`. If the connection drops, the browser **reconnects automatically** (after a few seconds, or the `retry:` value the server set) and sends a `Last-Event-ID` header, so the server can **resume** from where it stopped.

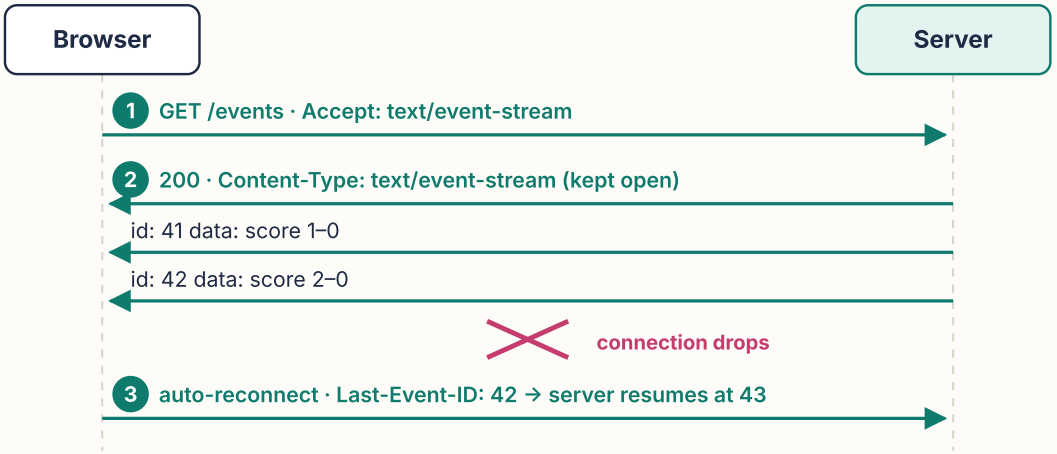


Figure 2. SSE recovery is built in: the browser reconnects by itself and tells the server the last event it saw. With WebSockets you write this logic yourself.

3 Example: streaming LLM tokens from FastAPI

The endpoint returns a `StreamingResponse` with the event-stream media type. An **async generator** yields each token as a `data:` line the moment the model produces it.

```
import json
from fastapi import FastAPI
from fastapi.responses import StreamingResponse
app = FastAPI()

@app.get("/chat/stream")
async def chat_stream(q: str):
    async def events():
        async for token in llm.stream(q):   # your model client
            yield f"data: {json.dumps(token)}\n\n"
        yield "data: [DONE]\n\n"
    return StreamingResponse(events(), media_type="text/event-stream",
                             headers={"Cache-Control": "no-cache", "X-Accel-Buffering": "no"})
```

The user sees the first words in a few hundred milliseconds instead of waiting ten seconds for the full answer. JSON-encoding each token keeps newlines inside a token from breaking the format, and `X-Accel-Buffering: no` stops nginx from buffering the stream.

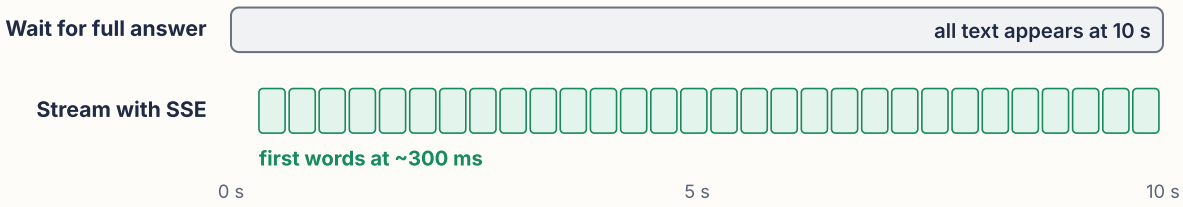


Figure 3. Same model, same total time. Streaming changes when the user starts reading: time to first token, not time to last token, is what feels fast.

4 How they compare

	Short polling	Long polling	SSE	WebSockets
Direction	Client pulls	Client pulls, server waits	Server → client	Both ways
Latency	Up to one interval	Near real time	Real time	Real time
Cost per message	Full request, often empty	Full request	A few bytes	2–14 bytes
Protocol	Plain HTTP	Plain HTTP	Plain HTTP	Upgrade to WebSocket
Reconnect	n/a	Next request	Built in, with Last-Event-ID	You build it
Infrastructure	Anything	Anything (long timeouts)	Anything that doesn't buffer	Upgrade-aware LBs, long timeouts

5 Choosing

- **Choose SSE** when updates flow mainly from server to client: notifications, live scores, dashboards, job progress and AI token streams. It is plain HTTP, so it works with HTTP/2, proxies, CDNs and your normal cookie or header authentication.
- **Choose WebSockets** when the client also sends frequent messages: chat, collaborative editing, multiplayer games.
- **Keep long polling** as a fallback for networks or proxies that break streaming, and short polling for rare, non-urgent checks.

CAVEAT: THE SIX-CONNECTION LIMIT

Over HTTP/1.1, browsers allow only about **six connections per host**, shared by all tabs. Each open EventSource uses one, so six tabs of your dashboard can freeze every other request to that host. Serve SSE over **HTTP/2**, where each stream is just one of ~100 multiplexed streams on a single connection.

WORKED EXAMPLE

A dashboard with 10,000 open users, and about 1 update per minute each. Short polling every 5 s: 10,000 / 5 = **2,000 requests/s**, about 98% of them empty, and updates arrive 2.5 s late on average. SSE: 10,000 idle connections and about 10,000 / 60 ≈ **170 small events/s**, delivered at once. Same features, roughly a tenth of the traffic.

COMMON MISTAKES

- A proxy or gzip layer buffering the stream, so events arrive in one lump at the end.
- Serving SSE over HTTP/1.1 and hitting the six-connection limit across tabs.
- Picking WebSockets for one-way updates and then building reconnection and auth by hand.

6 Key terms

Short polling: ask on a fixed interval

Long polling: server holds the request until news or timeout

SSE: one long HTTP response carrying events

EventSource: browser API for SSE

text/event-stream: SSE content type

Last-Event-ID: header sent on reconnect, to resume

StreamingResponse: FastAPI response fed by a generator

Time to first token: delay before the first words appear

Fallback: simpler transport used when the best one fails

7 Interview questions

Q1. When would you choose SSE over WebSockets? BASIC

When data flows one way, server to client, and you want plain HTTP, automatic reconnects with Last-Event-ID and simple infrastructure: notifications, dashboards, streaming LLM tokens.

Q2. How does long polling work? BASIC

The client sends a request that the server holds open until there is data or a timeout (e.g. 30 s). Then the client immediately sends another. Near real-time, works through any HTTP infrastructure, but costs a full request per message.

Q3. Design token streaming for an AI chat product. SCENARIO

POST the prompt, return a text/event-stream response, and yield each token as a data: line from an async generator; send a final [DONE] event. Disable proxy buffering, serve over HTTP/2, send keep-alive comments every ~15 s, and cancel generation if the client disconnects.

Q4. Users say live scores arrive all at once at the end. What's wrong? SCENARIO

Something buffers the response: nginx proxy_buffering, a gzip middleware, or a CDN. Set X-Accel-Buffering: no (or turn buffering off), don't compress the stream, and set Cache-Control: no-cache.

Q5. Your dashboard freezes when a user opens it in seven tabs. Why? SCENARIO

Over HTTP/1.1 the browser allows about six connections per host; six EventSource streams take them all and the seventh request waits. Serve over HTTP/2, or share one stream across tabs (e.g. a SharedWorker).

Q6. EventSource can't set an Authorization header. How do you authenticate SSE? TRICKY

Use cookies (sent automatically, withCredentials for cross-origin), or a short-lived token in the URL, or read the stream with fetch() instead of EventSource, which allows headers and POST. That is how most LLM APIs are consumed.

Q7. Is long polling obsolete? TRICKY

No. It needs nothing special from proxies or firewalls, so it is a reliable fallback; Socket.IO, for example, can start on long polling and upgrade to WebSockets. It is just less efficient per message than SSE.

Next: SD-N-026 · HTTP streaming, chunked transfer and backpressure backendroadmap.com

HTTP Streaming

chunked transfer, backpressure and gRPC streams

Watch the 4-min video

In one line: instead of building a huge response in memory, send it in pieces as you produce it (chunked transfer in HTTP/1.1, data frames in HTTP/2), read the data lazily, and let TCP backpressure keep memory flat.

1 Why stream, and how it works on the wire

A user clicks **Export** and asks for five million orders as a CSV file. If the server builds the whole file in memory first, it may need gigabytes of RAM. The user stares at a spinner for a minute, or the server simply runs out of memory and crashes. The fix is **streaming**: send the file in pieces while you are still producing it.

In **HTTP/1.1** a server normally sends a `Content-Length` header, so it must know the body size before it sends the first byte. With `Transfer-Encoding: chunked` it doesn't. The body goes out as a series of chunks; each one starts with its size in hexadecimal, and a **zero-length chunk** marks the end.

HTTP/2 and HTTP/3 don't use chunked encoding at all (it is not allowed there). A body is already a series of **DATA frames**, and the last one carries an end-of-stream flag. Same idea, built into the protocol.

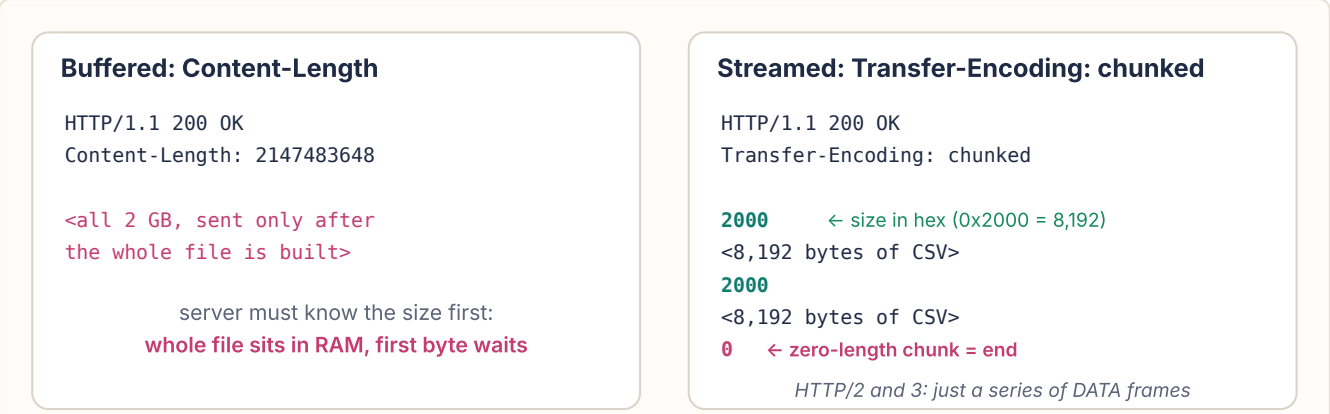


Figure 1. Left: with Content-Length the server must finish the whole body first. Right: chunked encoding sends each piece with its size, and a 0 chunk ends the body.

So the server can start sending before it has finished computing: the **first bytes leave in milliseconds**. **Memory stays flat**, because only one chunk is held at a time. And the client can **process data as it arrives**.

2 Example: a streaming CSV export in FastAPI

Read rows with a **server-side cursor**, in batches of about 1,000, so the database driver never loads the full result. A **generator** turns each batch into CSV lines and yields them. Return a `StreamingResponse` with media type `text/csv` and a `Content-Disposition` header so the browser saves it as a file.

WORKED EXAMPLE

```
async def rows(): async for batch in cursor_batches(size=1000): yield to_csv(batch)
return StreamingResponse(rows(), media_type="text/csv", headers={"Content-Disposition":
"attachment; filename=orders.csv"})
```

Five million rows at about 100 bytes each is roughly **500 MB** of CSV. Buffered, that is 500 MB or more in RAM per export; ten users at once is 5 GB. Streamed, each export holds one batch: 1,000 × 100 bytes ≈ **100 KB**, and the download starts immediately. (SQLAlchemy: `stream_results=True` + `yield_per(1000)` ; psycopg: a named cursor.)

3 Backpressure: when the client reads slowly

What if the client downloads slowly, say on a weak mobile link? **TCP flow control** handles it. The client's receive window fills, then the server's send buffer fills. Once that buffer is full, the server's write call waits. In an async framework, the generator simply isn't asked for the next chunk until there is room.

So a slow reader automatically slows down the producer, instead of data piling up in memory. This only works if you produce lazily: a generator that builds a list of every row first has already lost.

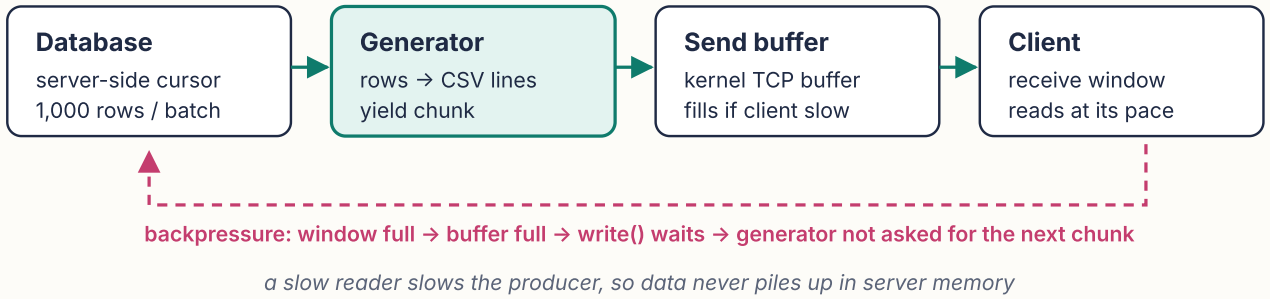


Figure 2. Data flows left to right; pressure flows back right to left. Each stage only pulls the next piece when the stage after it has room.

4 gRPC: streaming as a first-class idea

gRPC runs over HTTP/2 and builds streaming into its contract. A method in the `.proto` file can mark either side as a `stream`, giving four call types:

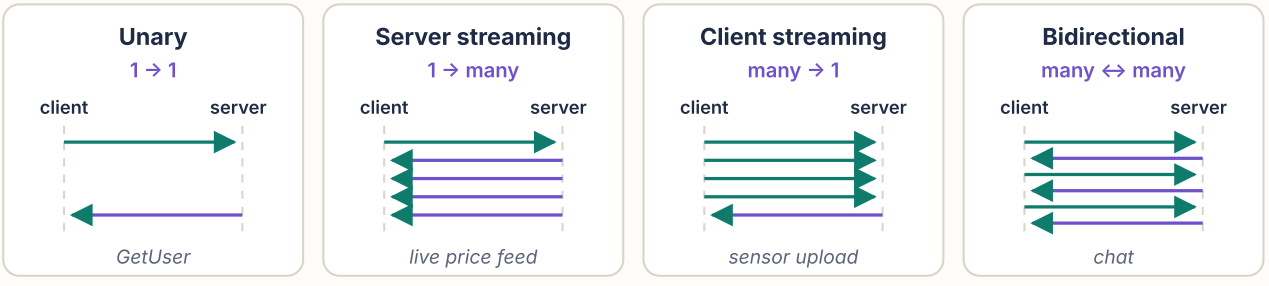


Figure 3. Black arrows are client messages, purple are server messages. In bidirectional mode both sides send independently, at any time.

5 Pitfalls

- **Buffering proxies.** A proxy may hold the whole response before forwarding it. In NGINX set `proxy_buffering off;` or send `X-Accel-Buffering: no`.

- **Errors mid-stream.** The status `200` was sent with the first chunk, so a failure halfway cannot change it. Add an error marker or a row count / checksum at the end. (gRPC sends its status in HTTP/2 trailers, after the data, so it handles this.)
- **Timeouts.** Set idle and total timeouts so a stuck client can't hold a connection open forever.

COMMON MISTAKES

- Using a `StreamingResponse` but fetching all rows first with `fetchall()`: memory is still huge.
- Testing locally (works) and deploying behind a buffering proxy (download starts only at the end).
- Assuming a 200 means the file is complete.

6 Key terms

Chunked transfer: body as size-prefixed chunks

DATA frame: HTTP/2 and 3 unit of body data

Backpressure: slow consumer slows the producer

Content-Length: body size, sent before the body

Server-side cursor: rows fetched in batches

Trailers: headers sent after the body

7 Interview questions

Q1. How do you stream a large HTTP response? BASIC

Send it in pieces as you produce it: `Transfer-Encoding: chunked` in HTTP/1.1, or DATA frames in HTTP/2 and 3. Yield rows lazily from a server-side cursor, so memory stays at one batch, not the whole file.

Q2. What does chunked transfer encoding look like on the wire? BASIC

Each chunk is its size in hex, CRLF, the bytes, CRLF. For example `2000` then 8,192 bytes. A chunk of size `0` ends the body, optionally followed by trailers. No Content-Length is sent.

Q3. What streaming modes does gRPC support? BASIC

Four: unary (1 request, 1 response), server streaming (1 request, many responses), client streaming (many requests, 1 response) and bidirectional streaming (both sides stream at once).

Q4. Streaming works locally, but in production the download starts only after 60 s. Why? SCENARIO

Something in between is buffering the whole response, usually NGINX with `proxy_buffering on` (the default) or a compression layer. Turn buffering off for that route or send `X-Accel-Buffering: no`.

Q5. A 1 Mbit/s client downloads a 500 MB export. Does server memory grow? SCENARIO

No, if the producer is lazy. TCP flow control fills the client's window and the server's send buffer, then the write waits and the generator pauses. But it takes 500 × 8 / 1 ≈ 4,000 s (over an hour), so set a total timeout.

Q6. The database fails at row 3 million of a streamed export. How does the client find out? TRICKY

The status 200 is already sent, so it can't change. Close the connection abnormally (in HTTP/1.1, no final 0 chunk; in HTTP/2, a stream reset) and/or write an explicit error marker, and include a row count or checksum at the end so clients can detect a truncated file.

Q7. Why can't you use Transfer-Encoding: chunked with HTTP/2? TRICKY

HTTP/2 forbids it: the body is already split into DATA frames with their own lengths, and the end-of-stream flag marks the end. Proxies convert between the two.

Next: SD-N-027 · Forward and reverse proxies backendroadmap.com

Forward vs Reverse Proxy

egress control, TLS termination and NGINX

Watch the 4-min video

In one line: a forward proxy acts for clients, controlling and hiding their outbound traffic; a reverse proxy acts for servers, sitting in front of them, hiding the backends and taking work off the app.

1 Two machines in the middle

Two kinds of machines sit in the middle of web traffic, and both are called **proxies**. One works for the clients. The other works for the servers. Mixing them up is a classic interview slip.

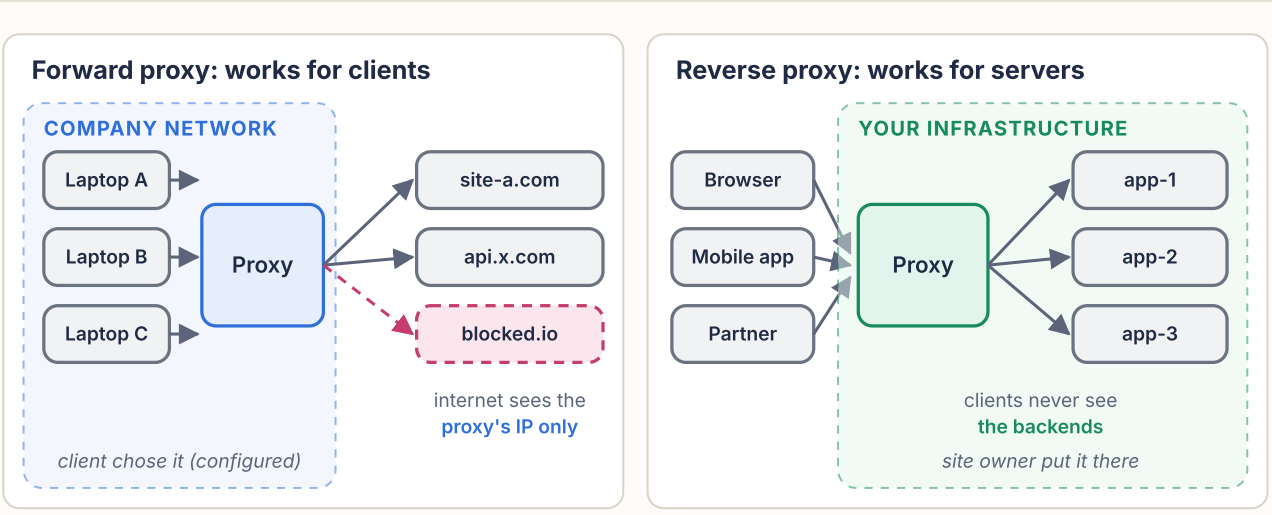


Figure 1. Left: laptops send outbound requests through a forward proxy, which can block sites. Right: every client hits the reverse proxy, which forwards to a backend the client never sees.

2 Forward proxy: egress control for clients

A **forward proxy** sits in front of clients. Laptops inside a company send their outbound requests to the proxy, and the proxy talks to the internet on their behalf. Websites see the proxy's address, not the employee's.

Companies use it for **egress control**: blocking certain sites, enforcing **allow-lists** so servers can only call approved APIs (a payment provider, a package mirror), **logging** who called what, and **caching** shared downloads. Squid is a classic example. In the cloud, an egress proxy or a firewall in front of a **NAT gateway** plays the same role: the NAT gives private servers one public address, and the firewall decides which domains they may reach.

3 Reverse proxy: the front door for servers

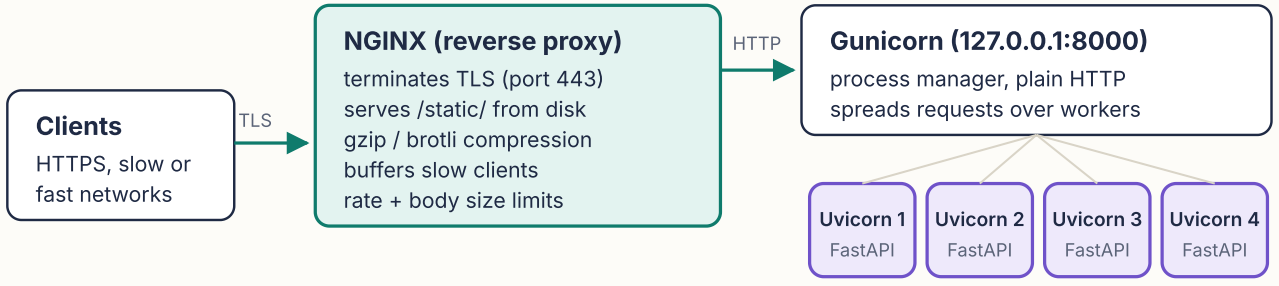
A **reverse proxy** sits in front of servers. Clients think they are talking to your site, but they are really talking to the proxy, which forwards each request to one of your backends. The client never sees the backends.

Ask	Forward proxy	Reverse proxy
Who put it there?	The client side (browser or OS setting, corporate policy)	The site owner; clients don't know it exists
Whose identity does it hide?	Hides the clients from servers	Hides the servers from clients
Typical jobs	Block sites, allow-list APIs, log, cache	TLS, routing, load balancing, caching, compression
Examples	Squid, corporate web proxy, cloud egress firewall	NGINX, HAProxy, Envoy, API gateway, cloud load balancer

A reverse proxy takes a lot of work off app servers:

- **TLS termination**: the proxy holds the certificates and decrypts HTTPS, so apps don't.
- **Routing** by host or path (`api.` to one service, `/images/` to another) and **load balancing** across instances.
- **Caching** static files and responses, and **compression** with gzip or brotli.
- **Protection**: rate limiting, request size limits, and **buffering slow clients**.

4 The classic setup: NGINX, Gunicorn, Uvicorn



Python workers only see fast, complete, plain-HTTP requests for dynamic pages

Figure 2. NGINX faces the internet and does the generic work. It forwards only dynamic requests to Gunicorn on a local port, which spreads them across Uvicorn worker processes running the FastAPI app.

WORKED EXAMPLE: WHY BUFFERING SLOW CLIENTS MATTERS

Your app runs 4 Uvicorn workers. A user on a weak mobile link uploads a 2 MB form at 100 KB/s: that is **20 seconds** of sending. Without a proxy, a worker holds that connection for the whole upload. NGINX reads the full request body first (`client_max_body_size` rejects anything too large), then hands Gunicorn the complete request over localhost in about a millisecond. On the way back, NGINX buffers the response and trickles it to the slow client, so the Python worker is free again almost at once.

Config sketch: `location /static/ { root /srv/app; }` and `location / { proxy_pass http://127.0.0.1:8000; proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for; }`

5 Common reverse proxies

NGINX, **HAProxy** and **Envoy** are the usual choices. Envoy adds features for microservices: retries, circuit breaking, rich metrics, and dynamic configuration, and it is the data plane behind service meshes such as **Istio**. **API gateways** and **cloud load balancers** (AWS ALB, Azure Application Gateway) are reverse proxies too, with extra features such as authentication and API keys on top.

COMMON MISTAKES

- Calling a load balancer a “forward proxy”: it acts for the servers, so it is a reverse proxy.
- Logging the proxy's IP as the client IP: read `X-Forwarded-For`, but only trust it from your own proxy.
- Exposing Gunicorn directly on the internet: no TLS, and slow clients tie up workers.
- Assuming a NAT gateway alone controls egress: it hides addresses but does not filter by domain.

6 Key terms

Forward proxy: acts for clients' outbound traffic

Egress control: limiting what inside systems can reach

X-Forwarded-For: header carrying the original client IP

Reverse proxy: acts for servers, in front of them

TLS termination: decrypt HTTPS at the proxy

Service mesh: sidecar proxies (Envoy) per service

7 Interview questions

Q1. What's the difference between a forward and a reverse proxy? BASIC

A forward proxy acts for clients: they send outbound traffic through it, and it controls and hides them. A reverse proxy acts for servers: it sits in front of them, receives all client traffic and hides the backends. Ask who put it there and whose identity it hides.

Q2. What can a reverse proxy offload from app servers? BASIC

TLS termination, routing by host or path, load balancing, caching, gzip or brotli compression, rate limiting, request size limits, and buffering slow clients.

Q3. Your servers must call only Stripe and your package mirror. How do you enforce it? SCENARIO

Put outbound traffic through an egress proxy or a cloud firewall with an allow-list of domains, block direct internet access in the network rules, and log every request. Deny by default; add domains by review.

Q4. Why put NGINX in front of Gunicorn and Uvicorn? SCENARIO

NGINX terminates TLS, serves static files from disk, compresses, enforces limits and buffers slow clients. With 4 workers, four slow uploads at 20 s each would otherwise block the whole app; with NGINX, each worker is busy for milliseconds.

Q5. After adding a reverse proxy, every request in your logs shows the same client IP. Why? SCENARIO

The app now sees the proxy's address. Have the proxy set `X-Forwarded-For` (or the standard `Forwarded` header) and configure the app to trust it only from the proxy's IP, so clients can't fake it.

Q6. Is a load balancer a forward or a reverse proxy? TRICKY

A reverse proxy. The site owner runs it, clients don't know the backends behind it, and it spreads requests across them. API gateways are reverse proxies too.

Q7. Can a corporate forward proxy read your HTTPS traffic? TRICKY

Normally no. The browser sends `CONNECT host:443` and the proxy only tunnels encrypted bytes, so it sees the hostname, not the content. It can read traffic only with TLS interception: the company installs its own root certificate on the laptops and the proxy re-encrypts.

Next: SD-N-028 · CDNs

CDNs Explained

edge PoPs, cache keys, origin shield and purges

Watch the 5-min video

In one line: a CDN serves your content from edge servers near users, so handshakes and round trips are short and cache hits never reach the origin; after a deploy, change file names instead of purging.

1 The problem: content far from users

Your store's servers are in Virginia. A shopper in Mumbai loads the home page: forty product images and three JavaScript bundles, each crossing the world at around **200 ms per round trip**. The page crawls. A **content delivery network** (CDN) fixes this by bringing your content close to the user.

A CDN is a set of edge servers, called **points of presence** (PoPs), spread across the world. Your own servers are the **origin**. Users connect to the nearest PoP, often found through **anycast** or **DNS**. If that PoP has the file cached, it serves it right there, in a few milliseconds.

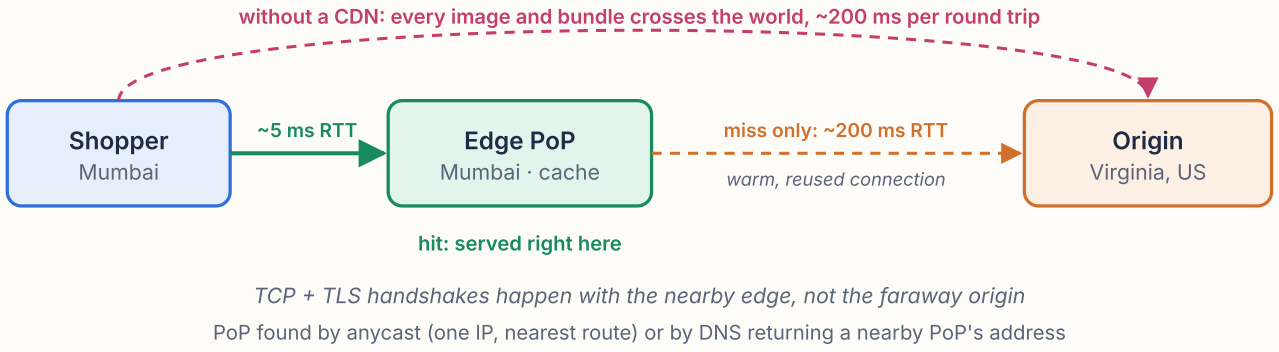


Figure 1. The edge sits a few milliseconds from the user. Only cache misses travel to the origin, over connections the CDN keeps open and warm.

2 How a CDN reduces latency

- **Shorter distance.** The TCP and TLS handshakes happen with a nearby edge, not the faraway origin.
- **Cache hits skip the long trip entirely.**
- **Even misses are faster.** The edge reuses warm, tuned connections back to the origin, so it skips new handshakes.
- **Bonus: offload.** The origin serves far less traffic and survives spikes.

WORKED EXAMPLE

A new HTTPS connection needs about **2 round trips** (TCP, then TLS 1.3) before the first request, and 1 more to get the response. From Mumbai to Virginia: 3 × 200 ms = **600 ms** before the first byte. To a Mumbai PoP at 5 ms: 3 × 5 = **15 ms**, plus a few ms to read from cache. At a **95% hit ratio**, 1 million requests an hour become only **50,000** at the origin (1 in 20).

3 Hit ratio, pull vs push, cache keys and shielding

The key metric is the **cache hit ratio**: the share of requests served from the edge. Most CDNs are **pull CDNs**: on a miss, the edge fetches the file from the origin, caches it and serves later requests itself. **Push CDNs** work the other way: you upload content ahead of time, which suits large, rarely changing files such as video libraries or software downloads.

Each cached object is stored under a **cache key**: usually the URL, plus any headers, cookies or query parameters you choose to include. Include too much, like a session cookie, and every user gets their own copy, so the hit ratio collapses. **Origin shielding** adds a middle cache layer: edge misses go to one shield PoP first, so the origin isn't hit by hundreds of edges at once.

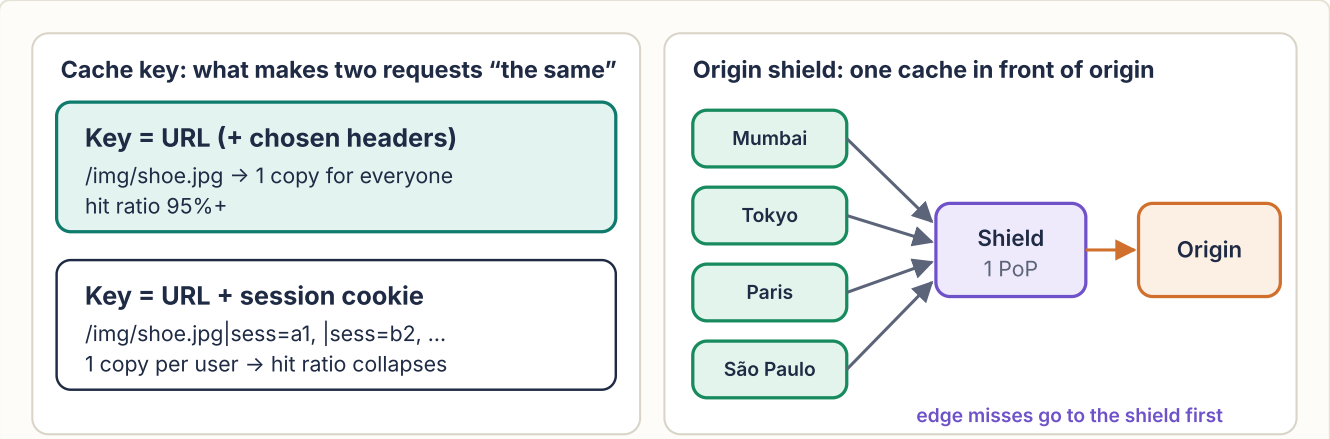


Figure 2. Left: keep the cache key small, or one image becomes thousands of copies. Right: with a shield, a new file costs the origin one request, not one per PoP.

4 Example: images and bundles on CloudFront or Azure Front Door

The build outputs files with a **content hash** in the name, like `app.3f9a.js`. They are served with `Cache-Control: public, max-age=31536000, immutable` (one year, and the browser need not even revalidate). The HTML page itself gets a **short TTL**, such as `max-age=60`, and after each deploy simply points to the new file names.

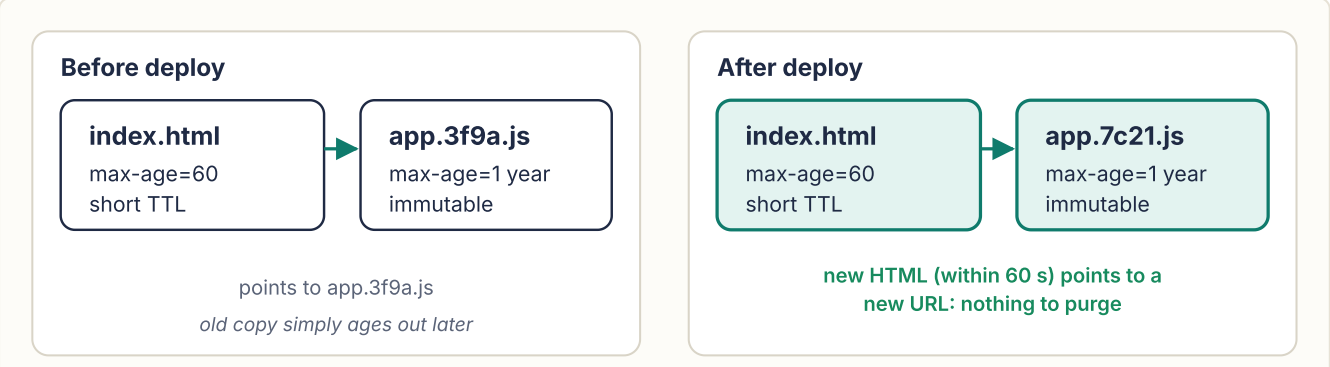


Figure 3. The content hash changes when the file changes, so a new version is a new URL. Only the short-lived HTML has to update.

5 Invalidating after a deploy

The best answer is: **you don't**. Versioned file names mean new content gets a new URL, and old copies simply age out. For things that must keep the same URL, like the HTML or an API response, use **short TTLs**, or issue a **purge**: an invalidation request for specific paths. Purges take time to spread to every PoP (seconds to minutes), and some CDNs charge per path (CloudFront, beyond a monthly free allowance), so they are the exception, not the plan.

COMMON MISTAKES

- Long TTLs on `app.js` without a hash in the name: users keep old code for days.
- Putting cookies or tracking query strings (`?utm_source=…`) in the cache key.
- Caching personalised API responses as `public`: one user's data shown to another.
- Relying on a purge in every deploy script.

6 Key terms

PoP: an edge location with cache servers

Cache hit ratio: share of requests served at the edge

Origin shield: middle cache in front of the origin

Origin: your own servers, the source of truth

Cache key: what identifies one cached object

Purge: invalidate specific cached paths

7 Interview questions

Q1. How does a CDN reduce latency? BASIC

It serves content from edge PoPs near users: handshakes and round trips are short (5 ms instead of 200 ms), cache hits skip the origin entirely, and misses reuse warm edge-to-origin connections.

Q2. Pull CDN vs push CDN? BASIC

Pull: the edge fetches from the origin on the first miss and caches it; easy, and the default for websites. Push: you upload content to the CDN ahead of time; good for large, rarely changing files like video libraries and downloads.

Q3. What is the cache hit ratio, and why does it matter? BASIC

The share of requests served from the edge. At 95%, the origin sees 1 request in 20; at 80%, 1 in 5, which is 4× more origin load. It drives both latency and origin cost.

Q4. How do you invalidate CDN content after a deploy? SCENARIO

Prefer versioned (content-hashed) file names with `max-age=31536000, immutable`, and keep the HTML short-lived (e.g. 60 s) so it points to new names. Purge only the few paths that must change in place.

Q5. Your hit ratio dropped from 95% to 30% after a marketing launch. What do you check? SCENARIO

The cache key. Likely new query parameters (`utm_*`) or a cookie are now part of it, so each visitor creates a new copy. Strip or ignore them in the key, and check `Vary` headers.

Q6. A new 50 MB video goes live and 300 PoPs miss at once. What protects the origin? SCENARIO

Origin shielding: all edge misses go to one shield PoP, which fetches the file once. The CDN also collapses concurrent misses for the same object into one origin request.

Q7. Can you put an API behind a CDN? TRICKY

Yes. Even uncached, it gains nearby TLS termination and warm connections. Cache only public, non-personalised GET responses with short TTLs; send `private` or `no-store` for per-user data.

Q8. Why is the HTML not given the same one-year TTL as the bundles? TRICKY

The HTML's URL never changes, so a long TTL would pin users to the old page, which points to old bundles. It is the entry point that must update, so it gets a short TTL or revalidation.

Next: SD-N-029 · Connection pooling and keep-alive backendroadmap.com

Connection Pooling & Keep-Alive

pool sizing, leaks and idle timeouts

Watch the 4-min video

In one line: opening a connection costs DNS, TCP and TLS round trips, so keep connections open and share them through a pool; size it with Little's Law, and watch idle timeouts and leaks.

1 Why new connections are expensive

Your FastAPI service calls a payments API on every request. Under load, p99 latency jumps and connection errors appear. The culprit is often one line: **creating a new HTTP client inside every request handler**.

Every new HTTPS connection has a setup cost: a **DNS lookup**, a **TCP handshake** and a **TLS handshake**. That is two or three round trips before the first byte of your request. At 50 ms per round trip, that's over **100 ms of pure overhead**, plus CPU for the cryptography on both sides.

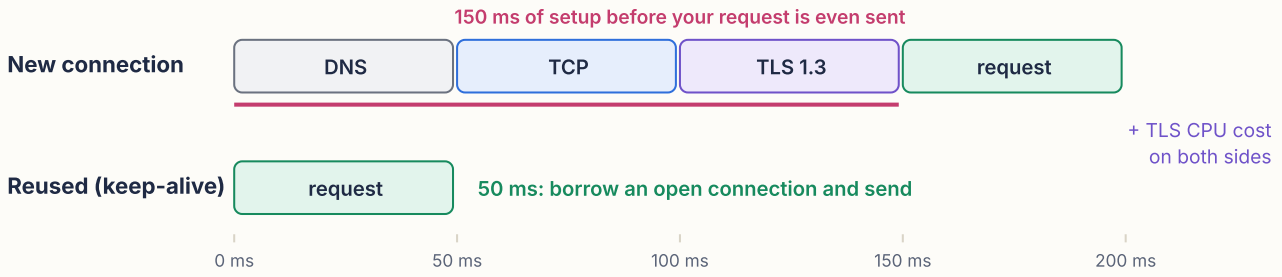


Figure 1. At 50 ms RTT a fresh connection spends ~150 ms on setup (DNS may be cached; TLS 1.2 adds a round trip). A reused one goes straight to the request.

2 Keep-alive and the connection pool

Keep-alive leaves the connection open after a response, so the next request can reuse it (the default in HTTP/1.1; HTTP/2 goes further and runs many requests over one connection). A **connection pool** manages a set of these open connections: you **borrow** an idle one, use it, and **give it back**. In Python that is `requests.Session` or an `httpx` client; database drivers and SQLAlchemy have their own pools. Without one, every call pays the full handshake again.

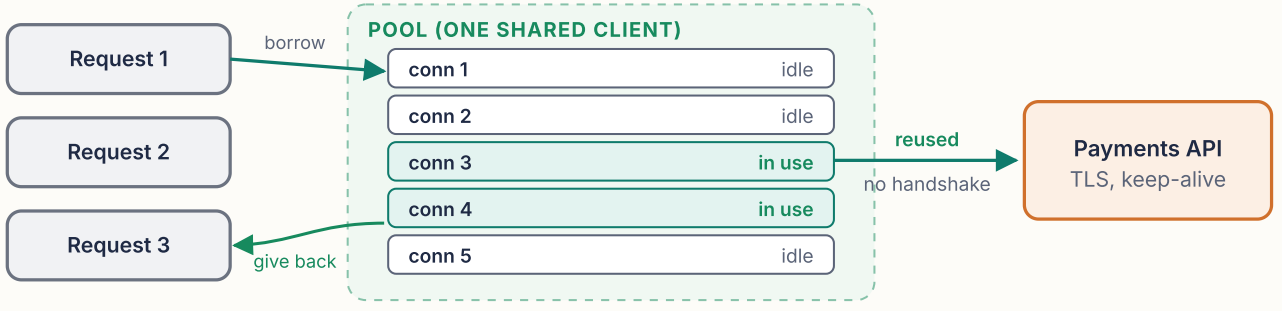


Figure 2. Requests borrow an idle connection, use it and return it. The pool keeps connections open, so calls skip the handshake.

Example: one shared client in FastAPI

Bad: a new `AsyncClient` inside the handler opens and tears down fresh connections on every request.

```
@app.get("/pay")
async def pay():
    async with httpx.AsyncClient() as c:   # new pool per request
        return (await c.post(URL, json=body)).json()
```

Fix: create one shared client at startup in FastAPI's `lifespan`, reuse it in every handler, and close it on shutdown.

```
@asynccontextmanager
async def lifespan(app):
    app.state.http = httpx.AsyncClient(timeout=5.0,
        limits=httpx.Limits(max_connections=30))
    yield
    await app.state.http.aclose()           # close on shutdown
app = FastAPI(lifespan=lifespan)
@app.get("/pay")
async def pay(request: Request):
    r = await request.app.state.http.post(URL, json=body)   # reused
    return r.json()
```

3 How big should the pool be?

Use **Little's Law**: connections needed = request rate × call duration.

```
200 calls/s × 0.1 s = 20 connections  (+ headroom, per worker or pod)
```

Too small, and requests queue for a free connection: latency climbs, then pool timeout errors appear, even though the downstream is perfectly healthy. **Too large**, and you overwhelm the downstream; big pools also waste memory and file descriptors.

WORKED EXAMPLE

Your service makes 200 payment calls per second, each taking 100 ms: about **20** connections busy at once. With ~50% headroom, aim for 30 in total. If that traffic is spread over 3 pods with 2 workers each (6 pools), each pool needs about 30 / 6 = **5**. Now a database with `max_connections = 100`, and 10 pods each holding a pool of 50 means up to **500**. The database starts refusing connections. Size each pool so the total across all pods fits, or put a pooler such as PgBouncer in front.

4 Two more traps: idle timeouts and leaks

Idle timeouts. If the server or load balancer closes idle connections after 60 s, but your pool keeps them for 90 s, you will reuse dead connections and see random resets. Keep the client's idle timeout **shorter** than the server's.

Leaks. If responses or streams aren't closed, their connections never return to the pool, until it is exhausted and every call waits for a pool timeout. Always use context managers (`async with client.stream(...)`, `with Session() as s`) so connections go back even on errors.

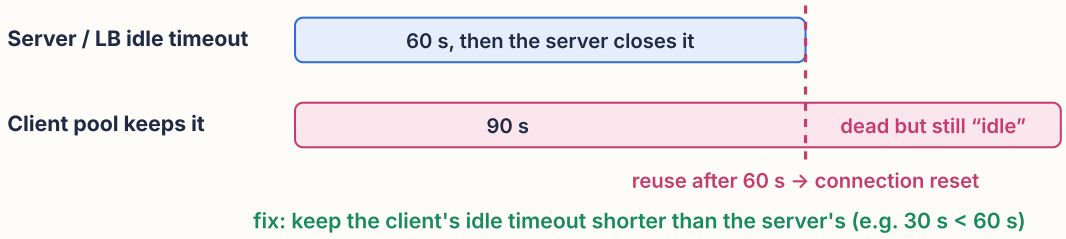


Figure 3. The server closed the connection at 60 s, but the client still thinks it is idle and usable. The next request on it fails.

COMMON MISTAKES

- A new client (or a new database engine) per request.
- Sizing each pod's pool on its own and forgetting to multiply by pods and workers.
- Raising the pool when the real problem is a leak or a slow downstream.

5 Key terms

Keep-alive: leave a connection open for reuse

Pool timeout: max wait to borrow a connection

Connection leak: borrowed, never returned

Connection pool: set of open connections to borrow

Idle timeout: close a connection unused this long

PgBouncer: external pooler in front of Postgres

6 Interview questions

Q1. Why reuse connections? BASIC

To skip DNS, TCP and TLS setup on every call. At 50 ms RTT that is about 100–150 ms and real CPU per call saved, which cuts latency and load on both sides.

Q2. What goes wrong when a pool is too small or too large? BASIC

Too small: requests queue for a connection and hit pool timeouts while the downstream is healthy. Too large: you exceed downstream limits like database max connections, and waste memory and file descriptors.

Q3. p99 jumped after traffic doubled, and logs show pool timeouts, but the payments API is healthy. What happened? SCENARIO

In flight doubled (say 20 → 40) and the pool stayed at 20, so requests wait for a connection. Raise the pool to rate × duration + headroom, and check for leaks (connections not returned).

Q4. You see random `connection reset` errors, mostly after quiet periods. Cause? SCENARIO

The pool reuses connections the server or load balancer already closed as idle (say at 60 s) while the client keeps them for 90 s. Set the client's keepalive expiry below the server's, e.g. 30 s.

Q5. Does HTTP/2 remove the need for a pool? TRICKY

Mostly for HTTP: one HTTP/2 connection multiplexes many concurrent requests, so you need few connections. You still reuse them, and database connections still need a pool.

Q6. The pool is exhausted at low traffic. Why? TRICKY

A leak: responses or streams not closed. Connections never return, so a small rate still drains it. Use context managers and watch the in-use count.

Next: SD-N-030 · Timeouts: connect, read, deadlines and the random 502 backendroadmap.com

Network Timeouts

connect, read, deadlines and the random 502

Watch the 4-min video

In one line: every outbound call needs a connect, read and pool timeout plus an overall deadline; pass the remaining budget downstream, make inner timeouts shorter than outer ones, and keep the app's keep-alive longer than the load balancer's.

1 Why timeouts matter

A downstream service hangs. It doesn't fail, it just stops answering. Without timeouts, every request that calls it **waits forever**. Threads pile up, connection pools drain, and soon your whole service is down too. Timeouts stop one slow dependency from taking everything else with it.

2 The kinds of timeout

Timeout	Limits how long you wait to...	Typical value
Connect	establish the TCP and TLS connection	1–2 s
Read	receive the *next bytes* of the response (not the whole response)	a little above the dependency's p99
Write	send a large request body	a few seconds
Pool	borrow a connection from the pool	short, e.g. 0.5–1 s
Idle	keep an unused connection open before closing it	shorter than the server's

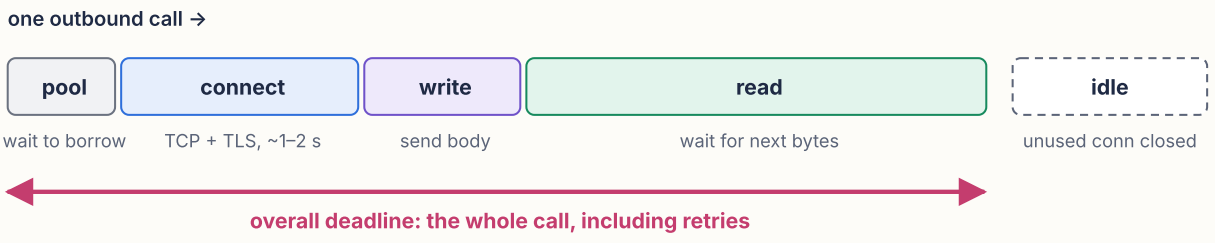
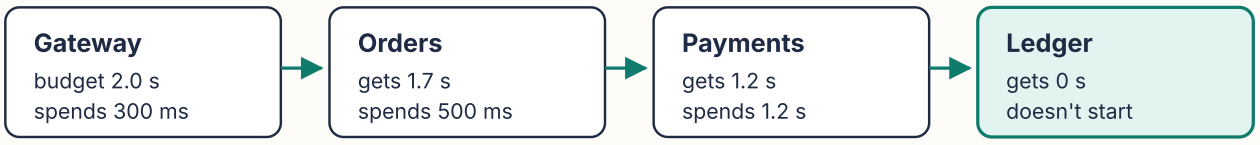


Figure 1. Each phase of a call has its own timeout. The overall deadline caps the sum, including any retries.

3 Per-operation timeouts are not enough

A read timeout of 5 seconds still allows a response that trickles in one byte every 4 seconds, forever: each read finishes in time. So also set a **total deadline** for the whole call, including retries. If the user-facing request has 2 seconds, no single call inside it should be allowed to take 10.

In **microservices, propagate the deadline**. The gateway receives a request with a 2-second budget, spends 300 ms, and passes the remaining 1.7 s to the next service, and so on. gRPC does this natively with deadlines; with HTTP you pass a header. If the budget is already spent, the downstream service doesn't even start the work.



pass the remaining budget down: gRPC deadlines natively, a header (e.g. X-Request-Deadline) over HTTP

if the budget is already spent, the downstream service returns at once instead of doing useless work

Figure 2. Each hop subtracts what it used. By the time the budget reaches zero, nobody downstream wastes effort on an answer no one will wait for.

4 Shrink timeouts inward, and the classic 502

Timeouts should get **shorter as you go deeper**. If the load balancer allows 60 s, the app's request timeout might be 55 s, and the database query timeout 50 s. Then the innermost call fails first with a clear error, and the caller still has time to retry or return a fallback, instead of the outer layer cutting the connection mid-work.

Now the classic bug. The load balancer has a **60-second idle timeout**, but the app server's keep-alive timeout is only **5 seconds** (Uvicorn's default). The app quietly closes an idle connection at the same moment the load balancer reuses it to send a request. The result: random **502 Bad Gateway** errors, often under *light* traffic, when connections sit idle. The fix: make the app's keep-alive longer than the load balancer's, say **75 s**, so the load balancer always closes first.

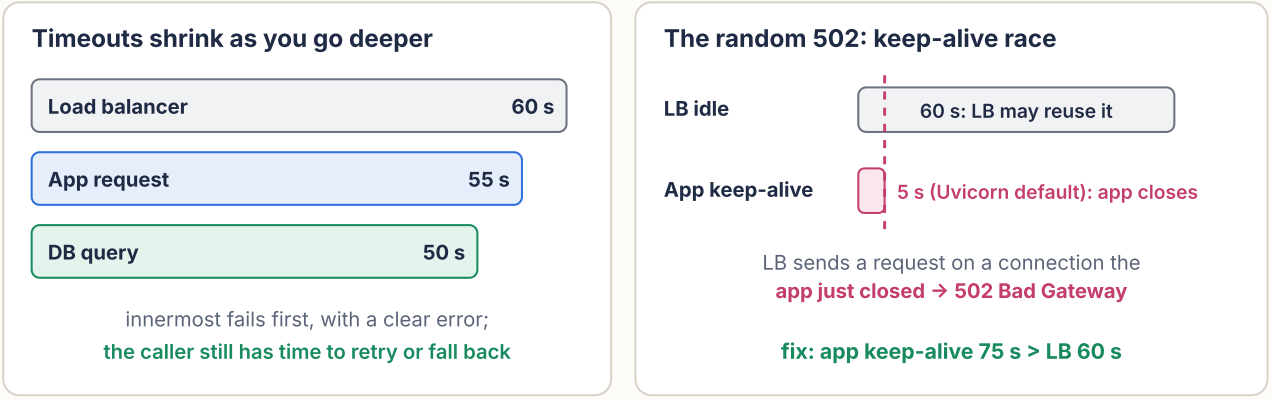


Figure 3. Left: outer layers wait longer than inner ones. Right: when the app closes idle connections before the load balancer does, they race and the loser is a 502.

WORKED EXAMPLE: PRACTICAL DEFAULTS

A payments API has a p99 of 400 ms; your endpoint promises 2 s. Use connect 1 s, read 1 s, pool 0.5 s, and an overall deadline of 1.5 s, leaving 0.5 s for your own work. That allows one retry with backoff, and only because the call carries an idempotency key. In httpx: `httpx.Timeout(1.0, connect=1.0, pool=0.5)` plus `asyncio.timeout(1.5)` around the call. Behind a 60 s load balancer, run `uvicorn --timeout-keep-alive 75`.

Base timeouts on the dependency's **real p99 latency**, not on guesses. Add retries with backoff **only when the operation is idempotent**. And never trust the library default, which is sometimes **no timeout at all**, as with Python's `requests`.

COMMON MISTAKES

- Calling `requests.get(url)` with no `timeout=`: it can wait forever.
- Retries without a total deadline: 3 tries × 5 s read timeout turns a 2 s budget into 15 s.
- An inner timeout longer than the outer one, so the outer layer cuts the work off with no clear error.
- App keep-alive shorter than the load balancer's idle timeout.

5 Key terms

Connect timeout: max time to set up TCP + TLS

Pool timeout: max wait to borrow a connection

Deadline propagation: pass the remaining budget downstream

Read timeout: max gap waiting for the next bytes

Deadline: absolute time budget for the whole call

502 Bad Gateway: proxy got no valid upstream response

6 Interview questions

Q1. What timeouts should every outbound call have? BASIC

A connect timeout (1–2 s), a read timeout (above the dependency's p99), a pool or acquire timeout, and an overall deadline that covers retries.

Q2. Why isn't a read timeout enough on its own? BASIC

It limits the gap between bytes, not the whole response. A server trickling one byte every 4 s never trips a 5 s read timeout, so add a total deadline.

Q3. A dependency hangs and your whole service goes down. Why, and how do you prevent it? SCENARIO

Without timeouts, each request waits forever, holding a thread and a connection; pools drain and everything queues. Set timeouts and a deadline on every call, and add a circuit breaker to fail fast while it is down.

Q4. Your gateway has a 2 s budget and calls three services in a chain. How do you set their timeouts? SCENARIO

Propagate the deadline: each hop gets what's left (2.0 → 1.7 → 1.2 s...), via gRPC deadlines or a header. A service that receives an already-expired budget returns at once without starting work.

Q5. You see random 502s from the load balancer, mostly at night with little traffic. Cause? SCENARIO

A keep-alive race: the LB keeps idle connections for 60 s, the app (Uvicorn's default 5 s) closes them sooner, and the LB reuses one the app just closed. Set the app's keep-alive above the LB's, e.g. 75 s.

Q6. Why should a client timeout be shorter than the server's? TRICKY

So the inner side gives up first. A client that closes idle connections before the server never reuses a dead one, and inner calls fail with a clear error while the caller still has time to retry or fall back.

Q7. Is it safe to retry every timed-out call? TRICKY

No. A timeout doesn't tell you whether the server did the work. Retry only idempotent operations (GET, or POST with an idempotency key), with backoff and jitter, and inside the overall deadline.

Next: SD-C-001 · REST Principles (API Design) backendroadmap.com